

**University of Stuttgart**  
Germany  
Institute of Industrial Automation and  
Software Engineering

# Probabilistic Action Prediction of Human Drivers Using Dynamic Bayesian Networks

**Master Thesis 1111**

**Submitted at the University of Stuttgart by  
Amal Chulliyat Jose**

**Electrical Engineering**

**Examiner: Prof. Dr.-Ing. Andrey Morozov**

**Supervisor: Georgios Katranis, M.Sc.**

**15 April 2026**



# Contents

|                                                          |             |
|----------------------------------------------------------|-------------|
| <b>List of Figures</b>                                   | <b>v</b>    |
| <b>List of Tables</b>                                    | <b>vi</b>   |
| <b>List of Abbreviations</b>                             | <b>vii</b>  |
| <b>Glossary</b>                                          | <b>xi</b>   |
| <b>Kurzfassung</b>                                       | <b>xiii</b> |
| <b>Abstract</b>                                          | <b>xv</b>   |
| <b>1. Introduction</b>                                   | <b>1</b>    |
| <b>2. Theoretical Background</b>                         | <b>3</b>    |
| 2.1. Probability Theory . . . . .                        | 3           |
| 2.1.1. Bayes' Theorem . . . . .                          | 4           |
| 2.2. Probabilistic Graphical Models . . . . .            | 4           |
| 2.2.1. Bayesian Networks . . . . .                       | 5           |
| 2.2.2. Dynamic Bayesian Networks . . . . .               | 6           |
| 2.3. Inference in Chain-Structured DBNs . . . . .        | 6           |
| 2.3.1. Forward-Backward Algorithm . . . . .              | 7           |
| 2.4. Parameter Learning in Latent-State Models . . . . . | 9           |
| 2.4.1. Expectation-Maximization Algorithm . . . . .      | 9           |
| <b>3. State of the Art</b>                               | <b>11</b>   |
| 3.1. Introduction . . . . .                              | 11          |
| 3.2. Rule-Based Models . . . . .                         | 11          |
| 3.2.1. Car-Following Models . . . . .                    | 11          |
| 3.2.2. Lane-Change Models . . . . .                      | 12          |
| 3.3. Probabilistic Models . . . . .                      | 12          |
| 3.3.1. Hidden Markov Models . . . . .                    | 13          |
| 3.3.2. Dynamic Bayesian Networks . . . . .               | 13          |
| 3.3.3. Gaussian-Based Probabilistic Models . . . . .     | 13          |
| 3.4. Learning-Based Models . . . . .                     | 14          |
| 3.4.1. Classical Machine Learning Models . . . . .       | 14          |
| 3.4.2. Recurrent Neural Network-Based Models . . . . .   | 14          |

## Contents

|           |                                                                                         |           |
|-----------|-----------------------------------------------------------------------------------------|-----------|
| 3.4.3.    | Graph Neural Networks . . . . .                                                         | 15        |
| 3.4.4.    | Transformers . . . . .                                                                  | 15        |
| 3.4.5.    | Conditional Variational Autoencoders . . . . .                                          | 16        |
| 3.4.6.    | Diffusion Models . . . . .                                                              | 17        |
| 3.4.7.    | Other Learning-Based Behavior Modeling Approaches . . . . .                             | 17        |
| 3.5.      | Frameworks for Autonomous Decision Making . . . . .                                     | 18        |
| 3.5.1.    | Game-Theoretic Models for Mixed-Traffic Interactions . . . . .                          | 18        |
| 3.5.2.    | Partially Observable Markov Decision Process . . . . .                                  | 18        |
| 3.5.3.    | Risk Modeling in Autonomous Decision Making . . . . .                                   | 18        |
| 3.6.      | Comparative Assessment of Approaches for HDV Behavior Modeling and Prediction . . . . . | 19        |
| <b>4.</b> | <b>Methodology</b>                                                                      | <b>21</b> |
| 4.1.      | Proposed Framework . . . . .                                                            | 21        |
| 4.2.      | Dataset . . . . .                                                                       | 22        |
| 4.3.      | Data Preparation and Preprocessing . . . . .                                            | 23        |
| 4.3.1.    | Preprocessing Pipeline . . . . .                                                        | 23        |
| 4.3.2.    | Data Analysis for Feature Design . . . . .                                              | 24        |
| 4.3.3.    | Feature Engineering . . . . .                                                           | 27        |
| 4.3.4.    | Window-Based Representation . . . . .                                                   | 28        |
| 4.4.      | Proposed DBN Model . . . . .                                                            | 29        |
| 4.4.1.    | Latent State Structure . . . . .                                                        | 29        |
| 4.4.2.    | Transition Model . . . . .                                                              | 30        |
| 4.4.3.    | Emission Model Variants . . . . .                                                       | 30        |
| 4.4.4.    | Modeling Assumptions . . . . .                                                          | 31        |
| 4.5.      | Model Training . . . . .                                                                | 32        |
| 4.5.1.    | Feature Scaling . . . . .                                                               | 32        |
| 4.5.2.    | Parameter Initialization . . . . .                                                      | 33        |
| 4.5.3.    | EM-Based Training . . . . .                                                             | 33        |
| 4.6.      | Semantic Interpretation of Latent States . . . . .                                      | 36        |
| 4.6.1.    | Feature-Based Semantic Profiles . . . . .                                               | 36        |
| 4.6.2.    | Rule-Based Labeling Procedure . . . . .                                                 | 37        |
| 4.7.      | Inference and Evaluation . . . . .                                                      | 37        |
| 4.7.1.    | Generative Evaluation Metrics . . . . .                                                 | 39        |
| 4.7.2.    | Predictive Evaluation Metrics . . . . .                                                 | 41        |
| 4.7.3.    | Posterior-Based Interpretation of Individual Predictions . . . . .                      | 43        |
| 4.7.4.    | Computational Efficiency . . . . .                                                      | 43        |
| <b>5.</b> | <b>Results and Discussion</b>                                                           | <b>44</b> |
| 5.1.      | Main S2A4 Configuration Study . . . . .                                                 | 44        |
| 5.1.1.    | Choice of CPD Initialization . . . . .                                                  | 44        |
| 5.1.2.    | Compared Configurations . . . . .                                                       | 45        |

## Contents

|           |                                                                |           |
|-----------|----------------------------------------------------------------|-----------|
| 5.1.3.    | Overview of S2A4 Comparison Results . . . . .                  | 45        |
| 5.1.4.    | PoE versus Hierarchical Emissions . . . . .                    | 46        |
| 5.1.5.    | Effect of Bernoulli Likelihood Terms . . . . .                 | 47        |
| 5.1.6.    | Effect of Learning the Initial Distribution . . . . .          | 47        |
| 5.1.7.    | Effect of Lane-Change Weighting . . . . .                      | 48        |
| 5.1.8.    | Summary of Selected Core Model . . . . .                       | 49        |
| 5.1.9.    | Semantic Interpretation of the Learned Latent States . . . . . | 49        |
| 5.1.10.   | Predictive Performance of the Selected Model . . . . .         | 51        |
| 5.1.11.   | Computational Efficiency . . . . .                             | 56        |
| 5.2.      | Ablation Studies on Latent-State Cardinality . . . . .         | 57        |
| 5.2.1.    | Ablation with 2 Style States and 3 Action States . . . . .     | 58        |
| 5.2.2.    | Ablation with 2 Style States and 5 Action States . . . . .     | 59        |
| 5.2.3.    | Ablation with 1 Style State and 4 Action States . . . . .      | 59        |
| 5.2.4.    | Ablation with 1 Style State and 5 Action States . . . . .      | 60        |
| <b>6.</b> | <b>Conclusions</b>                                             | <b>61</b> |
| 6.1.      | Future Work . . . . .                                          | 61        |
|           | <b>Bibliography</b>                                            | <b>62</b> |
| <b>A.</b> | <b>Direction-Wise Sign Convention in Raw Kinematics</b>        | <b>68</b> |
| <b>B.</b> | <b>Class-Wise Distribution Plots</b>                           | <b>70</b> |
| <b>C.</b> | <b>Complete Window-Level Observation Features</b>              | <b>74</b> |
| <b>D.</b> | <b>Semantic Mapping Rules</b>                                  | <b>77</b> |
| D.1.      | Threshold Values . . . . .                                     | 77        |
| D.2.      | Priority Order . . . . .                                       | 78        |
| D.3.      | Composite Definitions . . . . .                                | 78        |
| <b>E.</b> | <b>Emission Model Derivations</b>                              | <b>80</b> |
| E.0.1.    | EM Objective . . . . .                                         | 80        |
| E.1.      | Hierarchical Emission Model . . . . .                          | 83        |
| E.1.1.    | Parameter Updates . . . . .                                    | 84        |
| E.2.      | Product-of-Experts Emission Model . . . . .                    | 85        |
| E.2.1.    | Implication for the M-Step . . . . .                           | 87        |
| E.2.2.    | Gradient Form . . . . .                                        | 88        |

# List of Figures

|                                                                                                     |    |
|-----------------------------------------------------------------------------------------------------|----|
| 2.1. Example of a Bayesian Network (BN). . . . .                                                    | 5  |
| 2.2. Two-slice representation of a Dynamic Bayesian Network (DBN). . . . .                          | 6  |
| 4.1. Overview of the proposed framework . . . . .                                                   | 22 |
| 4.2. Example traffic scene from a selected highD subset . . . . .                                   | 23 |
| 4.3. Global distributions of the main kinematic variables. . . . .                                  | 24 |
| 4.4. Lane-change sparsity analysis in the highD data. . . . .                                       | 25 |
| 4.5. Global trajectory-level variability distributions of the main kinematic variables. . . . .     | 26 |
| 4.6. Selected neighbors considered in the observation . . . . .                                     | 27 |
| 4.7. Illustration of the sliding-window representation used in this work. . . . .                   | 29 |
| 4.8. Factorized latent structure of the proposed DBN. . . . .                                       | 29 |
| 4.9. Emission model variants considered in the proposed DBN. . . . .                                | 31 |
| 4.10. Causal online prediction and evaluation procedure. . . . .                                    | 38 |
| 5.1. Comparison of sticky vs. uniform CPD initialization . . . . .                                  | 45 |
| 5.2. Learned transition structure of the selected core model . . . . .                              | 50 |
| 5.3. Confusion matrix for exact one-step prediction . . . . .                                       | 51 |
| 5.4. Row and column-normalized confusion matrices . . . . .                                         | 52 |
| 5.5. Grouped confusion matrix for exact one-step prediction . . . . .                               | 54 |
| 5.6. Ungrouped horizon prediction results . . . . .                                                 | 55 |
| 5.7. Grouped horizon prediction results . . . . .                                                   | 55 |
| 5.8. Validation log-likelihood for two-style ablations . . . . .                                    | 57 |
| 5.9. Validation log-likelihood for one-style ablations . . . . .                                    | 58 |
| A.1. Direction-wise sign difference in raw longitudinal velocity. . . . .                           | 68 |
| A.2. Class-wise raw longitudinal velocity distributions. . . . .                                    | 69 |
| B.1. Class-wise frame-level distributions of the main kinematic variables. . . . .                  | 70 |
| B.2. Class-wise trajectory-level variability distributions of the main kinematic variables. . . . . | 72 |

# List of Tables

|                                                                                                    |    |
|----------------------------------------------------------------------------------------------------|----|
| 4.1. Frame-level feature groups used in the preprocessing pipeline . . . . .                       | 28 |
| 4.2. Lane-change weighting settings used in the experiments. . . . .                               | 35 |
| 5.1. Overview of the 24 compared configurations in the main S2A4 study. . .                        | 46 |
| 5.2. Class-wise precision, recall, and F1-score for exact one-step joint-state prediction. . . . . | 53 |
| 5.3. Mean steady-state latency of the selected model for different settings. . .                   | 56 |
| C.1. Window-level ego-motion features. . . . .                                                     | 74 |
| C.2. Window-level lane-change and lane-geometry features. . . . .                                  | 74 |
| C.3. Window-level front-risk features. . . . .                                                     | 75 |
| C.4. Window-level features related to the front neighbor. . . . .                                  | 75 |
| C.5. Window-level features related to the side neighbors. . . . .                                  | 75 |
| C.6. Window-level features related to the adjacent front neighbors. . . . .                        | 76 |
| D.1. Threshold values used by the semantic mapping rules. . . . .                                  | 77 |

# List of Abbreviations

**ADAS** Advanced Driver Assistance Systems

**AI** Artificial Intelligence

**AV** Autonomous Vehicle

**BIC** Bayesian Information Criterion

**BN** Bayesian Network

**CPD** Conditional Probability Distribution

**CPU** Central Processing Unit

**CVAE** Conditional Variational Autoencoder

**DAG** Directed Acyclic Graph

**DBN** Dynamic Bayesian Network

**DL** Deep Learning

**EM** Expectation-Maximization

**FVDM** Full Velocity Difference Model

## *List of Abbreviations*

**GCN** Graph Convolutional Network

**GMM** Gaussian Mixture Model

**GMR** Gaussian Mixture Regression

**GNN** Graph Neural Network

**GPR** Gaussian Process Regression

**HDV** Human-Driven Vehicle

**HiVT** Hierarchical Vector Transformer

**HMM** Hidden Markov Model

**IDM** Intelligent Driver Model

**IRL** Inverse Reinforcement Learning

**KI** Künstliche Intelligenz

**kNN** k-Nearest Neighbor

**LSTM** Long Short-Term Memory

**MAP** Maximum A Posteriori

**MaxEnt IRL** Maximum Entropy Inverse Reinforcement Learning

**MLE** Maximum Likelihood Estimation



## *List of Abbreviations*

**MOBIL** Minimizing Overall Braking Induced by Lane Changes

**MTR** Motion Transformer

**NLL** Negative Log Likelihood

**NN** Neural Network

**OVN** Optimal Velocity Model

**PCA** Principal Component Analysis

**PECNet** Predicted Endpoint Conditioned Network

**PoE** Product of Experts

**POMDP** Partially Observable Markov Decision Process

**RNN** Recurrent Neural Network

**RV** Random Variable

**SVM** Support Vector Machine

**T-GNN** Temporal Graph Neural Network

**THW** Time Headway

**TTC** Time to Collision

**TTE** Time To Event

## *List of Abbreviations*

**V2X** Vehicle-to-Everything

**VAE** Variational Autoencoder

# Glossary

**Autonomous Vehicle (AV)** A vehicle that can drive by itself without human control. It uses sensors and artificial intelligence to understand the surrounding environment and make driving decisions.

**Belief State** The probability distribution over the latent states given the observed data at a specific time step.

**Confusion Matrix** A matrix showing the performance of a classification model, across the true and predicted classes.

**Directed Acyclic Graph (DAG)** A graph made up of nodes and directed edges between them, where the edges do not form any closed loops.

**DBN** A probabilistic graphical model that uses a directed acyclic graph to represent time-series data, where each node represents a random variable and the edges show how these variables depend on each other within and across time steps.

**Emission Model** The probability distribution that describes how observations are generated from latent states.

**Entropy** A measure of uncertainty in a probability distribution. It also quantifies the amount of information needed to describe the distribution.

**F1 Score** The harmonic mean of precision and recall.

**Human-Driven Vehicle (HDV)** A vehicle that is controlled by a human driver who performs all driving tasks such as steering, accelerating, and braking.

**Inference** The process of estimating the probability distribution of latent states or unknown variables from observed data.

**Latent State** A variable that is not directly observed but is inferred from data.

**Likelihood** The probability of the observed data under a given model.

**Maximum A Posteriori (MAP)** The estimate of the most likely latent state, given the observed data. It is the state that maximizes the posterior distribution.

**Mixed Traffic** A traffic situation where both autonomous vehicles and human-driven vehicles operate together.

## *Glossary*

**Observation** The data that is directly measured from the system, represented as feature vectors.

**Posterior** The probability distribution over the latent states after observing the data.

**Precision** The ratio of true positive predictions to the total predicted positives. It indicates the accuracy of positive predictions.

**Prior** The probability distribution over the latent states before observing any data.

**Recall** The ratio of true positive predictions to the total actual positives. It indicates the completeness of positive predictions.

**Random Variable (RV)** A variable that can take on different values, each with an associated probability.

**State Transition** Probability of transitioning from one latent state to another.

**Time Headway (THW)** The time needed for a vehicle to reach the current position of the vehicle ahead if it maintains its current velocity.

**Time to Collision (TTC)** The time needed for a vehicle to collide with the vehicle ahead if both of them maintain their current velocity.

**Trajectory** The time-ordered sequence describing the movement of a vehicle over time.

# Kurzfassung

Künstliche Intelligenz (KI) verändert autonome Systeme durch bedeutende Fortschritte in den Bereichen Sensorik, Kommunikation und Rechenleistung. Obwohl vollständig autonome Fahrzeuge (Autonomous Vehicles (AVs)) voraussichtlich immer häufiger werden, werden sie auf absehbare Zeit gemeinsam mit von Menschen gesteuerten Fahrzeugen (Human-Driven Vehicles (HDVs)) am Straßenverkehr teilnehmen. Um in diesem gemischten Verkehrsumfeld sicher zu operieren, müssen autonome Systeme in der Lage sein, menschliche Fahrhandlungen zu modellieren und vorherzusagen.

Diese Arbeit adressiert dieses wichtige vorgelagerte Problem durch die Entwicklung eines transparenten, probabilistischen Rahmens zur Vorhersage hochrangiger menschlicher Fahrhandlungen unter Verwendung dynamischer Bayes'scher Netze (Dynamic Bayesian Networks (DBNs)). Ziel dieser Forschung ist die Entwicklung eines Modells, das zukünftige Fahreraktionen abschätzen kann und gleichzeitig Unsicherheiten in einer interpretierbaren Form darstellt. Ein solches Modell kann innerhalb eines AV-Systems als Belief-Modell dienen und Informationen für nachgelagerte Planungs- und sicherheitsrelevante Module bereitstellen.

Die Studie verwendet den naturalistischen Autobahnfahrdatensatz highD, um zeitliche Muster menschlichen Fahrverhaltens zu erlernen. Fahrzeugtrajektorien werden in sequenzielle, fensterbasierte Beobachtungen umgewandelt, die relevante Bewegungs- und Interaktionsinformationen über die Zeit erfassen. Diese Beobachtungen fassen zusammen, wie sich ein Fahrzeug und der umgebende Verkehr über kurze Zeitintervalle entwickeln. Anschließend wird ein DBN trainiert, um die zeitliche Entwicklung des Fahrerverhaltens zu modellieren und Aktionswahrscheinlichkeiten aus eingehenden Beobachtungen abzuleiten. Das vorgeschlagene DBN verwendet eine hierarchische latente Struktur, in der Aktionen von Fahrregimen abhängig sind. Genauer gesagt repräsentiert das Modell zwei latente Regime, ein Regime mit geringer Interaktion und ein Regime mit hoher Interaktion, und sagt Aktionszustände innerhalb jedes Regimes vorher. Dadurch kann das Modell sowohl das vorhergesagte Fahrmanöver als auch den Interaktionskontext darstellen, in dem es auftritt.

Eine zentrale Stärke dieses Ansatzes liegt in seiner transparenten probabilistischen White-Box-Struktur. Im Gegensatz zu vielen Black-Box-Lernmethoden erzeugt das DBN seine Vorhersagen über explizite probabilistische Beziehungen, die mithilfe der Wahrscheinlichkeitstheorie interpretiert werden können. Dies ist insbesondere für sicherheitsrelevante Anwendungen von Bedeutung, bei denen Interpretierbarkeit und die Darstellung von Unsicherheiten wichtig sind. Das Modell unterstützt daher nicht nur die Vorhersageleistung, sondern auch die Erklärbarkeit, was bei der Betrachtung solcher

Systeme für sicherheitskritische Verkehrsanwendungen von Vorteil ist. Die Studie leistet einen Beitrag zur Forschung im Bereich des autonomen Fahrens, indem sie Aktionsvorhersage, zeitliche probabilistische Schlussfolgerung und Interpretierbarkeit in einem einheitlichen Rahmen kombiniert.

Die Ergebnisse zeigen, dass das Modell in der Lage ist, aussagekräftige Verhaltensmuster aus naturalistischen Verkehrsdaten zu lernen und vielversprechende Manövervorhersagen über einen zukünftigen Zeithorizont zu generieren. In der aktuellen Evaluation erreichte das Modell eine exakte Next-Step-Genauigkeit von 46,4 %, eine Makropräzision von 44,3 % sowie eine Trefferquote von 55,2 % über einen Vorhersagehorizont von zwei Sekunden. Eine Vorhersage zum Zeitpunkt  $t$  wird dabei als Treffer gezählt, wenn das vorhergesagte Manöver innerhalb der nächsten fünf Fensterschritte auftritt, was etwa zwei Sekunden entspricht. Werden semantisch ähnliche Manövergruppen zusammengefasst, steigt die Leistung auf 62,4 % exakte Next-Step-Genauigkeit und 70,2 % Trefferquote über denselben 2-Sekunden-Horizont, was auf viele nahe Verwechslungen zwischen verwandten Klassen hinweist. Die durchschnittliche Time-to-Hit betrug 0,54 s und der Median 0,40 s, was zeigt, dass viele korrekte Vorhersagen früh innerhalb des Vorhersagehorizonts auftreten. Diese Ergebnisse deuten darauf hin, dass das Modell zukünftige menschliche Fahreraktionen innerhalb eines praktisch nutzbaren kurzfristigen Zeitintervalls antizipieren kann, anstatt nur einen einzelnen exakten zukünftigen Zeitpunkt vorherzusagen. Implementierungs-Benchmarks zeigen zudem geringe Online-Inferenzkosten mit einer mittleren CPU-Latenz von 0,164 ms pro Fahrzeug für Belief-Update und nächste Aktionsvorhersage.

Insgesamt zeigen die Ergebnisse, dass DBNs ein vielversprechender Ansatz für die probabilistische Vorhersage menschlicher Fahreraktionen sind. Die Studie demonstriert, dass hochrangige menschliche Fahrmanöver mithilfe naturalistischer Verkehrsdaten in einer interpretierbaren und zeitlich strukturierten Weise modelliert werden können. Dies stellt einen wichtigen Schritt hin zu einem besseren Verständnis menschlichen Fahrverhaltens im gemischten Verkehr dar und kann AV-Systeme dabei unterstützen, das Verhalten umliegender menschlicher Fahrer besser einzuschätzen.

**Stichwörter:** Vorhersage menschlichen Fahrverhaltens, Autonomes Fahren, Von Menschen gesteuerte Fahrzeuge, Dynamische Bayes'sche Netze, Manövervorhersage, Probabilistische Modellierung, Gemischte Verkehrsumgebungen

# Abstract

Artificial Intelligence (AI) is transforming autonomous systems through major advances in sensing, communication, and computation. Although fully Autonomous Vehicles (AVs) are expected to become more common, they will coexist on the road with Human-Driven Vehicles (HDVs) for the time being. To operate safely in this mixed-traffic setting, autonomous systems must be able to model human driver behavior and predict future maneuvers.

This work addresses this important upstream problem by developing a transparent, probabilistic framework for predicting high-level human driving behavior using Dynamic Bayesian Networks (DBNs). The purpose of this research is to develop a model that can estimate likely future maneuvers while also representing uncertainty in an interpretable form. Such a model can act as a belief model inside an AV framework and provide information for downstream planning and safety-related modules.

The study uses the highD naturalistic highway driving dataset to learn temporal patterns of human driving behavior. Vehicle trajectory data are converted into sequential window-based observations that capture relevant motion and interaction information over time. These observations summarize how the vehicle and its surrounding traffic evolve over short time intervals. A DBN is then trained to model the temporal evolution of driver behavior and to infer action probabilities from incoming observations. The proposed DBN uses a hierarchical latent structure in which actions are conditioned on driving regimes. More specifically, the model represents two latent regimes, low-interaction and high-interaction, and predicts discrete action states within each regime. This allows the framework to represent both the predicted maneuver and the interaction context in which it occurs.

A key strength of this approach is that it provides a transparent, white-box probabilistic structure. Unlike many black-box learning methods, the DBN makes predictions through explicit probabilistic relationships that can be interpreted using probability theory. This is valuable in safety-relevant applications, where interpretability and uncertainty representation are important. The model therefore supports not only prediction, but also explainability, which is useful when such systems are considered for use in safety-critical transport settings. The study contributes to science and engineering by combining maneuver prediction, temporal probabilistic reasoning, and interpretability in a single framework for autonomous driving applications.

The results show that the model can learn meaningful behavioral patterns from naturalistic traffic data and generate promising maneuver predictions over a future horizon. In the current evaluation, the model achieved 46.4% exact next-step accuracy, 44.3% macro

## *Abstract*

precision, and a 55.2% hit rate over a 2-second prediction horizon. Here, a prediction made at time  $t$  is counted as a hit if the predicted maneuver appears at any point within the next 5 window steps, corresponding to about 2 seconds. When semantically similar maneuver groups are merged, performance increases to 62.4% exact next-step accuracy and a 70.2% hit rate over the same 2-second horizon, indicating many near-misses between closely related classes. The mean time-to-hit was 0.54 s, and the median time-to-hit was 0.40 s, showing that many correct predictions occur early within the horizon. These results suggest that the model can anticipate future human driver maneuvers within a practically useful short-term interval rather than only at one exact future instant. Implementation benchmarks also indicate low online inference cost, with a mean single-vehicle Central Processing Unit (CPU) latency of 0.164 ms for belief update and next-maneuver prediction.

Overall, the findings indicate that DBNs are a promising framework for probabilistic human driver maneuver prediction. The study shows that high-level human driving maneuvers can be modeled in an interpretable and temporally structured way using naturalistic traffic data. This provides a useful step toward better behavior understanding in mixed traffic and may support AV systems that need to reason about nearby human drivers.

**Keywords:** Human Driver Behavior Modeling, Autonomous Driving, Human-Driven Vehicles, Dynamic Bayesian Networks, Maneuver Prediction, Probabilistic Modeling, Mixed Traffic Environments



# 1. Introduction

Automation is influencing almost every part of daily life, and transportation is one of the areas changing most rapidly. Autonomous Vehicles (AVs) are at the forefront of this change, revolutionizing the transportation sector. Their development has accelerated because of recent progress in Artificial Intelligence (AI), sensing systems, computing, and communication technologies. These innovations could enhance traffic efficiency, increase travel comfort, improve road safety, and, most importantly, reshape future mobility.

However, fully autonomous traffic systems do not yet exist anywhere in the world. AVs will have to operate in mixed traffic alongside Human-Driven Vehicles (HDVs). This creates important challenges because different road users provide different types and amounts of information. While AVs can rely on rich on-board sensing and may also use Vehicle-to-Everything (V2X) communication [1], the intentions of human drivers are not directly observable and must be inferred from their behavior in traffic [2]. Consequently, their intentions and decisions cannot be measured directly, but instead have to be estimated from observable cues such as vehicle motion, lane changes, relative positions, and interactions with nearby vehicles.

For this reason, human driver behavior prediction is an important problem in autonomous driving. If an AV can estimate what nearby human drivers are likely to do next, it can make safer and better-informed decisions. Anticipating likely braking, lane-changing, or car-following behavior can support downstream tasks such as risk assessment, trajectory prediction, and motion planning [2], [3], [4].

Over the last two decades, researchers have developed three broad families of behavioral models for road users: rule-based models, probabilistic models, and learning-based methods [3], [5], [6]. In parallel, several game-theoretic decision-making paradigms for AVs have emerged [7]. Rule-based models are simple and easy to understand, but they are often too rigid to capture the variability of real human behavior. Learning-based methods can learn complex patterns from large datasets, but many of them are difficult to interpret and are considered as black boxes. Probabilistic models provide a useful middle ground because they can explicitly represent uncertainty, capture temporal dependencies, and still remain interpretable.

This thesis focuses on probabilistic modeling of high-level human driving behavior in highway traffic. Rather than building the full control or planning stack of an AV, this work addresses an important upstream problem: predicting the likely future maneuvers of surrounding human drivers. The aim is to develop a model that can estimate likely future maneuvers while also representing uncertainty in a clear and interpretable way. In this thesis, future maneuvers are represented internally as discrete action states.

## *1. Introduction*

To achieve this, the thesis models the temporal relationship between observable driving features and hidden behavioral states using a Dynamic Bayesian Network (DBN). Since driver intentions are not directly observable, the model represents them through latent variables and infers them probabilistically from trajectory-based observations [8]. In particular, the DBN is designed to capture latent factors such as driving context and action dynamics, and to produce probabilistic predictions of future high-level maneuvers. The highD dataset, a realistic highway driving dataset with comprehensive vehicle trajectory recordings, is used to train and assess the model [9]. From these trajectories, relevant features are extracted to describe motion, surrounding traffic context, and interaction behavior. These features serve as observations for the DBN.

The thesis studies whether such a probabilistic framework can provide accurate, interpretable, and computationally efficient short-horizon predictions of future human driving maneuvers. In particular, it examines the suitability of the dataset, the structure of the DBN, the usefulness of the selected observation features, the quality of future maneuver prediction, the interpretability of the learned states, and the runtime performance of the model for online use.

## 2. Theoretical Background

This chapter presents the theoretical background required for the methods used later in this thesis. Section 2.1 introduces the main concepts from probability theory. Section 2.2 then describes probabilistic graphical models, including Bayesian Networks (BNs) and DBNs. Section 2.3 presents inference in chain-structured DBNs. Section 2.4 summarizes parameter learning using the Expectation-Maximization (EM) algorithm.

**Notation convention** Throughout this thesis, capital letters denote Random Variables (RVs), while lowercase letters denote specific realizations of those variables. Bold lowercase symbols denote vectors, bold uppercase symbols denote matrices, and calligraphic uppercase symbols denote third-order (or higher-order) tensors.

### 2.1. Probability Theory

Probability theory provides a mathematical foundation for representing and reasoning about uncertainty [10]. Instead of assigning a single deterministic outcome to an uncertain event, probabilistic models provide a probability distribution over possible outcomes. Each uncertain quantity is represented as a RV [10].

Complex systems require modeling interactions between many RVs. When several RVs are considered together, their behavior can be described using a joint probability distribution [10]. For two RVs  $X$  and  $Y$ , the joint probability is written as  $P(X, Y)$  and indicates the likelihood that both variables take specific values simultaneously. This idea can be extended to any number of variables.

Dependencies between RVs are described using conditional probability [10]. The conditional probability of  $X$  given  $Y$  is defined as

$$P(X | Y) = \frac{P(X, Y)}{P(Y)},$$

provided that  $P(Y) > 0$ . Conditional probability describes how the probability of one RV changes when information about another RV is known.

Another important concept is marginal probability, which gives the probability of a subset of RVs irrespective of the values of the remaining variables [10]. For example, if the joint distribution  $P(X, Y)$  is known, the marginal probability of  $Y$  can be obtained by summing over all possible values of  $X$ :

$$P(Y) = \sum_X P(X, Y)$$

## 2. Theoretical Background

In addition to describing dependencies between RVs, it is also important to identify cases in which variables do not influence each other. Two RVs are said to be independent if the value of one variable has no effect on the probability of the other. For independent variables  $X$  and  $Y$ , the joint probability can be written as:

$$P(X, Y) = P(X)P(Y)$$

In many practical problems, variables are not completely independent but may become independent when conditioned on another variable. This property is known as conditional independence. Given a third variable  $Z$ , two variables  $X$  and  $Y$  are conditionally independent if the joint conditional distribution factorizes as follows:

$$P(X, Y | Z) = P(X | Z)P(Y | Z)$$

Conditional independence is a key concept in probabilistic graphical models because it allows complex joint probability distributions to be represented using simpler conditional relationships between variables [11].

### 2.1.1. Bayes' Theorem

A fundamental principle in probabilistic reasoning is Bayes' theorem, which allows beliefs about uncertain events to be updated when new observations become available [10]. It relates conditional and marginal probabilities and is expressed as:

$$P(X | Y) = \frac{P(Y | X)P(X)}{P(Y)} \quad (2.1)$$

Here,  $P(X)$  is the prior probability,  $P(Y | X)$  is the likelihood,  $P(Y)$  is the evidence or marginal likelihood, and  $P(X | Y)$  is the posterior probability. Bayes' theorem therefore provides a systematic way to update probabilistic beliefs when new observations become available. This principle forms the basis for probabilistic inference in many models, including DBNs.

## 2.2. Probabilistic Graphical Models

Probabilistic graphical models provide an efficient way to represent complex joint probability distributions over many RVs [10], [11]. As the number of variables increases, the full joint distribution becomes harder to model because possible dependencies also grows. Probabilistic graphical models address this problem by representing RVs and their probabilistic relationships using a graph. In this representation, nodes correspond to RVs, while edges describe dependencies between them. The graph structure also encodes conditional independence assumptions. These assumptions allow the joint probability distribution to be factorized into simpler Conditional Probability Distributions (CPDs).

There are two main types of probabilistic graphical models:

## 2. Theoretical Background

1. **Directed graphical models**, where edges have a direction and represent conditional dependencies between variables.
2. **Undirected graphical models**, where edges do not have a direction and represent symmetric relationships between variables.

One important class of probabilistic graphical models is the BN. Its temporal extension is the DBN, which is particularly suitable for modeling sequential processes over time.

### 2.2.1. Bayesian Networks

A BN is a directed probabilistic graphical model that uses a Directed Acyclic Graph (DAG) to describe a set of RVs and their conditional dependencies [11]. Every variable in the network has an associated CPD, that explains how it depends on its parent variables. A prior probability distribution provides the distribution of a variable if it has no parents in the graph.

In general, the joint probability distribution of all variables in a BN can be factorized according to the graph structure. The joint distribution for a set of variables  $X_1, X_2, \dots, X_n$ , can be expressed as follows:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Pa}(X_i)) \quad (2.2)$$

where  $\text{Pa}(X_i)$  denotes the set of parent variables of  $X_i$  in the graph. This factorization reflects the conditional independence assumption embedded in the network structure.

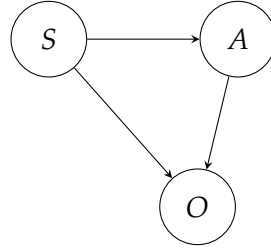


Figure 2.1.: Example of a BN.

The joint distribution for the network shown in Figure 2.1 can be expressed as follows:

$$P(S, A, O) = P(S)P(A \mid S)P(O \mid S, A)$$

Standard BNs describe relationships between variables within a single time step. However, in many real-world problems, the system evolves over time. In such cases, a temporal extension of BNs, called a DBN, is used.

### 2.2.2. Dynamic Bayesian Networks

In a DBN, the system is represented as a sequence of time-indexed RVs that describe how the system evolves over time. A common representation of a DBN uses a two-slice temporal model that captures dependencies both within a single time step and across consecutive time steps. This representation typically relies on the first-order Markov assumption, which holds that the current state depends solely on the previous state and not on earlier states. A general DBN may have several latent states with structured dependencies between them, in contrast to a Hidden Markov Model (HMM), which can be thought of as a specific case of a DBN with a single latent state per time step.

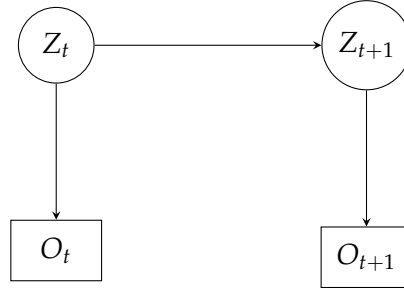


Figure 2.2.: Two-slice representation of a DBN.

In the example shown in Figure 2.2, let  $Z_t$  represent the latent state at time step  $t$  and  $O_t$  denote the corresponding observation. The joint probability distribution over a series of states and observations under the Markov assumption can be expressed as follows:

$$P(Z_{1:T}, O_{1:T}) = P(Z_1) \prod_{t=2}^T P(Z_t | Z_{t-1}) \prod_{t=1}^T P(O_t | Z_t) \quad (2.3)$$

In this formulation,  $P(Z_t | Z_{t-1})$  represents the state transition model, which captures how the latent state evolves over time, while  $P(O_t | Z_t)$  represents the emission model that relates the hidden state to the observed data.

### 2.3. Inference in Chain-Structured DBNs

Inference in probabilistic graphical models refers to estimating the probability distribution of unknown variables conditioned on observed evidence. In temporal models like the DBN shown in Figure 2.2, inference involves estimating the distribution of latent states from a sequence of observations, since only the observations  $O_{1:T}$  are available while the state sequence  $Z_{1:T}$  remains hidden [10], [11].

In temporal probabilistic models, three common inference tasks arise [12]:

1. **Filtering:** Estimating the current latent state using observations up to the current time step,  $P(Z_t | O_{1:t})$ .

## 2. Theoretical Background

2. **Smoothing:** Estimating the latent state using the full observation sequence,  $P(Z_t | O_{1:T})$ .
3. **Prediction:** Estimating future latent states from the observations available up to the current time,  $P(Z_{t+k} | O_{1:t})$ .

For the chain-structured DBN shown in Figure 2.2, the latent state at time  $t$  depends only on the previous state  $Z_{t-1}$ , and the observation  $O_t$  depends only on the current state  $Z_t$ . This structure corresponds to a first-order Markov model with conditionally independent observations, which is equivalent to the structure of a HMM. Therefore, exact inference can be performed using the forward-backward algorithm [13]. This algorithm is widely used in HMMs, which are a special case of DBNs.

### 2.3.1. Forward–Backward Algorithm

The forward-backward algorithm computes posterior probabilities of latent states by combining information from past and future observations [13]. More specifically, it estimates the posterior distribution of the hidden state at each time step given the complete observation sequence.

Let the observation sequence be  $O_{1:T} = \{O_1, O_2, \dots, O_T\}$  and the corresponding latent states be  $Z_{1:T}$ . The main objective is to compute the posterior distribution,  $P(Z_t | O_{1:T})$ , which is called the smoothing distribution.

The forward-backward algorithm performs this computation using two recursive quantities: the forward messages and the backward messages.

#### Forward Pass

The joint probability of seeing the partial sequence up to time  $t$  and being in state  $z_t$  is represented by the forward variable  $\alpha_t(z_t)$ :

$$\alpha_t(z_t) = P(O_{1:t}, Z_t = z_t) \quad (2.4)$$

It can be computed recursively as:

$$\alpha_t(z_t) = P(O_t | Z_t = z_t) \sum_{z_{t-1}} P(Z_t = z_t | Z_{t-1} = z_{t-1}) \alpha_{t-1}(z_{t-1}) \quad (2.5)$$

with initialization:

$$\alpha_1(z_1) = P(Z_1 = z_1) P(O_1 | Z_1 = z_1)$$

Here,  $P(Z_t = z_t | Z_{t-1} = z_{t-1})$  is the transition probability,  $P(O_t | Z_t = z_t)$  is the observation likelihood, and  $P(Z_1 = z_1)$  is the initial state distribution.

## 2. Theoretical Background

### Backward Pass

The likelihood of seeing the remaining sequence from time  $t + 1$  to  $T$  given the current state is represented by the backward variable  $\beta_t(z_t)$ :

$$\beta_t(z_t) = P(O_{t+1:T} \mid Z_t = z_t) \quad (2.6)$$

It is computed recursively in the reverse direction as:

$$\beta_t(z_t) = \sum_{z_{t+1}} P(Z_{t+1} = z_{t+1} \mid Z_t = z_t) P(O_{t+1} \mid Z_{t+1} = z_{t+1}) \beta_{t+1}(z_{t+1}) \quad (2.7)$$

with initialization:

$$\beta_T(z_T) = 1$$

### Posterior State Estimation

The posterior probability of the hidden state at time step  $t$  can be estimated after the forward and backward messages have been computed. The likelihood that the system is in state  $z_t$  at time  $t$  given the whole observation sequence is represented by this quantity, which is typically written as  $\gamma_t(z_t)$ .

$$\gamma_t(z_t) = P(Z_t = z_t \mid O_{1:T})$$

Using the forward and backward variables, this can be written as:

$$\gamma_t(z_t) = \frac{\alpha_t(z_t) \beta_t(z_t)}{P(O_{1:T})} \quad (2.8)$$

where the likelihood of the entire observation sequence is represented by  $P(O_{1:T})$ . Using the forward and backward variables, for any time step  $t$ , it can be written as:

$$P(O_{1:T}) = \sum_{z_t} \alpha_t(z_t) \beta_t(z_t)$$

The forward-backward algorithm therefore combines information from earlier observations through the forward pass and from later observations through the backward pass.

### Posterior Transition Probability

In addition to the posterior probability of individual states, it is also useful to compute the joint posterior probability of two consecutive states. Given the whole observation sequence, this quantity provides the posterior probability of a transition from state  $z_t$  to state  $z_{t+1}$ :

$$\xi_t(z_t, z_{t+1}) = P(Z_t = z_t, Z_{t+1} = z_{t+1} \mid O_{1:T})$$



## 2. Theoretical Background

Using the forward and backward variables, this can be written as:

$$\xi_t(z_t, z_{t+1}) = \frac{\alpha_t(z_t) P(Z_{t+1} = z_{t+1} \mid Z_t = z_t) P(O_{t+1} \mid Z_{t+1} = z_{t+1}) \beta_{t+1}(z_{t+1})}{P(O_{1:T})} \quad (2.9)$$

This posterior transition probability is useful in parameter learning, especially in EM-based training methods [13].

### 2.4. Parameter Learning in Latent-State Models

A probabilistic model contains several unknown parameters that must be learned from data. For a chain-structured temporal model such as the one shown in Figure 2.2, these parameters can be collectively written as:

$$\theta = \{\pi, A, \Theta\}$$

where  $\pi$  is the initial-state probability vector,  $A$  is the state transition matrix, and  $\Theta$  represents the set of emission parameters.

If both the observation sequence  $O_{1:T}$  and the latent state sequence  $Z_{1:T}$  were known, parameter learning could be performed by maximizing the complete-data likelihood:

$$\theta^* = \arg \max_{\theta} P(O_{1:T}, Z_{1:T} \mid \theta) \quad (2.10)$$

However, in latent-variable models the state sequence  $Z_{1:T}$  is not observed. Therefore, learning must be based only on the observed data, which gives the Maximum Likelihood Estimation (MLE) objective:

$$\theta^* = \arg \max_{\theta} P(O_{1:T} \mid \theta) \quad (2.11)$$

Since the latent state sequence is unknown, the likelihood of the observations is obtained by marginalizing over all possible latent state sequences:

$$P(O_{1:T} \mid \theta) = \sum_{Z_{1:T}} P(O_{1:T}, Z_{1:T} \mid \theta) \quad (2.12)$$

Since this summation involves many possible latent state sequences, direct maximization of the observed-data likelihood in (2.11) is difficult. To address this problem, the EM algorithm is commonly used [12]. For HMMs, this learning procedure is known as the Baum–Welch algorithm [13].

#### 2.4.1. Expectation–Maximization Algorithm

The EM algorithm is an iterative technique for parameter estimation in probabilistic models with latent variables. It addresses the difficulty of maximizing the observed-data likelihood by alternating between estimating the latent states and updating the model parameters [12].

## 2. Theoretical Background

### Expectation Step

In the expectation step (E-step), the current parameter estimate is used to compute the posterior distribution of the latent states given the observations [12]. For the chain-structured model considered here, this is done using the forward-backward algorithm described by (2.5) and (2.7). This step provides the posterior state probabilities and the posterior transition probabilities:

$$\gamma_t(i) = P(Z_t = i \mid O_{1:T})$$

$$\xi_t(i, j) = P(Z_t = i, Z_{t+1} = j \mid O_{1:T})$$

As shown in (2.8) and (2.9), these posterior quantities represent the probabilities of state occupancy and state transitions conditioned on the full observation sequence.

### Maximization Step

In the maximization step (M-step), the model parameters are updated using the posterior quantities computed in the E-step [12].

For discrete-state chain-structured models, the initial state distribution is updated as:

$$\pi_i = \gamma_1(i) \tag{2.13}$$

The transition probabilities can be re-estimated as:

$$A_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)} \tag{2.14}$$

The parameters of the emission model are updated using the observations weighted by the posterior state probabilities. The exact form of this update depends on the chosen emission model.

The E-step and M-step are repeated until convergence. Under standard EM, each iteration does not decrease the observed-data likelihood, and the algorithm converges to a local optimum or stationary point [12].

## **3. State of the Art**

### **3.1. Introduction**

Existing approaches for modeling and predicting human driver behavior can be broadly grouped into rule-based models, probabilistic models, and learning-based models. These families differ in how they represent driving behavior, uncertainty, and interactions with the surrounding traffic environment.

This chapter reviews the main state-of-the-art approaches for human driver behavior modeling and maneuver prediction, with particular attention to their suitability for uncertainty representation and interpretability. Since human driver maneuver prediction is intended to support autonomous driving systems, the chapter also briefly discusses how such predictions are used in downstream decision making, including interaction-aware planning and risk-aware reasoning.

### **3.2. Rule-Based Models**

Rule-based models characterize driving behavior through explicit mathematical or logical rules. These models relate observable variables such as spacing, relative velocity, and desired speed to longitudinal acceleration or lane-change decisions. Their main advantages are interpretability and low computational cost [14]. For this reason, they are widely used in traffic simulation, Advanced Driver Assistance Systems (ADAS), and baseline behavior modeling. However, as they rely on hand-crafted assumptions about driver decision-making, they often represent behavior in a deterministic or simplified manner.

#### **3.2.1. Car-Following Models**

##### **Intelligent Driver Model**

The Intelligent Driver Model (IDM) is one of the most popular car-following models [14]. It expresses a vehicle's acceleration as a continuous function of its current velocity, desired velocity, spacing to the leading vehicle, and relative velocity [15]. The formulation also includes parameters related to desired time headway and comfortable braking, which make the model suitable for safety-oriented traffic simulation. Because of its simple structure and physically meaningful parameters, the IDM remains a common reference model in intelligent driving research.

#### **Gipps' Model**

Gipps' model is one of the earliest rule-based car-following models. It is based on the idea that a driver chooses a speed that is both desirable and safe with respect to the preceding vehicle [16]. This safety-oriented formulation allows the model to capture conservative driving behavior and ensures collision-free motion under typical traffic conditions.

#### **Optimal Velocity Model and Full Velocity Difference Model**

Another classical approach is the Optimal Velocity Model (OVM), in which the driver's acceleration depends on the difference between the current velocity and an optimal velocity determined by the spacing to the leading vehicle [17]. This model captures the driver's intuition to accelerate when the gap ahead increases and decelerate when it decreases. An extension of the OVM is the Full Velocity Difference Model (FVDM), where, instead of only considering the gap between the vehicles, the model also captures the velocity difference between the ego and preceding vehicles when calculating the acceleration [18]. This helps reduce the oscillations observed in the original OVM.

#### **3.2.2. Lane-Change Models**

##### **MOBIL Model**

The Minimizing Overall Braking Induced by Lane Changes (MOBIL) model is a lane-changing model built upon the IDM to describe lateral driving behavior [14], [19]. It incorporates incentive and safety criteria to evaluate the feasibility of lane changes. The incentive criterion indicates whether a lane change could be advantageous in terms of the driver's acceleration, whereas the safety criterion ensures that the lane change does not cause unnecessary deceleration for surrounding vehicles.

Despite their interpretability and efficiency, rule-based models have limited ability to represent uncertainty and inter-driver variability in complex traffic scenarios. These limitations motivate the use of probabilistic and learning-based approaches.

#### **3.3. Probabilistic Models**

Probabilistic models describe driving behavior as a stochastic process, where driving actions are mapped to probability distributions, and represent uncertainty explicitly [20]. In this framework, a driver's decision to accelerate or change lanes is modeled as a probability function that depends on the current traffic context. This approach allows for multiple possible behavioral outcomes, rather than a single deterministic response. This makes them suitable for modeling the variability and ambiguity that are inherent in human driving behavior.

#### 3.3.1. Hidden Markov Models

The HMM is a widely used probabilistic framework for modeling time-varying stochastic processes in which the underlying system states are not directly observable but produce observable outputs [13]. HMM assumes that human driving behavior evolves through a sequence of decision states, such as lane-keeping, braking, and overtaking, that are not directly measurable [21]. These states produce observable vehicle signals like speed, acceleration, and lane position. The objective of the HMM is to infer the most probable state sequence and predict future states from the observed driving data. The main strengths of HMMs are conceptual simplicity and efficient inference. However, HMMs rely on the Markov property, which simplifies the model but limits its ability to capture long-term temporal dependencies and complex contextual interactions [22].

#### 3.3.2. Dynamic Bayesian Networks

DBNs extend static BNs to temporal settings and model dependencies among variables across successive time steps [8], [23]. Compared with HMMs, DBNs provide a more flexible representation because, instead of using a single discrete variable, the system state can be described in factored form. This allows DBNs to represent multiple interacting hidden and observed variables, which makes them suitable for modeling the joint evolution of driver state, vehicle behavior, and surrounding traffic context [24]. For instance, Młynarczyk et al. (2025) applied BNs to capture causal links between drivers' affective states and behavioral responses [25], while Talluri et al. (2025) employed DBNs to enhance safety standards in automated driving systems through causal reasoning for lane-change prediction and safety assessment [26]. By explicitly modeling dependencies between hidden cognitive states and observable signals, DBNs provide an excellent framework for intention recognition and maneuver prediction in autonomous driving systems [24].

#### 3.3.3. Gaussian-Based Probabilistic Models

Gaussian-based probabilistic models are widely used for modeling continuous-valued driving behaviors such as acceleration or trajectory changes. A single Gaussian distribution is unimodal, whereas driver behavior in a given traffic context may admit multiple plausible future actions or trajectories [27]. To capture such variability, Gaussian Mixture Models (GMMs) represent the joint distribution of relevant variables, such as positions, velocities, headways, or steering angles, as a weighted sum of multiple Gaussian components. For the behavioral modeling of HDVs at the operational level, probabilistic regression techniques such as Gaussian Mixture Regression (GMR) and Gaussian Process Regression (GPR) are often employed to model continuous driving actions, like acceleration or steering angle, for a given traffic context [28], [29]. For example, Zhang et al. use a GMR model to capture feature correlations and temporal dynamics in car-following behavior [28], while Bethge et al. use Gaussian processes to predict human driving behavior and explicitly account for predictive uncertainty [29]. These models

provide low-level behavior representations by capturing the stochastic nature of human responses.

## 3.4. Learning-Based Models

Deep Learning (DL) has enabled powerful data-driven models that predict future trajectories directly from historical motion, map information, and interaction context. Unlike traditional rule-based or probabilistic frameworks that rely on explicit assumptions about driver intent or traffic interactions, these models learn patterns directly from extensive driving data. By making use of large datasets, they can capture complex, nonlinear relationships between driving context and human decisions with high accuracy.

### 3.4.1. Classical Machine Learning Models

Classical machine learning methods formed the earliest data-driven foundation for maneuver prediction before the advent of modern DL. Algorithms such as k-Nearest Neighbors (kNNs), Support Vector Machines (SVMs), Decision Trees, and Random Forests were commonly utilized to predict driver intent, classify maneuvers, or recognize driving styles [30]. These approaches rely on manually designed features rather than automatically learned representations, as used in modern DL approaches [31].

### 3.4.2. Recurrent Neural Network-Based Models

Recurrent Neural Networks (RNNs) form a specialized class of Neural Networks (NNs) developed to model temporal relationships in sequential data [32]. They incorporate internal states and cyclic connections that enable them to process time-dependent information, making them suitable for driver intention recognition and trajectory prediction in behavioral prediction tasks [33], [34]. However, traditional RNNs suffer from the vanishing gradient problem [35]. To address this limitation, the Long Short-Term Memory (LSTM) architecture was introduced. LSTM networks incorporate a dedicated memory cell that helps preserve contextual information over longer time horizons. Although such models perform well in vehicle prediction tasks, they still face limitations in long-horizon forecasting and in handling complex road geometries [33], [36].

Xin et al. (2019) proposed an intention-aware dual LSTM network that separately models the driver's intention and motion dynamics [33]. The first LSTM module captures the latent driving intention using past motion data and environmental context, while the second predicts the future trajectory conditioned on the inferred intention. This dual-stage design allows the model to reason about how inferred driver intentions influence subsequent maneuvers.

Hao et al. (2024) present a hybrid framework that combines a DBN for lane-change intention prediction with a driving-style estimator constructed from K-means clustering and Bayesian filtering [34]. These probabilistic cues are then integrated into an LSTM

### 3. State of the Art

encoder—decoder, which produces personalized future trajectories conditioned on both driver intent and style. This fusion allows the network to capture maneuver likelihoods and behavioral variability more effectively than pure LSTM models.

#### 3.4.3. Graph Neural Networks

Modeling the relational structure among traffic participants is important in dynamic traffic scenes. Graph Neural Networks (GNNs) provide an effective framework for representing and learning such multi-agent interactions. In a GNN, each vehicle can be represented as a node, while edges encode the interactions between vehicles [37]. Recent research has explored integrating GNNs with temporal sequence models such as LSTMs and Transformers to jointly capture spatial and temporal dependencies. Abdelraouf et al. (2023) proposed a Temporal Graph Neural Network (T-GNN) that combines Graph Convolutional Networks (GCNs) with LSTM layers to personalize trajectory prediction for different driving styles [38].

However, GNNs performance is highly dependent on the design of the interaction graph, since graph construction determines which neighboring agents and relations are modeled [37]. An overly sparse graph structure may lead to loss of critical interaction information, while overly dense connections can introduce noise and computational complexity. Moreover, most existing GNN-based trajectory prediction models are highly data-driven and require large, diverse datasets for generalization.

#### 3.4.4. Transformers

Transformers are a recent and powerful DL architecture that are very good at capturing long-range dependencies in sequential data. Unlike recurrent networks, they rely on self-attention that explicitly model interactions between all elements in a sequence, regardless of their temporal distance. In addition, self-attention enables parallel processing of the entire input sequence, which improves computational efficiency [39], [40]. In the multi-head self-attention mechanism, different attention heads can focus on different aspects of the input, and the resulting representations are aggregated to predict future maneuvers or trajectories.

SceneTransformer introduced a model that unified motion prediction and planning through a scene-level attention mechanism that jointly predicts the trajectories of all agents within a traffic scene [41]. Instead of treating each vehicle as an independent entity, SceneTransformer predicts the joint future of multiple interacting agents. Motion Transformer (MTR) extended this approach by introducing a hierarchical encoder—decoder framework that explicitly models behavioral uncertainty, producing multiple possible future trajectories per agent [42]. Building upon these advances, the Hierarchical Vector Transformer (HiVT) introduces a multi-scale hierarchical attention structure designed to handle large numbers of interacting agents efficiently [43]. HiVT first applies local attention to encode trajectory and map features for each agent, then applies a global

### 3. State of the Art

attention layer to model interactions among all agents in the scene. The decoder uses these representations to forecast each agent’s future trajectory in a single forward pass.

However, despite their strong representation capacity, Transformer-based models generally require large datasets and careful regularization to avoid overfitting. Moreover, their interpretability remains limited compared to probabilistic graphical models or physics-based approaches.

#### 3.4.5. Conditional Variational Autoencoders

Conditional Variational Autoencoders (CVAEs) are probabilistic deep generative models that extend the traditional Variational Autoencoder (VAE) framework by incorporating contextual information into both the encoder and decoder networks [44]. In trajectory prediction, the input observation  $x$  captures contextual information while the corresponding output  $y$  represents the future trajectory to be predicted. A latent variable  $z$  is introduced such that its conditional posterior is approximated through  $q_\phi(z | x, y)$ , which is learned by the encoder.

By conditioning both the encoder and decoder on the contextual variable  $x$ , a CVAE can capture complex, multimodal relationships between the observed scene and possible futures, enabling diverse yet context-consistent predictions. The latent variable  $z$  captures hidden factors, such as intent or other stochastic influences, that are not directly observable from the input context. As a result, the model can generate a range of possible future trajectories that represent different behavioral modes within the same traffic scenario.

Trajectron++ represents one of the popular CVAE-based frameworks for multimodal trajectory prediction. It combines LSTM-based temporal encoding, graph-structured interaction modeling, and probabilistic latent variable reasoning [45]. Each traffic participant is represented as a node in a dynamically evolving spatiotemporal graph, where edges capture interactions with surrounding agents. The past motion history of each agent is first processed through an LSTM network to encode temporal dependencies, after which a GNN aggregates contextual information across agents. The resulting embeddings are passed into a CVAE decoder, which generates diverse future trajectories by sampling from a learned latent space.

While Trajectron++ focuses on interaction-aware multimodality, Predicted Endpoint Conditioned Network (PECNet) follows a related goal-conditioned approach to trajectory prediction [46]. It models driver intent by predicting a distribution over potential final endpoints, which are spatial points that represent the driver’s intended goal. The model then generates complete trajectories conditioned on these sampled endpoints. This two-step design separates high-level intention from detailed motion planning.

More recent research has explored hierarchical extensions of the CVAE framework to reduce long-horizon prediction error and maintain trajectory consistency over time. Single-stage CVAE models can suffer from reduced prediction quality at longer horizons. For example, the Cascaded CVAE addresses this by refining predictions through multiple



### 3. State of the Art

CVAE stages, where each stage predicts short-term motion and subsequently extends or corrects it over longer horizons [47].

#### 3.4.6. Diffusion Models

Diffusion models are generative models that use a forward process to progressively introduce noise into training samples and a reverse process to learn how to eliminate this noise in order to learn a data distribution [48]. The model begins with random noise during inference and iteratively denoises it into a future trajectory conditioned on the observed scene context. In trajectory prediction, this allows the model to generate multiple plausible future motions instead of a single deterministic output, which is useful because vehicle behavior is inherently uncertain and often multimodal [49], [50].

TrajDiffuse is a recent development in trajectory prediction that applies diffusion modeling. It generates accurate and diverse single-agent trajectory predictions while maintaining consistency with environmental constraints [51]. MotionDiffuser extends this concept to multi-agent interaction scenarios [52]. For several interacting agents, it uses a conditional diffusion process to produce permutation-invariant joint trajectories. To increase computational efficiency, it incorporates a dimensionality reduction method based on Principal Component Analysis (PCA).

#### 3.4.7. Other Learning-Based Behavior Modeling Approaches

Maximum Entropy Inverse Reinforcement Learning (IRL) is a learning-based behavior modeling approach that infers a latent reward function from demonstrations and models behavior as a probabilistic distribution over trajectories [53], [54]. IRL aims to recover the latent reward or cost function that a human driver may implicitly optimize during driving, rather than directly mapping states to actions. Without taking into consideration differences in real human behavior, classical IRL assumes that the expert follows an optimal strategy for an uncertain reward [55]. However, this can lead to ambiguity, since multiple reward functions may explain the same observed behavior. To address this, Maximum Entropy Inverse Reinforcement Learning (MaxEnt IRL) introduces the principle of maximum entropy [54]. Instead of selecting a single deterministic explanation, it learns the maximum-entropy distribution over trajectories that remains consistent with the demonstrated feature statistics. This makes the model better suited to capturing uncertainty and variability in human driving behavior. Recent work has also applied MaxEnt IRL to autonomous driving trajectory planning and interaction-aware decision making, for example in merging scenarios [56].

### 3.5. Frameworks for Autonomous Decision Making

#### 3.5.1. Game-Theoretic Models for Mixed-Traffic Interactions

**Level-k Reasoning** Level-k models assume that different drivers operate at different reasoning levels. A level-0 driver is nonstrategic and follows a fixed, non-strategic policy like a standard car-following policy. A level-1 driver best responds to the forecasted behavior of level-0 drivers, a level-2 driver responds to level-1 behavior, and so on [57].

**Stackelberg Games** In Stackelberg games, AV is often modeled as the leader and the other traffic participants as followers [58]. The leader chooses its action while anticipating the followers' best responses. Correspondingly, each follower chooses a strategy that maximizes its own reward given the leader's decision. The resulting solution is commonly formulated in terms of a Stackelberg equilibrium. This framework has been applied, for example, to merging and lane-changing scenarios where a self-driving car interacts with a human driver [59]. For AVs, Stackelberg formulations are well suited because they align with the idea of the AV as a proactive but cautious decision-maker.

**Bayesian Games** Bayesian games model interactions where agents do not have full knowledge of each other's characteristics, such as driving style, reaction time, or capability level. Each agent maintains probabilistic beliefs over the other agent's "type" and chooses actions that maximize their expected gain under this uncertainty [60]. This framework is appealing to mixed-traffic risk assessment, where behavioral variability and environmental uncertainty play a crucial role.

#### 3.5.2. Partially Observable Markov Decision Process

Although not a game-theoretic model, Partially Observable Markov Decision Processes (POMDPs) are often discussed alongside strategic frameworks because they explicitly reason about hidden intentions and uncertain environment states through belief updates. In a POMDPs, the AV maintains a probability distribution over latent variables, such as driver intent, and updates this belief through Bayesian filtering. Prior work has applied POMDPs to interactive driving scenarios, where accounting for uncertainty in other road users' behavior is essential for safe and anticipatory maneuver planning [61]. However, POMDP-based planners remain computationally demanding, which restricts their use in real-time autonomous driving.

#### 3.5.3. Risk Modeling in Autonomous Decision Making

In autonomous driving, risk is often characterized using criticality indicators that estimate how close a traffic interaction is to becoming unsafe. Many existing approaches rely on surrogate safety measures derived from the relative motion between the ego vehicle and surrounding traffic. In particular, time-based indicators such as Time to Collision (TTC)

### 3. State of the Art

and Time Headway (THW) remain widely used, especially for collision risk estimation and longitudinal safety assessment [62], [63].

Such indicators are frequently incorporated into planning and collision-avoidance systems as safety constraints or risk terms. However, they are usually most suitable for specific interaction types, such as longitudinal following or merging, and they do not always capture complex multi-vehicle interactions well [62], [63].

To address these limitations, Nyberg et al. (2021) proposed a risk measure derived directly from a formal safety specification rather than relying only on surrogate indicators [64]. Their method computes both the probability that the AV violates a safety rule and the expected severity of that violation under uncertainty. This severity-aware violation measure is then incorporated into the planner, enabling the decision maker to select trajectories that explicitly balance risk and driving progress [64].

## 3.6. Comparative Assessment of Approaches for HDV Behavior Modeling and Prediction

Rule-based models are simple, interpretable, and computationally efficient. Their parameters have a direct physical meaning, and they are widely used in microscopic traffic simulation and as baselines for evaluating more complex models. However, they usually ignore the uncertainty and unpredictability present in human decision-making and portray driving as a mostly deterministic activity. As a result, they are limited in their ability to capture latent intent and behavioral adaptation in complex real-world scenarios.

Probabilistic models address these limitations by treating driver behavior as a stochastic process. They can represent multiple possible maneuvers and quantify uncertainty over future actions. Within this family, DBNs offer a useful compromise between expressive power and interpretability, since they allow richer dependency structures than simpler models such as HMMs while retaining an explicit probabilistic representation of latent variables. This makes them attractive for HDV modeling, especially when uncertainty, partial observability, and belief updating are important. However, probabilistic models still require careful model design and representative data, and simpler variants may rely on restrictive conditional independence or parametric assumptions.

Learning-based models, especially deep neural architectures, currently form a major part of the state of the art in trajectory prediction on large benchmark datasets. They can capture complex nonlinear dependencies, model interactions between multiple agents, and generate multimodal futures through latent-variable or generative formulations. However, these benefits come at a cost. First, they often depend on large, diverse, annotated datasets, which may be limited for the specific mixed-traffic scenarios. Second, their internal representations are often difficult to interpret, making it harder to relate model outputs to driver intent, uncertainty sources, or downstream risk reasoning.

Given the goals of this thesis, probabilistic models, and DBNs in particular, offer the most suitable compromise. They are expressive enough to encode latent driver states

### *3. State of the Art*

and maneuver tendencies. At the same time, they remain interpretable and compatible with Bayesian belief updating and downstream risk-aware reasoning. Rule-based models remain useful as baselines and for scenario construction, whereas DL models provide an important reference for the current trajectory prediction frontier.

## 4. Methodology

This chapter presents the methodology used to model and predict short-horizon HDV maneuvers in highway traffic. Starting from raw vehicle trajectories in the highD dataset, the pipeline constructs a vehicle-centric representation, extracts interaction and risk-related features, summarizes them over short time windows, and uses the resulting observation sequences to train and evaluate the proposed DBN.

In highway traffic, a similar traffic situation can lead to several plausible driver responses. For example, when approaching a slower vehicle, a driver may brake and continue following or change lanes to overtake, depending on the surrounding traffic context. The resulting maneuver is therefore uncertain. To capture this uncertainty, the proposed method adopts a probabilistic model.

The goal is to predict a probability distribution over discrete high-level action classes, such as acceleration, braking, lane changes, and stable following, rather than an exact future trajectory. To achieve this, the observation space combines ego-motion variables with contextual information from surrounding vehicles and risk-related quantities.

The remainder of this chapter describes the overall framework in Section 4.1, the dataset in Section 4.2, the preprocessing and feature-engineering pipeline in Section 4.3, the proposed DBN model in Section 4.4, the training procedure in Section 4.5, the inference and evaluation procedure in Section 4.7, and the semantic interpretation of latent states in Section 4.6.

### 4.1. Proposed Framework

The framework converts raw highD trajectory data into temporal observations for action prediction. Its main stages are data preparation, DBN training, and inference.

The pipeline starts from frame-level trajectories and transforms them into a consistent vehicle-centric representation. Based on the ego vehicle and its local traffic environment, additional features are derived to capture interaction context, lane-related information, and risk-related properties such as THW and TTC. These steps provide a compact description of the current driving situation beyond the ego vehicle.

Action prediction depends on both the current traffic state and its recent evolution. For this reason, frame-level signals are aggregated over short sliding windows to form a window-based sequence representation. Each window becomes one observation for a DBN with two discrete latent variables: a style state and an action state. The engineered window features are linked to these latent states through the emission model, while temporal behavior is captured by the state transition model.

## 4. Methodology

Model parameters are estimated from the training data using an EM-based learning procedure. Each windowized observation sequence corresponds to a single vehicle trajectory. To prevent leakage across the training, validation, and test sets, the data are partitioned at the vehicle level into disjoint subsets. Specifically, the split contains 35,318 training vehicles (70%), 5,045 validation vehicles (10%), and 10,092 test vehicles (20%). The training set is used for estimating model parameters, the validation set is used to monitor validation log-likelihood and to apply early stopping during EM training, and the test set is reserved for the final evaluation. After training, inference is performed on the held-out test set to estimate the posterior distribution over latent states and to predict the distribution over future maneuvers.

An overview of the complete framework is shown in Figure 4.1. The individual stages are described in detail in the following sections.

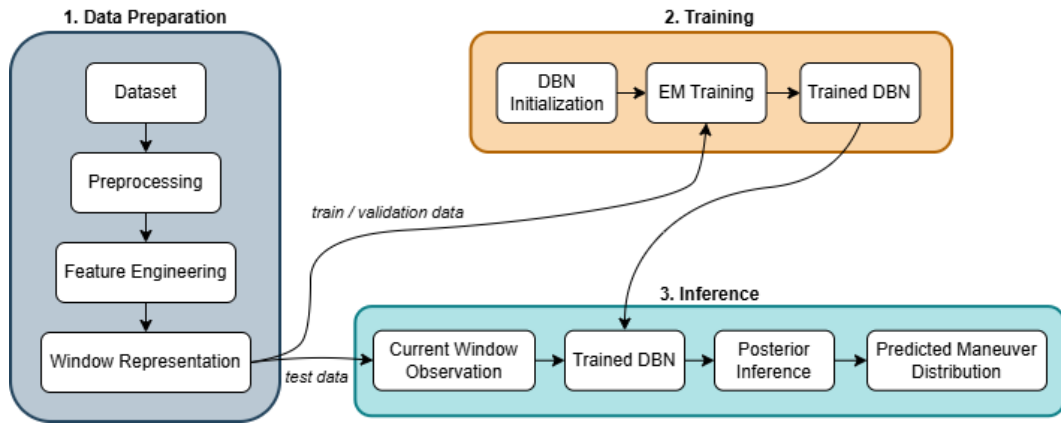


Figure 4.1.: Overview of the proposed framework

### 4.2. Dataset

This thesis uses the highD dataset [9], a naturalistic highway-traffic dataset recorded from drone videos on German highways. Its aerial perspective provides frame-level trajectories together with surrounding-vehicle information, making it well suited for the analysis of human driving behavior in highway scenarios.

The full highD dataset contains 60 recordings collected at several highway locations. In this work, only the first 25 recordings with three lanes in each driving direction are used. This restriction provides a more uniform road geometry and traffic setting across the selected subset, which supports more consistent behavior modeling. Figure 4.2 shows an example traffic scene from the selected subset and illustrates the multi-lane highway layout together with the surrounding-vehicle context available for each target vehicle.

## 4. Methodology

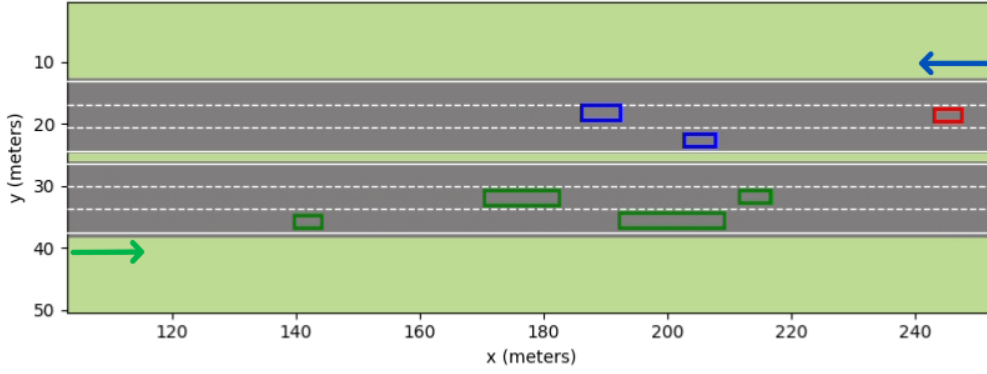


Figure 4.2.: Example traffic scene from a selected highD subset. The red box denotes the ego vehicle, while the blue and green boxes represent other vehicles traveling in the two opposite driving directions.

For each vehicle, highD provides frame-level kinematic variables such as longitudinal and lateral position, velocity, and acceleration. It also provides lane-related information, driving-direction labels, and identifiers for neighboring vehicles in 8 surrounding positions. In addition, the dataset provides traffic-safety indicators such as TTC and THW, which support the construction of interaction and risk-related features. These signals form the basis for the preprocessing and feature-engineering pipeline described in Section 4.3.

For the selected subset, 50,543 vehicle trajectories are available after loading the chosen recordings. With a window length of  $W = 50$  frames, 50,455 trajectories satisfy  $T \geq W$ , while only 88 are shorter than the required window length. Most trajectories are long enough for temporal modeling: half have about 349 frames, and 80% fall between 280 and 466 frames. This indicates that the selected subset is sufficiently large and temporally rich for constructing window-based observation sequences.

### 4.3. Data Preparation and Preprocessing

#### 4.3.1. Preprocessing Pipeline

The raw highD files are first merged into a single per-frame representation that combines trajectory data with vehicle-level and recording-level metadata. Since the dataset contains multiple recordings, each trajectory is identified by the pair  $(\text{recording\_id}, \text{vehicle\_id})$ . Vehicle-center coordinates are then computed from the original bounding-box positions to provide a consistent geometric reference.

The selected recordings contain traffic moving in two opposite roadway directions. In the raw data, forward longitudinal motion has opposite signs across these directions, and the same inconsistency affects related kinematic variables. Therefore, the raw signals from highD cannot be used directly as a single consistent representation. A set of plots

#### 4. Methodology

illustrating this effect is provided in Appendix A. To resolve this issue, a vehicle-centric normalization step is applied using the driving-direction label. This transformation makes the sign conventions for longitudinal and lateral motion consistent across both traffic directions. As a result, the same maneuver is represented with the same sign convention regardless of the original roadway direction.

After these normalization steps, the processed dataframe provides a consistent per-frame representation that is used for feature engineering and subsequent windowization.

##### 4.3.2. Data Analysis for Feature Design

Before defining the final observation vector, an exploratory analysis of the highD variables was carried out to understand the statistical behavior of the available signals and to identify informative features. The analysis focused mainly on kinematic variables, namely longitudinal and lateral acceleration ( $a_x, a_y$ ), longitudinal and lateral velocity ( $v_x, v_y$ ), and simple lane-change indicators.

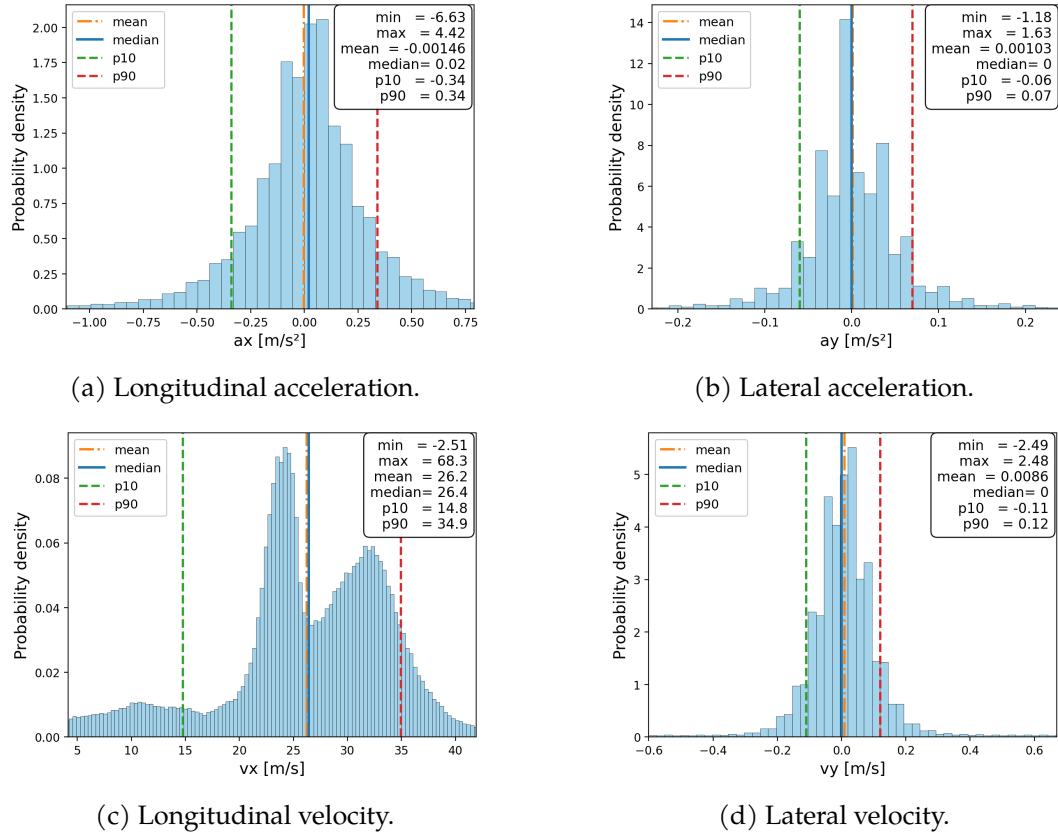


Figure 4.3.: Global distributions of the main kinematic variables.

As shown in Figure 4.3a, the distribution of  $a_x$  is centered near zero but has visible tails on both sides, indicating that most frames correspond to near-steady motion, while

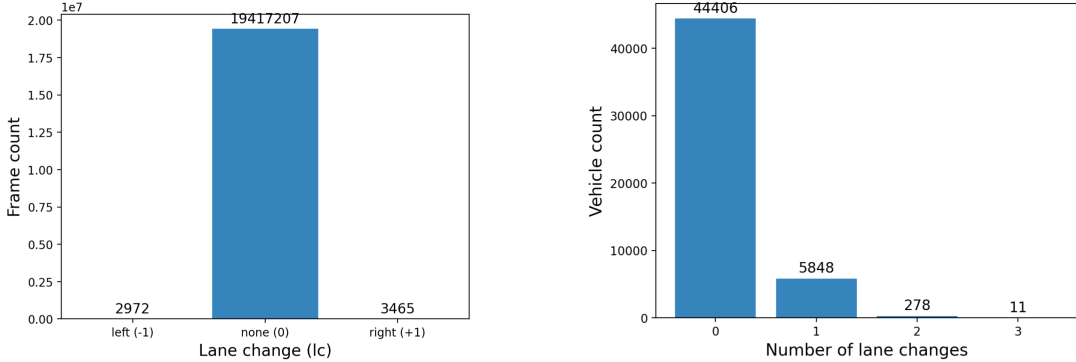


#### 4. Methodology

stronger acceleration and braking occur less often. The longitudinal velocity distribution in Figure 4.3c spans a broad range of values and exhibits a multimodal structure. This is mainly caused by differences between vehicle classes, with trucks concentrated at lower speeds and cars occupying a wider and generally higher-speed range. These class-dependent differences show that raw continuous features can have different natural ranges depending on the vehicle type. Class-wise kinematic distributions for cars and trucks are provided in Appendix B. For this reason, continuous variables are normalized class-wise before training, as described in Section 4.5.

On the other hand, the lateral variables are much more concentrated around zero. In Figure 4.3b and Figure 4.3d, both  $a_y$  and  $v_y$  show sharp peaks near zero, which is expected because most vehicles remain within their lane for most of the time. However, the non-zero tails still reflect lateral adjustments and lane-change-related motion. Together, these observations support the inclusion of both longitudinal and lateral kinematic variables, since the former describe speed adaptation and car-following behavior, whereas the latter provide cues for lateral maneuvers.

In addition to the raw frame-level signals, the variability of each kinematic variable was also analyzed at trajectory level. For each vehicle trajectory, the standard deviation of  $a_x$ ,  $a_y$ ,  $v_x$ , and  $v_y$  was computed over time, producing one variability value per trajectory. Unlike the raw kinematic distributions, these plots do not describe the absolute signal values, but rather how strongly the kinematic signals fluctuate within individual trajectories. The resulting distributions are shown in Figure 4.5a to Figure 4.5d.



(a) Raw lane-change indicator at frame level.

(b) Number of lane changes observed per trajectory.

Figure 4.4.: Lane-change sparsity analysis in the highD data.

The variability analysis shows that longitudinal motion is generally more dynamic than lateral motion. In Figure 4.5a and Figure 4.5c, both  $\text{std}(a_x)$  and  $\text{std}(v_x)$  exhibit broader distributions with long right tails, indicating that some vehicles undergo substantial longitudinal speed and acceleration changes. In contrast, Figure 4.5b shows that  $\text{std}(a_y)$  remains concentrated at low values, which is consistent with lateral acceleration being close to zero except during lane changes. A similar pattern appears in Figure 4.5d,

#### 4. Methodology

although the visible tail suggests that some trajectories contain stronger lateral motion. Class-wise trajectory-level variability distributions are provided in Appendix B. These results support the use of variability-related features, since instantaneous values alone do not capture whether a trajectory is stable or dynamically changing over time.

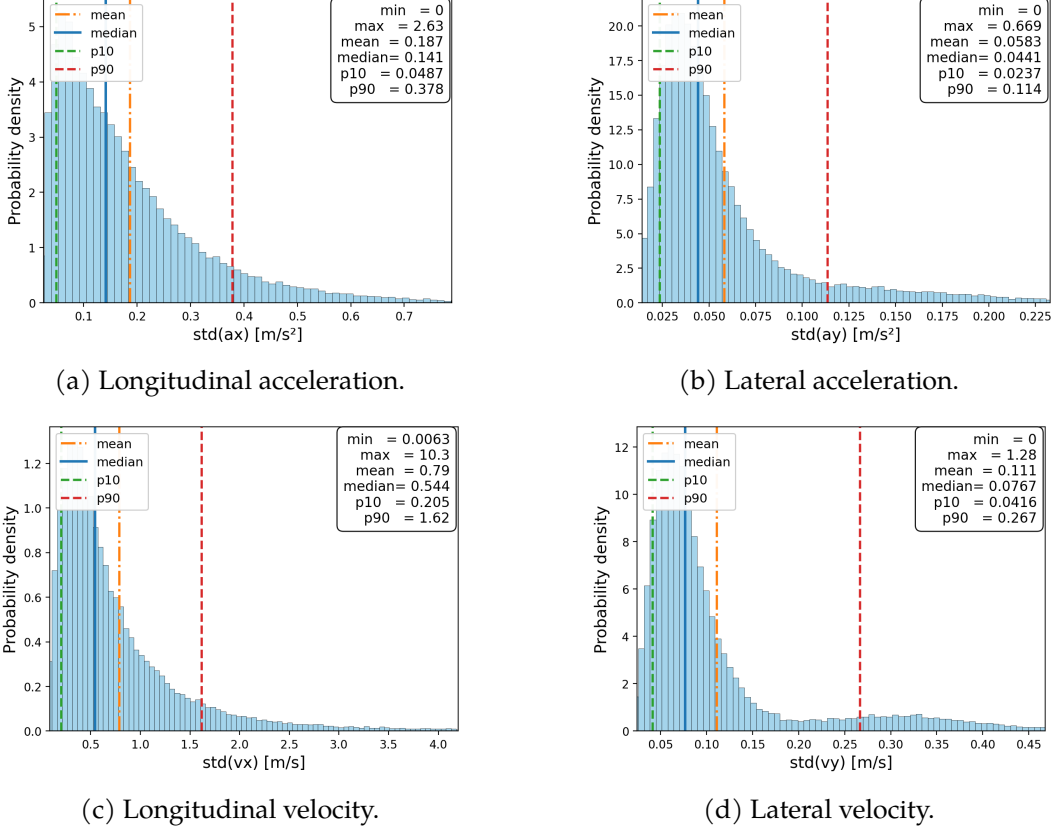


Figure 4.5.: Global trajectory-level variability distributions of the main kinematic variables.

The lane-change analysis further highlights an important property of the dataset. As shown in Figure 4.4a, almost all frames belong to the lane-keeping class, while left and right lane changes occur only rarely. The same sparsity appears at trajectory level in Figure 4.4b, where most vehicles perform no lane change and only a small fraction perform one or more. This imbalance shows that lane-change events are very rare compared to straight driving. It also shows that the raw lane-change indicator alone is not sufficient as a learning signal, especially because in highD the lane change is marked only at the frame where the vehicle crosses the lane boundary. This motivated the use of temporal aggregation together with contextual kinematic and interaction features.

Overall, the data analysis led to three main conclusions. First, longitudinal and lateral kinematics provide complementary information for maneuver prediction. Second,

trajectory-level variability contains useful information beyond single-frame values. Third, feature scale and class-dependent distribution differences must be handled carefully during preprocessing. These findings guided the final feature design for the proposed DBN.

### 4.3.3. Feature Engineering

In the feature-engineering stage, the available neighboring positions include the front, rear, left-front, left-side, left-rear, right-front, right-side, and right-rear slots. For each available neighbor, binary existence flags are created and relative kinematic quantities are derived in the form of relative longitudinal distance, relative lateral distance, relative longitudinal velocity, and relative lateral velocity. Missing neighbors remain marked through the corresponding existence indicators.

In the final observation vector, only the subset of neighboring vehicles shown in Figure 4.6 is retained. This choice makes the model focus on the vehicles that are most relevant for immediate vehicle-following constraints in the current lane. It also captures the local traffic state of the adjacent lanes while keeping the feature space compact and less complex. Rear and rear-diagonal neighbors may also provide useful information, but they are excluded here to keep the observation space lower-dimensional.

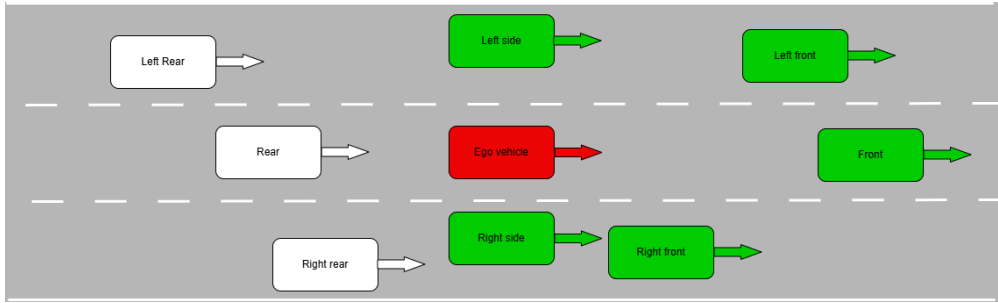


Figure 4.6.: Selected neighbors considered in the observation are marked in green.

In addition to interaction features, lane-related and risk-related quantities are derived. Distances to the left and right lane boundaries are computed from the recording metadata and the ego-vehicle center position. The lane-change indicator is obtained from changes in lane ID and represented as a ternary variable corresponding to lane change to left, no lane change, and lane change to right. The front-interaction safety signals provided by highD are mapped to the variables `front_thw` and `front_ttc`. Furthermore, ego-vehicle jerk is computed from the frame-to-frame change in longitudinal and lateral acceleration.

Based on the preprocessing and feature-engineering steps described above, the final frame-level observation was defined as the compact feature set listed in Table 4.1. The selected variables describe ego motion, lane-change evidence, lane geometry, local interaction context, and front-interaction risk. This frame-level representation serves as the basis for the subsequent window-based observation model.

Table 4.1.: Frame-level feature groups used in the preprocessing pipeline

| Feature group        | Variables                                                                    | Purpose                                                                  |
|----------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Ego kinematics       | $v_x, v_y, a_x, a_y, \text{jerk}_x, \text{jerk}_y$                           | Describe recent ego motion and control variation                         |
| Lane-change evidence | $lc$                                                                         | Encode left / none / right lane-change evidence                          |
| Lane geometry        | $d_{\text{left\_lane}}, d_{\text{right\_lane}}$                              | Represent lateral position within the current lane                       |
| Front risk           | $\text{front\_thw}, \text{front\_ttc}$                                       | Describe front-interaction criticality                                   |
| Neighbor existence   | Existence indicators for the selected front and adjacent-lane neighbor slots | Indicate whether relevant surrounding vehicles are present               |
| Relative interaction | For each selected neighbor slot: $dx, dy, dv_x, dv_y$                        | Capture relative position and motion with respect to surrounding traffic |

#### 4.3.4. Window-Based Representation

The proposed model operates on short temporal windows rather than on individual frames. Each vehicle trajectory is therefore converted into a sequence of overlapping windows, as illustrated in Figure 4.7. This allows the model to capture short-term temporal dynamics and local interaction context instead of relying only on instantaneous frame values.

Let  $W$  denote the window length and  $S$  the stride between consecutive windows. In this work,  $W = 50$  frames and  $S = 10$  frames are used. Since highD is recorded at 25 Hz, each window covers exactly 2.0 s of history, while consecutive windows start every 0.4 s. A 2 s observation horizon is also consistent with prior highway maneuver prediction work on highD [65]. For a trajectory of length  $T$  frames, overlapping windows are extracted every  $S$  frames as long as  $T \geq W$ .

Each window is then aggregated into one observation vector. Depending on the variable, the aggregation uses the most recent value, a minimum over the window, a temporal slope, sign-based fractions, percentile-based summaries, binary event indicators, or fractions of valid or available frames. For neighbor-dependent variables, the summaries are computed only when the corresponding neighbor is present. The complete list of window-level features used in the final observation vector is provided in Appendix C.

## 4. Methodology

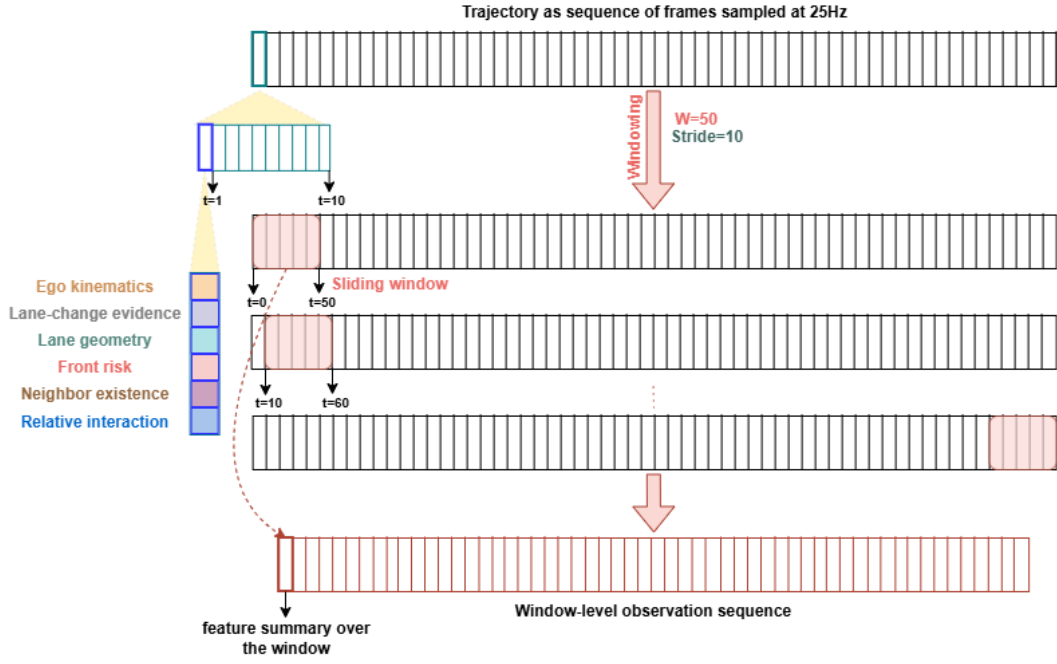


Figure 4.7.: Illustration of the sliding-window representation used in this work.

### 4.4. Proposed DBN Model

This section formalizes the structure of the DBN. The style state  $S_t$  captures a broader driving regime, while the action state  $A_t$  represents the maneuver at the current time step. The observation vector  $O_t$  corresponds to the window-level features.

#### 4.4.1. Latent State Structure

The model uses a factorized latent representation. As shown in Figure 4.8, the style state evolves as its own Markov chain, while the action state depends on both the previous action state and the current style state.

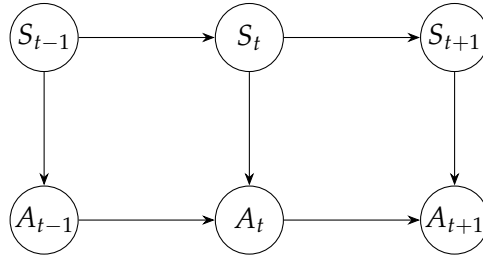


Figure 4.8.: Factorized latent structure of the proposed DBN.

#### 4.4.2. Transition Model

The transition model specifies how the latent states evolve from one time step to the next. The initial distribution is modeled as:

$$p(S_1 = s_1) = \boldsymbol{\pi}_{s_1}^{(s)} \quad (4.1)$$

$$p(A_1 = a_1 \mid S_1 = s_1) = \boldsymbol{\Pi}_{s_1, a_1}^{(a)} \quad (4.2)$$

For later time steps, with  $t=2, \dots, T$ , the transition model is factorized as:

$$p(S_t = s_t \mid S_{t-1} = s_{t-1}) = \boldsymbol{A}_{s_{t-1}, s_t}^{(s)} \quad (4.3)$$

$$p(A_t = a_t \mid A_{t-1} = a_{t-1}, S_t = s_t) = \boldsymbol{\mathcal{A}}_{s_t, a_{t-1}, a_t}^{(a)} \quad (4.4)$$

Here,  $\boldsymbol{\pi}^{(s)}$  is the initial style-probability vector,  $\boldsymbol{\Pi}^{(a)}$  is the conditional initial action matrix,  $\boldsymbol{A}^{(s)}$  is the style transition matrix, and  $\boldsymbol{\mathcal{A}}^{(a)}$  is the action transition tensor.

This factorization reflects the assumption that the style state evolves more gradually, while the current maneuver depends on both the recent maneuver history and the current driving regime.

#### 4.4.3. Emission Model Variants

At each time step, the emission model specifies the conditional probability of the window-level observation vector given the latent state pair  $(S_t, A_t)$ . Two alternative emission formulations are considered in this work, as illustrated in Figure 4.9. Both share the same latent transition structure and differ only in how the observation model is parameterized. In both cases, the observation vector is split into continuous and binary components, modeled by diagonal Gaussian and Bernoulli distributions, respectively.

##### Product-of-Experts Emission

In the Product of Experts (PoE) formulation, the observation likelihood is decomposed into two experts: one conditioned on the style state and one conditioned on the action state. Their contributions are combined multiplicatively and then normalized:

$$p(O_t \mid S_t = s_t, A_t = a_t) = \frac{p_s(O_t \mid S_t = s_t; \theta_{s_t}) p_a(O_t \mid A_t = a_t; \theta_{a_t})}{Z_{s_t, a_t}} \quad (4.5)$$

Here,  $\theta_{s_t}$  denotes the parameters of the style expert associated with style state  $s_t$ , and  $\theta_{a_t}$  denotes the parameters of the action expert associated with action state  $a_t$ . Each parameter set contains the Gaussian parameters for the continuous features and the Bernoulli parameters for the binary features. The term  $Z_{s_t, a_t}$  is the partition function that normalizes the product of experts for state pair  $(s_t, a_t)$ , ensuring a valid probability distribution, as in standard PoE formulations [66].

#### 4. Methodology

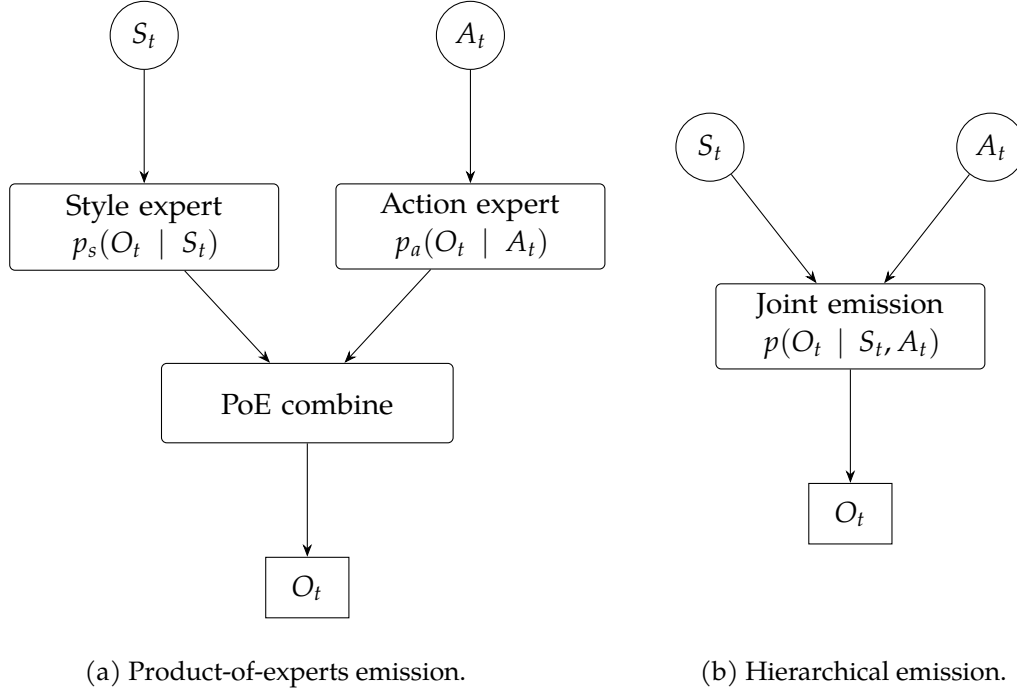


Figure 4.9.: Emission model variants considered in the proposed DBN.

#### Hierarchical Emission

In the hierarchical formulation, a single emission distribution is learned directly for each joint latent-state pair:

$$p(O_t | S_t = s_t, A_t = a_t) = p(O_t | S_t = s_t, A_t = a_t; \theta_{s_t, a_t}) \quad (4.6)$$

Here,  $\theta_{s_t, a_t}$  denotes the parameters of the emission distribution associated with the joint state pair  $(s_t, a_t)$ . As in the PoE case, these parameters include the Gaussian terms for the continuous features and the Bernoulli terms for the binary features. This formulation gives the model greater flexibility for each joint latent-state pair, at the cost of a larger number of emission parameters.

#### 4.4.4. Modeling Assumptions

The proposed DBN is modeled based on the following assumptions:

- **First-order temporal dependency:** The style state  $S_t$  depends only on the previous style state  $S_{t-1}$ . The action state  $A_t$  depends only on the previous action state  $A_{t-1}$  and the current style state  $S_t$ . This gives a first-order Markov structure for the latent process.
- **Observation dependence on the current latent state only:** The observation vector  $O_t$  depends only on the latent variables at the same time step. Once  $S_t$  and  $A_t$  are

## 4. Methodology

known, the current observation is assumed to be conditionally independent of past observations and past latent states.

- **Window-level observation representation:** Each model time step corresponds to one windowed observation vector rather than one raw frame from the dataset.
- **Split into continuous and binary features:** The observation vector is divided into continuous and binary components. Among the window features, the lane-change indicator variables are binary, while the remaining features are treated as continuous.
- **Naive Bayes-style conditional independence:** Given the relevant latent state pair, features are assumed conditionally independent within each observation.
- **Diagonal Gaussian assumption for continuous features:** Continuous features are modeled using diagonal Gaussian distributions. This corresponds to conditionally independent continuous feature dimensions given the latent states.
- **Bernoulli assumption for binary features:** Binary features are modeled using Bernoulli distributions. The binary dimensions are treated as conditionally independent given the latent states.
- **Missing-feature handling:** Some features may be undefined at certain time steps, especially interaction-related quantities when neighboring vehicles are absent. Such values are treated as missing and are excluded from the corresponding likelihood terms through masking, rather than being interpreted as informative values.

### 4.5. Model Training

As described in Section 4.1, the windowized vehicle sequences are divided into training, validation, and test sets. This section explains how the observation sequences are prepared, how the model parameters are initialized, and how the proposed DBN is trained and monitored during learning.

#### 4.5.1. Feature Scaling

Before training, the continuous window-level features are normalized to make them comparable in scale. This is necessary because the observation vector contains different physical quantities, such as velocity, acceleration, lane-relative distances, and interaction-related measures, which naturally have different numerical ranges. Without scaling, features with larger magnitudes could dominate the likelihood computation and make parameter estimation less stable.

In the implemented pipeline, z-score scaling statistics, namely the mean and standard deviation, are computed from the training observations and then reused unchanged



## 4. Methodology

for the training, validation, and test splits. The same transformation is also applied during inference to any new observation sequence. Only continuous features are normalized in this way. The binary lane-change indicators, namely `lc_left_present` and `lc_right_present`, are kept in their original form. These scaling values are fixed pre-processing quantities and are not part of the learned emission model.

### 4.5.2. Parameter Initialization

Before iterative learning begins, the model parameters must be assigned initial values. These include the initial-state and transition probabilities of the DBN, as well as the parameters of the emission model. This step is important because the EM algorithm is sensitive to its starting point and may otherwise converge to a poor local optimum.

In the implemented framework, the emission model is initialized from the scaled training observations using K-means clustering. For the continuous part of the observation vector, the cluster centers and cluster variances are used as the initial Gaussian emission parameters. For the binary part, the initial Bernoulli probabilities are derived from the clustered observations. This provides a more informative starting point than a purely random assignment.

The initial latent-state distribution and the transition probabilities are set separately. Depending on the experiment, different schemes can be used, including uniform or sticky initialization. In the sticky case, self-transitions are assigned higher initial probability than transitions to different latent states.

For reproducibility, the transition CPD initializations used in this work are:

$$\text{uniform: } A_{s,s'}^{(s)} = \frac{1}{K_S} \quad \mathcal{A}_{s,a,a'}^{(a)} = \frac{1}{K_A} \quad (4.7)$$

$$\text{sticky (style CPD): } A_{s,s}^{(s)} = \rho \quad A_{s,s'}^{(s)} = \frac{1-\rho}{K_S-1} \quad (s' \neq s) \quad (4.8)$$

with  $\rho = 0.8$  in the implemented configuration. In the sticky-vs-uniform CPD comparison, the action-transition CPD  $\mathcal{A}^{(a)}$  and the initial latent-state distribution are kept uniform so that only the style-transition initialization differs.

### 4.5.3. EM-Based Training

After initialization, the DBN is trained iteratively on the windowized observation sequences. At each iteration, the current model is applied to all training sequences to compute the posterior state probabilities and expected transition counts. These quantities are accumulated over all sequences and then used to update the model parameters. The procedure is repeated until the stopping criterion is met.

The re-estimation of the initial-state and transition parameters follows the EM update equations introduced in Section 2.4. The emission-parameter update depends on the chosen observation model. The theoretical derivations are given in Appendix E.

For the initial latent-state distribution, two variants are used in this work:

---

**Algorithm 1** EM-based training procedure

---

**Require:** scaled training and validation sequences, initialized model parameters

- 1: **repeat**
  - 2:   **for** each training sequence **do**
  - 3:     compute emission log-likelihoods
  - 4:     run forward-backward inference
  - 5:     accumulate posterior state probabilities and expected transition counts
  - 6:   **end for**
  - 7:   update initial-state, transition, and emission parameters
  - 8:   evaluate the current model on the validation set
  - 9: **until** stopping criterion is satisfied
- 

- Uniform (fixed) initial distribution:

$$\pi_s^{(s)} = \frac{1}{K_S}, \quad (4.9)$$

$$\Pi_{s,a}^{(a)} = \frac{1}{K_A} \quad (4.10)$$

for all valid  $s$  and  $a$ . These parameters are kept fixed during training.

- Learned initial distribution: using the first-step posterior of each sequence, the M-step update is

$$\pi_s^{(s)} = \frac{\sum_i \sum_{a=1}^{K_A} \gamma_1^{(i)}(s, a) + \alpha_\pi}{\sum_{s'=1}^{K_S} \sum_i \sum_{a=1}^{K_A} \gamma_1^{(i)}(s', a) + K_S \alpha_\pi}, \quad (4.11)$$

$$\Pi_{s,a}^{(a)} = \frac{\sum_i \gamma_1^{(i)}(s, a) + \alpha_\pi}{\sum_{a'=1}^{K_A} \sum_i \gamma_1^{(i)}(s, a') + K_A \alpha_\pi} \quad (4.12)$$

where  $\gamma_1^{(i)}(s, a) = p(S_1 = s, A_1 = a \mid O_{1:T_i}^{(i)})$ , and  $\alpha_\pi \geq 0$  is an optional Dirichlet pseudo-count [10], [13].

In the hierarchical emission model, one emission distribution is learned directly for each joint latent-state pair  $(S_t, A_t)$ . After the E-step, the posterior probabilities of the joint states are used as weights for re-estimating the emission parameters. For the continuous features, posterior-weighted means and diagonal variances are computed. For the binary features, posterior-weighted Bernoulli probabilities are computed. Thus, the hierarchical emission parameters can be updated in closed form.

In the PoE emission model, because of the normalization term, the style and action emission parameters are coupled during learning. As a result, the M-step does not reduce to a closed-form update as in the hierarchical case. Instead, the PoE emission parameters are updated by an inner gradient-based optimization loop within each EM iteration. In

#### 4. Methodology

the implementation, the inner loop performs a fixed number of gradient-ascent steps on the exact PoE Q-function. For the experiments, the inner-loop learning rate is set to  $3 \times 10^{-3}$  and the number of steps is 20. Gradient norms are clipped for stability, and early stopping is used inside the inner loop when the objective no longer improves. Optional weak quadratic penalties on the Gaussian means, log-variances, and Bernoulli logits can also be enabled to stabilize training, although the main experimental configuration keeps these penalties at zero.

Lane-change-related events are much rarer than lane-keeping behavior, as shown in Figure 4.4. For this reason, additional weighting was investigated during training so that rare observations could have a stronger influence on learning. This follows the general idea of cost-sensitive weighting for imbalanced data [67]. In this case, lane-change-active time steps are upweighted using an inverse-frequency rule computed from the training data, with an upper clipping threshold to avoid excessively large weights.

Across the experimental study, three lane-change weighting settings are compared: a baseline without additional weighting (lc-none), and two weighted variants (lc-A and lc-B). Their operational difference is summarized in Table 4.2.

Table 4.2.: Lane-change weighting settings used in the experiments.

| Setting | Definition                                                                                                                            |
|---------|---------------------------------------------------------------------------------------------------------------------------------------|
| lc-none | No additional lane-change weighting is applied.                                                                                       |
| lc-A    | Lane-change weights are applied directly to emission log-likelihood terms before forward-backward inference.                          |
| lc-B    | Forward-backward inference is run under the original model, then weights are applied only to posterior statistics used in the M-step. |

Since multiplying a log-likelihood term by a positive weight is equivalent to raising the corresponding likelihood term to a power, the first variant, lc-A, can be interpreted as a power-likelihood formulation [68]. As a result, this variant modifies the posterior inference itself.

Let  $\mathcal{I}_{lc}$  denote the set of lane-change indicator feature indices in the observation vector, and define the lane-change activity indicator at time step  $t$  as:

$$c_t = \mathbf{1}[\exists j \in \mathcal{I}_{lc} : O_{t,j} > 0.5] \quad (4.13)$$

With lane-change weight  $w_{lc} > 1$ , the per-time-step training weight is:

$$w_t = 1 + (w_{lc} - 1)c_t, \quad (4.14)$$

so non-lane-change steps have weight 1 and lane-change-active steps have weight  $w_{lc}$ .

In lc-A, this weight is applied before forward-backward by scaling emission log-likelihoods:

$$\widetilde{\log B_t}(s, a) = w_t \log p(O_t \mid S_t = s, A_t = a), \quad (4.15)$$

#### 4. Methodology

which is equivalent to

$$\tilde{B}_t(s, a) = B_t(s, a)^{w_t} \quad (4.16)$$

The second variant, lc-B, is closer to weighted-data EM, in which observations contribute to parameter estimation with different strengths through weighted responsibilities and weighted update equations [69]. In this case, the pass itself remains unweighted, and only the parameter update is weighted.

Here, forward-backward is run with the unweighted likelihood  $B_t(s, a)$ , then the sufficient statistics are reweighted:

$$\tilde{\gamma}_t(s, a) = w_t \gamma_t(s, a), \quad t = 1, \dots, T \quad (4.17)$$

For transition statistics, let  $\xi_t^{(S)}(s, s')$  and  $\xi_t^{(A)}(s, a, a')$  be per-edge posteriors for edges  $t \rightarrow t+1$ ,  $t = 1, \dots, T-1$ . The edge weight is:

$$w_t^{\text{edge}} = \frac{w_t + w_{t+1}}{2} \quad (4.18)$$

and the weighted transition counts become:

$$\tilde{\Xi}^{(S)}(s, s') = \sum_{t=1}^{T-1} w_t^{\text{edge}} \xi_t^{(S)}(s, s'), \quad (4.19)$$

$$\tilde{\Xi}^{(A)}(s, a, a') = \sum_{t=1}^{T-1} w_t^{\text{edge}} \xi_t^{(A)}(s, a, a') \quad (4.20)$$

After each iteration, the model is evaluated on the validation set using the average validation log-likelihood per observation. Training stops when this value no longer improves meaningfully over several successive iterations.

### 4.6. Semantic Interpretation of Latent States

Since the DBN is trained in an unsupervised manner, the learned latent states do not initially correspond to named driver behaviors. A post-hoc semantic analysis is therefore performed to assign interpretable labels to the learned joint states.

#### 4.6.1. Feature-Based Semantic Profiles

The semantic interpretation is based on posterior-weighted feature statistics computed from the original observation features. For a selected feature  $o_t^{(f)}$  and a joint latent state pair  $(s, a)$ , the posterior-weighted mean and standard deviations are computed as:

$$\mu_{s,a,f} = \frac{\sum_n \sum_t \gamma_t^{(n)}(s, a) o_t^{(n,f)}}{\sum_n \sum_t \gamma_t^{(n)}(s, a)} \quad (4.21)$$

## 4. Methodology

$$\sigma_{s,a,f} = \left( \frac{\sum_n \sum_t \gamma_t^{(n)}(s,a) \left( o_t^{(n,f)} - \mu_{s,a,f} \right)^2}{\sum_n \sum_t \gamma_t^{(n)}(s,a)} \right)^{1/2} \quad (4.22)$$

where  $n$  indexes the analyzed vehicle trajectories. These statistics define a feature profile for each joint state  $(s, a)$  and indicate its typical motion, interaction, and risk characteristics.

### 4.6.2. Rule-Based Labeling Procedure

For the selected model, labeling rules are defined from the posterior-weighted feature profiles and domain knowledge. They map latent-state characteristics to interpretable behavior classes, including free flow, braking, acceleration, lane change, and following.

For test-set evaluation, these rules are applied to window-level observations. If a window satisfies multiple conditions, the predefined priority order resolves ambiguities.

The complete semantic mapping rules, thresholds, composite rule definitions, and the full priority order are provided in Appendix D.

If no rule is satisfied, the window is labeled as unknown and encoded as  $z = -1$ . Unknown-labeled windows are excluded from predictive scoring; the remaining labels are used as reference targets for causal online prediction evaluation.

## 4.7. Inference and Evaluation

After training, the model is applied to unseen sequences from the held-out test split. Evaluation is carried out in two complementary settings. First, full-sequence inference is used to assess how well the trained DBN explains unseen observation sequences. Second, causal online inference is used to predict the next latent state from observations available up to the current time step.

Across the experimental study, multiple model configurations are first compared using generative evaluation metrics on unseen observation sequences. Based on this comparison, a smaller set of candidate models is then selected for predictive evaluation in a strictly causal online setting.

For full-sequence analysis, inference is performed over the complete observation sequence to obtain the smoothed joint posterior distribution  $p(S_t, A_t \mid O_{1:T})$ , for each time step  $t$ . Since this posterior is conditioned on the full sequence, including future observations relative to time  $t$ , it is not a causal prediction setting. Instead, it is used to evaluate how well the model explains unseen data under the learned probabilistic structure. A better generative fit indicates that the model captures the temporal and contextual structure of driver behavior more faithfully.

In this work, generative fit is assessed using likelihood-based measures such as average log-likelihood per time step and Bayesian Information Criterion (BIC), together with posterior-based diagnostics such as state occupancy.

#### 4. Methodology

For predictive evaluation, inference is performed online. At time step  $t$ , the current observation  $O_t$  is used to update the filtered belief  $p(S_t, A_t \mid O_{1:t})$ , and the transition model is then used to compute the one-step-ahead predictive distribution. Using the transition factorization from Section 4.4.2, this is given by:

$$p(S_{t+1} = s', A_{t+1} = a' \mid O_{1:t}) = \sum_{s_t} \sum_{a_t} p(S_t = s_t, A_t = a_t \mid O_{1:t}) p(S_{t+1} = s' \mid S_t = s_t) p(A_{t+1} = a' \mid A_t = a_t, S_{t+1} = s') \quad (4.23)$$

The one-step prediction is then taken as the Maximum A Posteriori (MAP) estimate of the joint latent state:

$$(\hat{S}_{t+1}, \hat{A}_{t+1}) = \arg \max_{s, a} p(S_{t+1} = s, A_{t+1} = a \mid O_{1:t}) \quad (4.24)$$

During online evaluation, each test trajectory is processed sequentially. The first five window-based observations are excluded from scoring so that the model can update its initial belief state based on the recent context of the current vehicle. After this warmup period, the model outputs a one-step-ahead prediction at each valid time step, using only observations available up to that time step (strictly causal inference). For horizon-based metrics, the forecast horizon is defined as the next five future observations (2.0 s), and prediction times that do not provide this full horizon are excluded from scoring. These predictions are compared with rule-based reference labels, whose generation and relationship to model predictions are detailed in Section 4.7.2. Figure 4.10 illustrates the causal prediction and evaluation procedure used in this work.

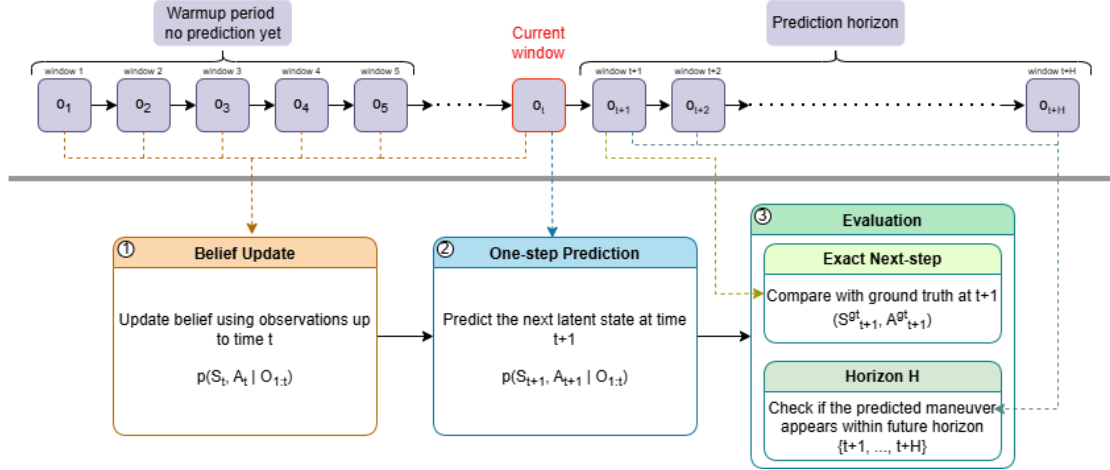


Figure 4.10.: Causal online prediction and evaluation procedure.

#### 4.7.1. Generative Evaluation Metrics

##### Average Log-Likelihood per Time Step

For a set of test vehicles indexed by  $i$ , the average log-likelihood per time step is defined as:

$$\bar{\ell}_{\text{step}} = \frac{1}{\sum_i T_i} \sum_i \log p(O_{1:T_i}^{(i)}) \quad (4.25)$$

where  $T_i$  is the length of the trajectory sequence  $i$ .

This metric measures how well the trained model explains unseen observation sequences on average. Since total log-likelihood scales with sequence length, normalizing by  $\sum_i T_i$  makes the score comparable across trajectories of different lengths. Higher values, that is, values that are less negative, indicate a better fit. For the proposed DBN, the likelihood of a sequence is obtained by marginalizing over all latent style and action sequences:

$$p(O_{1:T_i}^{(i)}) = \sum_{s_{1:T_i}} \sum_{a_{1:T_i}} p(O_{1:T_i}^{(i)}, s_{1:T_i} = s_{1:T_i}, a_{1:T_i} = a_{1:T_i}) \quad (4.26)$$

In practice, this quantity is evaluated with the forward algorithm rather than by explicit summation over all latent-state sequences [10], [13].

To complement the global average score, a normalized log-likelihood is also computed for each vehicle trajectory:

$$\ell_i = \frac{1}{T_i} \log p(O_{1:T_i}^{(i)}) \quad (4.27)$$

The distribution of  $\ell_i$  over the test split is then summarized using percentiles such as the 5th, 50th, and 95th percentiles.

This diagnostic shows whether good average likelihood is obtained uniformly across vehicles or is dominated by only a subset of trajectories.

##### Bayesian Information Criterion

To compare model variants with different numbers of free parameters, a complexity-penalized score based on the BIC is also computed on the training set:

$$\text{BIC} = k \log N - 2 \log \hat{L} \quad (4.28)$$

where  $k$  is the number of free model parameters,  $N = \sum_i T_i$  is the total number of training time steps, and  $\hat{L}$  denotes the fitted likelihood on the training set used for model comparison. A better balance between model complexity and goodness of fit is indicated by lower BIC values [70].

For the proposed model,

$$k = k_{\pi} + k_{\text{trans}} + k_{\text{em}} \quad (4.29)$$

#### 4. Methodology

With  $K_S$  style states,  $K_A$  action states,  $D_c$  continuous features, and  $D_b$  binary features, the number of free parameters is:

$$k_\pi = (K_S - 1) + K_S(K_A - 1), \quad (4.30)$$

$$k_{\text{trans}} = K_S(K_S - 1) + K_S K_A(K_A - 1) \quad (4.31)$$

For the hierarchical emission model:

$$k_{\text{em}}^{\text{hier}} = K_S K_A (2D_c + D_b), \quad (4.32)$$

assuming diagonal Gaussian parameters for the continuous part and Bernoulli parameters for the binary part. For the PoE emission model:

$$k_{\text{em}}^{\text{PoE}} = (K_S + K_A)(2D_c + D_b) \quad (4.33)$$

#### State Occupancy and Effective Number of States

Posterior state occupancies are analyzed to determine how strongly the learned latent states are used on the test split. The joint occupancy is defined as:

$$C_{\text{joint}}(s, a) = \sum_i \sum_{t=1}^{T_i} \gamma_t^{(i)}(s, a) \quad (4.34)$$

The marginal occupancies for style and action can be written directly from the joint posterior as:

$$C_{\text{style}}(s) = \sum_i \sum_{t=1}^{T_i} \sum_{a=1}^{K_A} \gamma_t^{(i)}(s, a), \quad (4.35)$$

$$C_{\text{action}}(a) = \sum_i \sum_{t=1}^{T_i} \sum_{s=1}^{K_S} \gamma_t^{(i)}(s, a) \quad (4.36)$$

The corresponding normalized occupancy distributions are:

$$p_{\text{joint}}(s, a) = \frac{C_{\text{joint}}(s, a)}{\sum_{s', a'} C_{\text{joint}}(s', a')}, \quad (4.37)$$

$$p_{\text{style}}(s) = \frac{C_{\text{style}}(s)}{\sum_{s'} C_{\text{style}}(s')}, \quad (4.38)$$

$$p_{\text{action}}(a) = \frac{C_{\text{action}}(a)}{\sum_{a'} C_{\text{action}}(a')} \quad (4.39)$$

In addition, the maximum joint-state occupancy is defined as:

$$p_{\text{max}}^{\text{joint}} = \max_{s, a} p_{\text{joint}}(s, a) \quad (4.40)$$



#### 4. Methodology

A concentrated occupancy distribution indicates that posterior mass is dominated by a small subset of states, whereas a flatter distribution indicates more balanced use of the state space.

To further quantify how many latent states are effectively used, the effective number of states can be computed from the entropy of the occupancy distribution:

$$N_{\text{eff}}^{\text{joint}} = \exp \left( - \sum_{s=1}^{K_S} \sum_{a=1}^{K_A} p_{\text{joint}}(s, a) \log p_{\text{joint}}(s, a) \right) \quad (4.41)$$

Analogous definitions are used for style and action. Larger values indicate that posterior mass is spread across more latent states, while smaller values indicate that the model relies on only a few states [71].

#### 4.7.2. Predictive Evaluation Metrics

##### Exact One-Step Prediction Accuracy

For each valid prediction time, the model produces a one-step-ahead prediction of the joint latent state  $\hat{z}_{t+1} = (\hat{S}_{t+1}, \hat{A}_{t+1})$  after observing the sequence up to time  $t$ . The exact one-step prediction accuracy is defined as the fraction of scored time steps for which the predicted joint state matches the corresponding rule-based reference label exactly:

$$\text{Accuracy} = \frac{1}{N} \sum_{n=1}^N \mathbf{1} \left[ \hat{z}_{t+1}^{(n)} = z_{t+1}^{(n)} \right], \quad (4.42)$$

where  $N$  is the total scored prediction instances pooled over all trajectories.

Since the joint latent space is multi-class and class frequencies can be imbalanced, macro-averaged precision, recall, and F1-score are also computed from the joint-state confusion matrix. These metrics complement accuracy by giving equal importance to all classes rather than being dominated by the most frequent states [72].

Let  $C_{ij}$  denote the confusion-matrix entry with reference class  $i$  and predicted class  $j$ . For class  $c$ , the per-class counts are defined as:

$$\text{TP}_c = C_{cc}, \quad (4.43)$$

$$\text{FP}_c = \sum_{i \neq c} C_{ic}, \quad (4.44)$$

$$\text{FN}_c = \sum_{j \neq c} C_{cj} \quad (4.45)$$

The class-wise precision, recall, and F1-score are then:

$$\text{Precision}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c}, \quad (4.46)$$

$$\text{Recall}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}, \quad (4.47)$$

$$\text{F1}_c = \frac{2 \text{Precision}_c \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}. \quad (4.48)$$

#### 4. Methodology

With  $K$  classes, the corresponding macro-averaged scores are:

$$\text{Precision}_{\text{macro}} = \frac{1}{K} \sum_{c=1}^K \text{Precision}_c, \quad (4.49)$$

$$\text{Recall}_{\text{macro}} = \frac{1}{K} \sum_{c=1}^K \text{Recall}_c, \quad (4.50)$$

$$\text{F1}_{\text{macro}} = \frac{1}{K} \sum_{c=1}^K \text{F1}_c \quad (4.51)$$

#### Hit Rate Within a Prediction Horizon

Exact next-step accuracy is intentionally strict, since it counts a prediction as correct only when the predicted latent state appears immediately at time  $t + 1$ . In behavior prediction, however, a forecast may still be useful when the predicted maneuver occurs a short time later. For this reason, a horizon-based hit metric is also used.

Let  $\hat{z}_{t+1}$  denote the one-step-ahead predicted joint state. For a future horizon of  $H$  steps, the prediction is counted as a hit if the predicted state appears at least once in the interval from  $t + 1$  to  $t + H$ :

$$\text{Hit@H}(t) = \mathbf{1}[\exists h \in \{1, \dots, H\} : z_{t+h} = \hat{z}_{t+1}] \quad (4.52)$$

The hit rate is then obtained by averaging this indicator over all scored prediction times.

#### Time-to-Event

To measure how early a correct prediction occurs within the prediction horizon, the Time To Event (TTE) is also computed. For a prediction made at time  $t$ , let  $h^*(t)$  denote the first future step at which the predicted state appears:

$$h^*(t) = \min \{h \in \{1, \dots, H\} : z_{t+h} = \hat{z}_{t+1}\} \quad (4.53)$$

If no hit occurs within the horizon, the prediction is treated as a miss.

For successful hits, the time-to-event in seconds is given by

$$\text{TTE}_{\text{sec}}(t) = h^*(t) \Delta t, \quad (4.54)$$

where  $\Delta t$  denotes the duration of one window step. With window stride measured in frames and dataset frame rate measured in frames per second, this is:

$$\Delta t = \frac{\text{stride}}{\text{frame rate}} \quad (4.55)$$

### 4.7.3. Posterior-Based Interpretation of Individual Predictions

To interpret selected predictions locally, the model output can be decomposed into four parts: the prior belief, the emission likelihood, the posterior belief, and the one-step forecast. At a chosen time step  $t$ , the prior belief describes the latent-state distribution before the current observation is incorporated. The emission likelihood measures how compatible the current observation is with each latent-state pair. The posterior belief then represents the updated latent-state distribution after observing the current traffic context, and the one-step forecast propagates this posterior one step forward.

This decomposition makes it possible to analyse whether a prediction is driven mainly by the transition dynamics, by strong evidence from the current observation, or by a combination of both.

### 4.7.4. Computational Efficiency

Since the intended application is online action prediction, computational efficiency is also evaluated. For this purpose, the trained model is benchmarked under several execution settings, including single-vehicle prediction, multi-vehicle sequential prediction, and batched prediction. These benchmarks are carried out both in a model-only setting, where precomputed window observations are given directly to the predictor, and in an end-to-end setting, where the full pipeline from the arrival of the final frame needed to complete a  $W$ -frame window through windowization, scaling, belief update, and prediction is included. These results are used to assess whether the proposed model is suitable for short-horizon online prediction in an AV context.

## 5. Results and Discussion

This chapter presents the experimental results of the proposed DBN framework and discusses the effect of the main modeling choices. The goal of the study is to identify which configuration provides the best trade-off between generative fit, predictive usefulness, and model complexity.

All compared models use the same preprocessing pipeline, the same vehicle-level train/validation/test split, the same window-based observation representation, and the same EM-based training procedure described in chapter 4. The compared settings differ mainly in the emission-model formulation, the treatment of the initial latent-state distribution, the use of lane-change weighting scheme, and the inclusion of Bernoulli likelihood terms.

In addition to these core comparisons, a set of latent-cardinality ablation studies is also carried out. These ablations vary the number of style and action states in order to assess whether the selected latent structure is appropriate for the prediction task. Besides the main setting with two style states and four action states, additional variants such as 2S3A, 2S5A, 1S4A, and 1S5A are evaluated. These experiments help determine whether the style variable is beneficial and whether increasing the number of action states leads to a more useful behavioral decomposition.

### 5.1. Main S2A4 Configuration Study

The main study considers the setting with two style states and four action states. This choice is based on the assumption that two latent behavioral styles are sufficient to represent the dominant driver regimes. Within this S2A4 setting, 24 configurations are compared. As described in Section 4.7, the models are assessed mainly using negative log-likelihood and BIC, together with selected posterior diagnostics. These quantities serve here as model-selection criteria.

#### 5.1.1. Choice of CPD Initialization

Before the main S2A4 study, a preliminary experiment compared two initialization schemes for the transition parameters: uniform initialization and sticky initialization. The comparison was carried out for the hierarchical model in the lc-none setting with Bernoulli likelihoods enabled and a fixed uniform initial latent-state distribution.

Sticky initialization showed slightly better convergence and a better final fit than uniform initialization. The posterior diagnostics remained very similar across the two runs,

## 5. Results and Discussion

indicating that both models used the latent-state space in a comparable way. Still, sticky initialization retained a small but consistent advantage in both training and validation. It was therefore adopted for all subsequent experiments in this chapter. Figure 5.1 shows the corresponding training and validation curves.

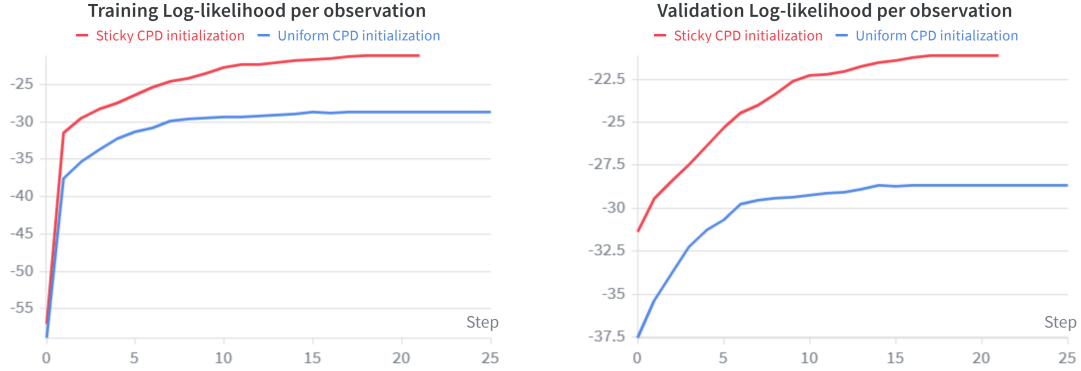


Figure 5.1.: Comparison of sticky vs. uniform CPD initialization for the hierarchical emission model (lc-none, Bernoulli enabled, uniform initial distribution).

### 5.1.2. Compared Configurations

The first comparison factor is the emission model. This factor examines whether a more structured but constrained emission model is preferable to a more flexible joint observation model.

The second factor is the treatment of the initial latent-state distribution. The two variants, are defined in Section 4.5.3. Here they are compared only in terms of their effect on overall predictive behavior.

The third factor is lane-change weighting. Three settings, as defined in Table 4.2, are compared.

The fourth factor is the inclusion of Bernoulli likelihood terms for the binary part of the observation vector. This makes it possible to assess whether explicit modeling of the binary components improves latent-state separation and prediction quality enough to justify the additional complexity.

### 5.1.3. Overview of S2A4 Comparison Results

Table 5.1 summarizes the results of the 24 configurations in the main S2A4 study. Lower values of Negative Log Likelihood (NLL) and BIC indicate better performance. Here,  $N_{\text{eff}}^{\text{joint}}$  denotes the effective number of jointly used latent states, and  $p_{\text{max}}^{\text{joint}}$  denotes the maximum joint-state occupancy. BIC values are reported in millions. Rather than discussing each

## 5. Results and Discussion

run separately, the following subsections analyze the results by grouping them according to the design factor under study.

Table 5.1.: Overview of the 24 compared configurations in the main S2A4 study.

| Exp. | Emission     | Init. dist. | LC mode | Bern. | NLL   | BIC [M] | $k$ | $N_{\text{eff}}^{\text{joint}}$ | $p_{\text{max}}^{\text{joint}}$ |
|------|--------------|-------------|---------|-------|-------|---------|-----|---------------------------------|---------------------------------|
| 1    | PoE          | uniform     | none    | off   | 28.05 | 66.66   | 518 | 6.37                            | 0.26                            |
| 2    | PoE          | uniform     | none    | on    | 28.16 | 66.71   | 530 | 6.36                            | 0.26                            |
| 3    | PoE          | uniform     | A       | off   | 28.08 | 128.76  | 518 | 6.37                            | 0.26                            |
| 4    | PoE          | uniform     | A       | on    | 28.13 | 131.95  | 530 | 6.36                            | 0.26                            |
| 5    | PoE          | uniform     | B       | off   | 30.20 | 71.38   | 518 | 4.89                            | 0.29                            |
| 6    | PoE          | uniform     | B       | on    | 30.18 | 71.35   | 530 | 4.88                            | 0.29                            |
| 7    | PoE          | learned     | none    | off   | 28.02 | 59.47   | 525 | 6.36                            | 0.26                            |
| 8    | PoE          | learned     | none    | on    | 28.20 | 65.08   | 537 | 6.36                            | 0.26                            |
| 9    | PoE          | learned     | A       | off   | 28.03 | 115.18  | 525 | 6.36                            | 0.26                            |
| 10   | PoE          | learned     | A       | on    | 28.17 | 129.12  | 537 | 6.36                            | 0.26                            |
| 11   | PoE          | learned     | B       | off   | 30.22 | 67.18   | 525 | 4.89                            | 0.29                            |
| 12   | PoE          | learned     | B       | on    | 30.16 | 70.85   | 537 | 4.88                            | 0.29                            |
| 13   | Hierarchical | uniform     | none    | off   | 21.93 | 53.28   | 682 | 6.95                            | 0.19                            |
| 14   | Hierarchical | uniform     | none    | on    | 20.97 | 50.82   | 698 | 6.82                            | 0.22                            |
| 15   | Hierarchical | uniform     | A       | off   | 20.92 | 99.33   | 682 | 6.84                            | 0.22                            |
| 16   | Hierarchical | uniform     | A       | on    | 21.98 | 104.35  | 698 | 6.95                            | 0.19                            |
| 17   | Hierarchical | uniform     | B       | off   | 22.81 | 55.43   | 682 | 6.13                            | 0.22                            |
| 18   | Hierarchical | uniform     | B       | on    | 22.86 | 55.50   | 698 | 6.13                            | 0.22                            |
| 19   | Hierarchical | learned     | none    | off   | 21.92 | 53.26   | 689 | 6.95                            | 0.19                            |
| 20   | Hierarchical | learned     | none    | on    | 21.99 | 53.39   | 705 | 6.94                            | 0.19                            |
| 21   | Hierarchical | learned     | A       | off   | 20.92 | 99.29   | 689 | 6.83                            | 0.22                            |
| 22   | Hierarchical | learned     | A       | on    | 21.97 | 104.33  | 705 | 6.94                            | 0.19                            |
| 23   | Hierarchical | learned     | B       | off   | 22.79 | 55.41   | 689 | 6.14                            | 0.22                            |
| 24   | Hierarchical | learned     | B       | on    | 22.83 | 55.46   | 705 | 6.13                            | 0.22                            |

Across the 24 compared configurations, the hierarchical models outperform the PoE models in NLL. Within the hierarchical family, lc-none gives the most reliable results, while lc-B consistently over-constrains the latent-state dynamics.

### 5.1.4. PoE versus Hierarchical Emissions

To isolate the effect of the emission-model formulation, each PoE configuration can be compared with its hierarchical counterpart under the same initial-distribution setting, lane-change weighting mode, and Bernoulli setting. Across all 12 matched pairs in Table 5.1, the hierarchical emission model achieves lower NLL than the corresponding PoE model. The same trend is reflected in BIC, despite the larger number of parameters in the hierarchical model.

## 5. Results and Discussion

This indicates that the additional flexibility of the hierarchical emission model is beneficial for the window-based highway-driving observations considered here, whereas the more structured PoE formulation appears too restrictive.

The posterior diagnostics also favor the hierarchical family. Its effective number of jointly used latent states is generally higher, while the maximum occupancy of a single joint state is lower than in the corresponding PoE runs. This suggests a more balanced use of the latent-state space, whereas the PoE models tend to concentrate more probability mass on fewer joint states.

Based on these results, the hierarchical emission model is retained as the stronger observation model.

### 5.1.5. Effect of Bernoulli Likelihood Terms

To examine the effect of explicitly modeling the binary components of the observation vector, each configuration with Bernoulli likelihoods disabled is compared with the corresponding configuration in which they are enabled, while keeping the emission model, initial-distribution setting, and lane-change weighting mode fixed. The effect is not uniform across the experiment grid.

For the PoE models, enabling the Bernoulli terms does not produce a consistent improvement. In most matched pairs, NLL becomes slightly worse, while the parameter count increases from  $k = 518$  to  $k = 530$  or from  $k = 525$  to  $k = 537$ . As a result, the BIC is usually unchanged. Only very small gains appear in the lc-B setting, and these are too marginal to indicate a meaningful advantage.

For the hierarchical models, the picture is mixed. In the principled lc-none setting with a fixed uniform initial distribution, enabling Bernoulli terms gives a clear benefit: the NLL improves from 21.93 in Exp. 13 to 20.97 in Exp. 14, and BIC improves from 53.28 M to 50.82 M despite the higher parameter count. This is the only case in which the binary likelihood terms clearly improve both measures.

However, this benefit does not generalize across the full hierarchical family. With learned initial distributions in the lc-none setting, the Bernoulli version is slightly worse than its non-Bernoulli counterpart (Exp. 20 versus Exp. 19). In the lc-A and lc-B settings, enabling Bernoulli terms is again neutral or negative, with a substantial degradation in the lc-A case. This indicates that the usefulness of the binary likelihood terms depends on the surrounding modeling choices rather than providing a uniform improvement by themselves.

### 5.1.6. Effect of Learning the Initial Distribution

To evaluate the role of the initial latent-state distribution, each configuration with a fixed uniform initial distribution is compared with the corresponding configuration in which the initial distribution is learned, while the emission model, lane-change weighting mode,

## 5. Results and Discussion

and Bernoulli setting remain unchanged. Overall, learning the initial distribution has only a limited effect on the final results.

In the PoE family, learning the initial distribution sometimes improves the likelihood-based scores, especially in the lc-none setting with Bernoulli terms disabled, where the NLL decreases slightly from 28.05 in Exp. 1 to 28.02 in Exp. 7 and the BIC improves more clearly from 66.66 M to 59.47 M. A similar pattern is visible for the lc-A setting without Bernoulli terms, where Exp. 9 performs slightly better than Exp. 3. However, these gains are not stable across all PoE runs, since some Bernoulli-enabled settings become slightly worse.

In the hierarchical family, the effect is even smaller. Across most matched pairs, the results with learned and fixed initial distributions are nearly identical. For example, in the lc-none setting with Bernoulli terms disabled, Exp. 19 and Exp. 13 differ only marginally in both NLL and BIC. When Bernoulli terms are enabled, the learned-initial-distribution variant is slightly worse than the fixed one in the same lc-none setting (Exp. 20 versus Exp. 14). The same weak pattern appears in the weighted settings, where the differences remain small compared with the stronger effects of the emission model and lane-change weighting.

These results suggest that the long-run behavior is governed much more by the transition and emission structure than by the precise choice of the initial latent belief. Since the observation sequences are long enough for filtering to move away from the initial distribution after only a few steps, changes in the initialization affect mainly the early part of each sequence and do not strongly alter the overall fit. For this reason, learning the initial distribution is not treated as a decisive factor in the final model choice. It can provide minor gains in some PoE runs, but within the better-performing hierarchical family it does not yield a consistent improvement. Accordingly, the final selection is based mainly on the emission model and the lane-change weighting setting rather than on this factor.

### 5.1.7. Effect of Lane-Change Weighting

For each emission model, initial-distribution setting, and Bernoulli setting, three variants are compared: the unweighted lc-none baseline, lc-A, and lc-B. This factor is especially relevant because lane-change events are sparse in the selected subset, which naturally motivates giving more emphasis to these rare but important maneuvers.

The clearest pattern is that lc-B consistently degrades the results. Across both emission families and across both initial-distribution settings, lc-B yields noticeably worse NLL values than lc-none. The posterior diagnostics also become less favorable. The effective number of jointly used latent states decreases, while the maximum occupancy of a single joint state increases. For example, in the PoE runs,  $N_{\text{eff}}^{\text{joint}}$  falls from about 6.36 in the lc-none setting to about 4.88 in lc-B, while  $p_{\text{max}}^{\text{joint}}$  rises from 0.26 to 0.29. A similar, though less pronounced, reduction in effective joint-state usage appears in the hierarchical family.



This indicates that lc-B pushes the model toward a less balanced use of the latent-state space and therefore over-constrains the learned dynamics.

The lc-A setting is more ambiguous. In some hierarchical runs, especially without Bernoulli terms, lc-A gives slightly lower NLL values than lc-none. For instance, Exp. 15 and Exp. 21 achieve lower NLL values than their lc-none counterparts Exp. 13 and Exp. 19. However, this effect is not stable across the full comparison grid. When Bernoulli terms are enabled, the advantage disappears or becomes very small, and in the PoE family lc-A does not provide a convincing overall benefit. Moreover, the BIC values under lc-A are consistently much less favorable than in the corresponding lc-none runs.

Although lc-A can sometimes improve the likelihood-based score, these gains are not treated as decisive because lc-A modifies inference heuristically rather than preserving the same probabilistic interpretation as the unweighted model. A lower score under lc-A therefore does not carry the same weight as an improvement obtained under the standard lc-none formulation.

Taken together, these findings support lc-none as the most principled and reliable choice. It avoids the clear degradation seen under lc-B and does not rely on the heuristic bias introduced by lc-A.

### 5.1.8. Summary of Selected Core Model

Based on the main S2A4 comparison, Exp. 14 is selected as the core model for the remainder of this chapter. This model uses hierarchical emissions, sticky CPDs initialization, a fixed uniform initial latent-state distribution, no lane-change weighting (lc-none), and Bernoulli likelihood terms enabled. It is selected because it provides the strongest overall combination of fit quality and balanced latent-state usage in the unweighted setting.

It achieves an NLL of 20.97 and a BIC of 50.82 M, while showing balanced use of the latent space with  $N_{\text{eff}}^{\text{joint}} = 6.82$  and  $p_{\text{max}}^{\text{joint}} = 0.22$ . On the held-out test set, the model uses both style states effectively, with  $k_{\text{eff}}^{\text{style}} \approx 2.00$ , and also makes substantial use of the action latent space, with  $k_{\text{eff}}^{\text{action}} \approx 3.72$ .

The learned transition structure is consistent with the intended interpretation of the model, with persistent style states and mostly persistent action states. This supports the factorization in which style captures the broader behavioral regime while action captures shorter-term maneuver dynamics within that regime.

Figure 5.2 further illustrates the learned transition structure of the selected model.

### 5.1.9. Semantic Interpretation of the Learned Latent States

Following the procedure described in Section 4.6, the latent states of the selected model are assigned semantic labels by examining posterior-weighted feature profiles and applying rule-based heuristics. This step is necessary because the DBN is trained unsupervised, so the learned states do not initially correspond to named driver behaviors.

For the S2A4 hierarchical model, the two style states capture the following regimes:

## 5. Results and Discussion

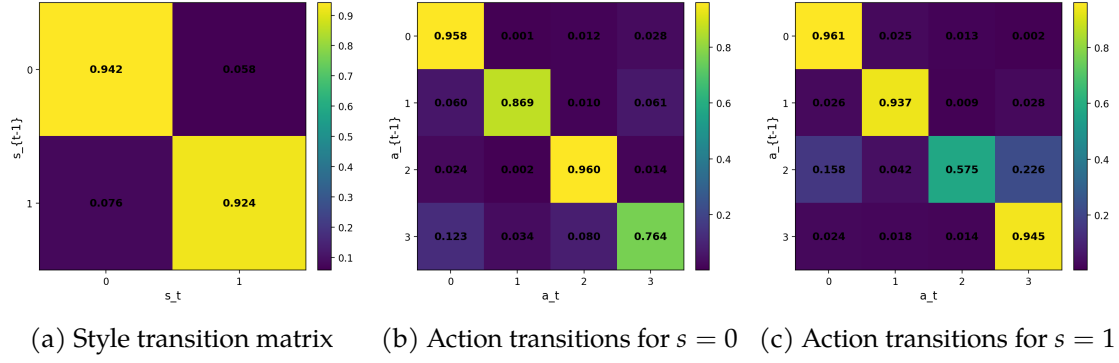


Figure 5.2.: Learned transition structure of the selected core model (Exp. 14).

- **Style  $s_0$  (low-interaction):** Motion is relatively smooth and less constrained by surrounding traffic.
- **Style  $s_1$  (high-interaction):** Driver behavior is more strongly influenced by nearby vehicles, gap availability, and tactical traffic constraints.

Within each style, the four action states represent maneuvers along the following interpretations:

- $(s_0, a_0)$  – **Free flow:** Unconstrained motion with little or no leader influence, typically at higher speed and with small speed corrections.
- $(s_0, a_1)$  – **Mild brake:** Sustained but not strongly tactical deceleration while following another vehicle.
- $(s_0, a_2)$  – **Constrained acceleration:** Acceleration present, but not unconstrained, since surrounding traffic still limits the maneuver.
- $(s_0, a_3)$  – **Stable following:** Relatively steady car-following with mild control effort and limited speed change.
- $(s_1, a_0)$  – **Strong acceleration:** Acceleration-dominant behavior in a tactical setting, often with a nearby leader still present.
- $(s_1, a_1)$  – **Lane change:** Clear lateral motion and associated speed adjustment.
- $(s_1, a_2)$  – **Constrained following:** Following behavior with tighter interaction, lower speed, or more frequent corrective adjustments than stable following.
- $(s_1, a_3)$  – **Tactical brake:** Sustained braking under stronger interaction constraints.

### 5.1.10. Predictive Performance of the Selected Model

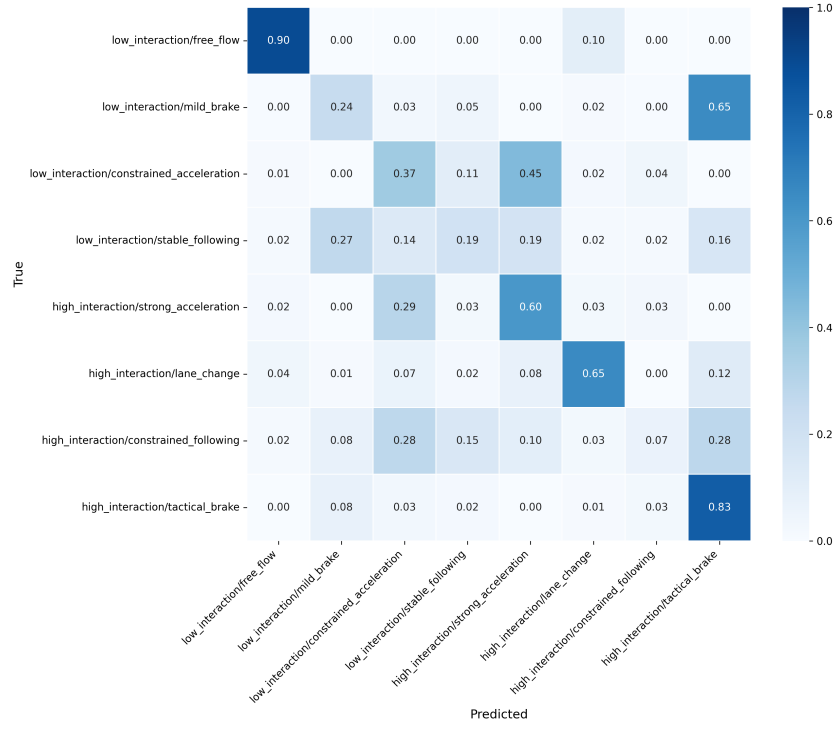
Using the semantic interpretation defined in Section 5.1.9, the predictive performance of the selected model is now evaluated in the causal online setting. The reference labeling, warmup handling, and causal prediction protocol follow Section 4.7.

#### Exact One-Step Prediction Performance

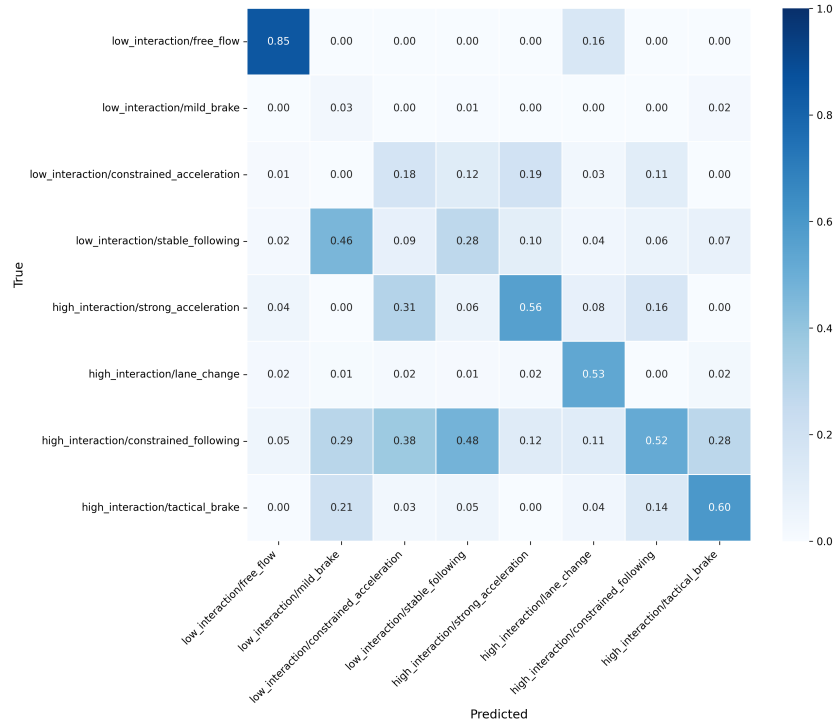


Figure 5.3.: Confusion matrix for exact one-step joint-state prediction on the test set. Rows denote reference labels and columns denote predicted labels.

## 5. Results and Discussion



(a) Recall view



(b) Precision view

Figure 5.4.: Row and column-normalized confusion matrices.

## 5. Results and Discussion

Under the strict joint-state evaluation, the selected model achieves an exact next-step accuracy of 46.37%. The macro precision, recall, and F1-score are 44.35%, 48.18%, and 42.25%, respectively. These values indicate that the one-step prediction task is challenging, especially because several maneuver classes are very similar to each other.

The confusion matrix in Figure 5.3 shows that the model identifies clearly distinguishable maneuvers such as free-flow driving, lane changes, strong acceleration, and tactical braking relatively well. However, semantically closer longitudinal behaviors such as mild braking, stable following, constrained acceleration, and constrained following remain strongly overlapped.

The row-normalized confusion matrix in Figure 5.4a shows how well each true class is recovered. Free flow is predicted well, and tactical braking is also recognized clearly. Strong acceleration is identified reasonably well. The main errors occur between semantically similar classes: mild brake is often confused with tactical brake, constrained acceleration with strong acceleration, and both following classes are distributed across nearby classes.

The column-normalized view in Figure 5.4b leads to a similar conclusion. It highlights prediction reliability: free-flow and lane-change predictions are comparatively precise, whereas mild-brake, constrained-acceleration, and following-related predictions are less precise because their predicted columns contain multiple related true classes.

The class-wise precision, recall, and F1-score further support this interpretation.

Table 5.2.: Class-wise precision, recall, and F1-score for exact one-step joint-state prediction.

| Class                                    | Precision [%] | Recall [%] | F1 [%] |
|------------------------------------------|---------------|------------|--------|
| low_interaction/free_flow                | 85.01         | 89.64      | 87.26  |
| low_interaction/mild_brake               | 3.07          | 24.34      | 5.45   |
| low_interaction/constrained_acceleration | 17.68         | 36.74      | 23.87  |
| low_interaction/stable_following         | 27.58         | 19.26      | 22.68  |
| high_interaction/strong_acceleration     | 56.41         | 60.46      | 58.37  |
| high_interaction/lane_change             | 53.40         | 65.50      | 58.83  |
| high_interaction/constrained_following   | 51.92         | 6.93       | 12.22  |
| high_interaction/tactical_brake          | 59.71         | 82.55      | 69.30  |

The Table 5.2 shows that free flow is the most clearly identifiable class, with high precision and recall. Tactical brake, Strong acceleration and lane change are also recognized moderately well. In contrast, mild brake performs very poorly, which suggests that weak braking behavior does not form a sufficiently distinct class under the current feature representation and semantic mapping. The two following-related classes are also difficult to separate cleanly. In particular, constrained\_following has moderate precision but extremely low recall, which means that the model predicts this class only rarely and

## 5. Results and Discussion

assigns many of its true samples to neighboring classes instead. Overall, these class-wise metrics confirm that the proposed model captures the broad behavioral structure, but still struggles with fine-grained distinctions between semantically similar longitudinal maneuvers.

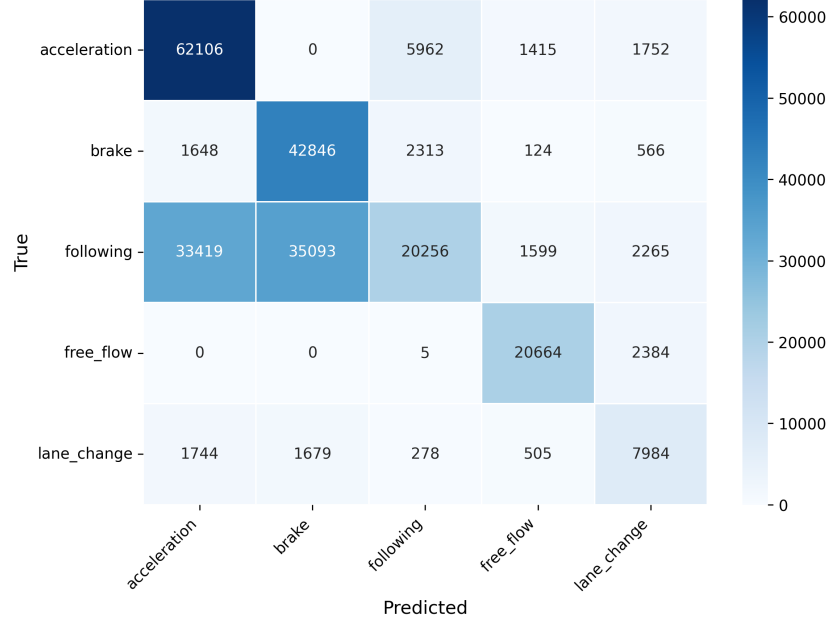


Figure 5.5.: Grouped confusion matrix for exact one-step prediction after merging semantically similar maneuver classes.

Figure 5.5 shows that when similar action classes are grouped together, the results improve clearly. The exact next-step accuracy increases to 62.39%. The grouped precision, recall, and F1-score increase to 65.06%, 70.88%, and 63.97%, respectively. This means that many wrong predictions are not fully wrong. In many cases, the model predicts a behavior that is close to the true class.

### Horizon-Based Prediction Performance

The exact next-step metric is very strict. A prediction is counted as correct only if the predicted class appears at the very next step. However, in highway traffic, a prediction can still be useful even if the maneuver happens a little later. For this reason, the selected model is also evaluated over a 2 s prediction horizon. As defined in Section 4.7, this horizon corresponds to the next five future observations.

Without grouping similar actions, the model achieves a hit rate of 55.21% within the 2 s horizon. The mean time-to-hit is 0.54 s, and the median time-to-hit is 0.40 s. This shows that when the model makes a successful forecast, the predicted maneuver usually appears very soon after the prediction. Figure 5.6 shows that the cumulative hit-rate

## 5. Results and Discussion

curve further supports this interpretation, since the largest gain occurs already at the first future step and the curve then rises only gradually over the remaining horizon. This shows that the model is mainly useful for immediate short-horizon prediction.

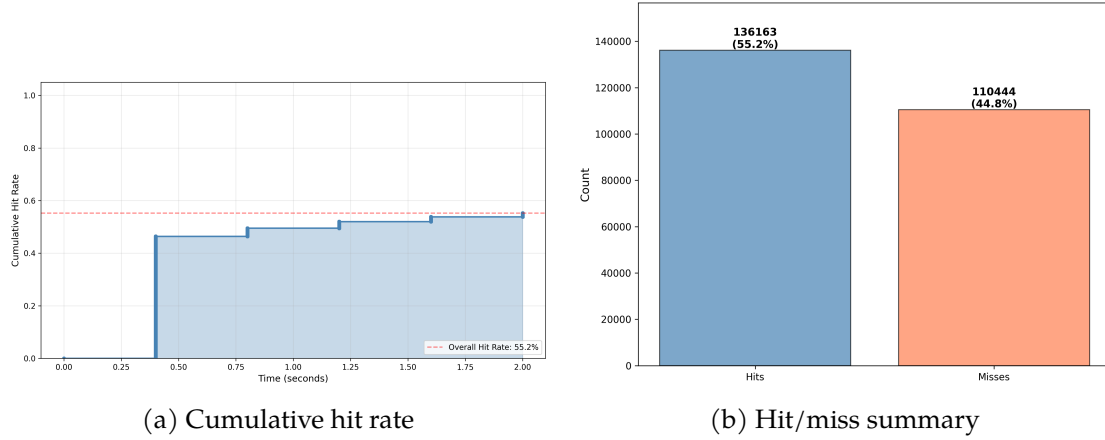


Figure 5.6.: Ungrouped horizon prediction results within the 2 s forecast horizon.

When semantically similar actions are grouped, the hit rate increases to 70.20%. The mean time-to-hit is 0.50 s, and the median remains 0.40 s. This confirms that many of the misses in the strict ungrouped evaluation are near-misses between related behaviors, especially within the acceleration, braking, and following classes. From an application point of view, this is an encouraging result, since predicting the correct broader maneuver tendency within a short time interval is often more important than identifying the exact fine-grained subclass at the very next step. Figure 5.7 shows the grouped horizon results.

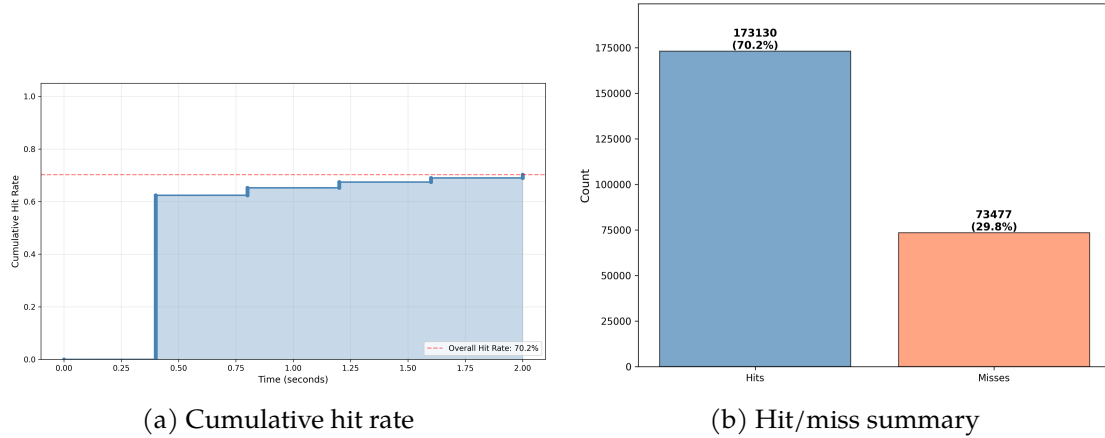


Figure 5.7.: Grouped horizon prediction results within the 2 s forecast horizon.

Overall, the selected DBN is more reliable as a probabilistic short-horizon maneuver predictor than as a strict fine-grained next-step classifier.

### 5.1.11. Computational Efficiency

To assess the practical feasibility of the selected DBN for online use, a latency benchmark was carried out on both CPU and GPU. The benchmark was performed on a Linux system with Python 3.11 and PyTorch 2.9.1, using an AMD Ryzen 9 7950X 16-Core CPU and an NVIDIA GeForce RTX 4090 GPU. Two settings were considered: a core *update+predict* pipeline that operates directly on window-level observations, and an end-to-end *frame→window→predict* pipeline that also includes online window construction.

For the present framework, the end-to-end latency is especially relevant. Since the dataset is sampled at 25 Hz and a new window is issued every 10 frames, a new prediction is required every 0.4 s (400 ms). The measured steady-state runtimes are far below this limit, as summarized in Table 5.3.

Table 5.3.: Mean steady-state latency of the selected model for different settings.

| Pipeline             | Mode                           | CPU [ms] | GPU [ms] |
|----------------------|--------------------------------|----------|----------|
| window→predict       | Single vehicle                 | 0.1642   | 6.0960   |
| window→predict       | Batched ( $B = 5$ vehicles)    | 0.2096   | 0.5483   |
| window→predict       | Sequential ( $B = 5$ vehicles) | 0.7957   | 15.8495  |
| frame→window→predict | Single vehicle                 | 0.5728   | 8.4220   |
| frame→window→predict | Batched ( $B = 5$ vehicles)    | 1.9084   | 13.1136  |
| frame→window→predict | Sequential ( $B = 5$ vehicles) | 2.6668   | 33.3626  |

For the practically most relevant end-to-end case, a single online prediction requires only about 0.57 ms on CPU. Even when five input streams are processed together in batched mode, the total runtime is only about 1.91 ms on CPU, which remains far below the available 400 ms decision interval. This indicates that the proposed framework is easily fast enough for real-time short-horizon prediction.

The batched results also show that parallel processing of multiple streams is feasible. Compared with a simple sequential loop over five streams, batching reduces the window-only runtime from about 0.80 ms to 0.21 ms on CPU, and from about 15.85 ms to 0.55 ms on GPU. For the end-to-end case, batching reduces the runtime from about 2.67 ms to 1.91 ms on CPU, and from about 33.36 ms to 13.11 ms on GPU. Thus, if predictions for several vehicles must be produced at the same time, the batched implementation is the more efficient choice.

The selected DBN is compact, so fixed GPU overheads dominate the total latency in the low-batch regime. These overheads are less visible only in the batched *window→predict* case, where the frame-to-window preprocessing is bypassed and GPU parallelism can be utilized more effectively.

For the model used in this work, CPU execution is generally more favorable than GPU execution in the low-batch online regime. Although the GPU can accelerate the batched *window→predict* case, it is slower in the more realistic end-to-end setting considered here.



Overall, the latency benchmark leads to three practical conclusions:

- (i) real-time deployment is easily feasible because all measured end-to-end runtimes are far below the 400 ms decision interval,
- (ii) CPU is the preferred target for low-batch per-vehicle online inference, and
- (iii) batching is still beneficial when many vehicles must be processed concurrently.

## 5.2. Ablation Studies on Latent-State Cardinality

This section examines how the performance of the proposed DBN changes when the number of latent style and action states is varied. The selected reference model uses two style states and four action states. The ablation study has two goals. First, it tests whether three or five action states are more suitable than four when the two-style structure is kept. Second, it evaluates whether the style variable itself is necessary by comparing the reference model with variants that use only one style state.

The comparison is based on validation log-likelihood during training, test-set NLL, effective occupied-state statistics, and semantic interpretability of the learned latent states. The occupancy-based quantity is used here only as a descriptive diagnostic and not as a model-selection criterion by itself. Together, these results help assess whether the chosen latent-state cardinality provides a good balance between model flexibility and meaningful behavioral structure.

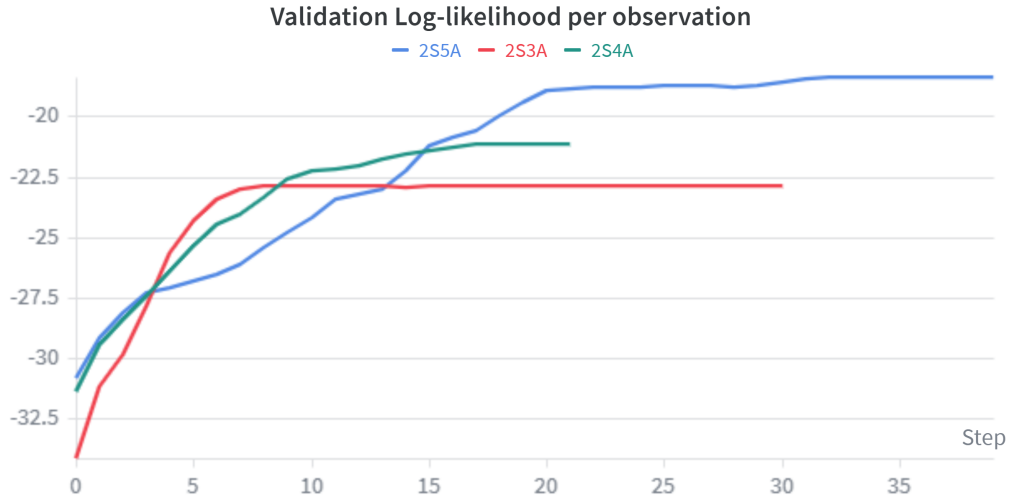


Figure 5.8.: Validation log-likelihood per observation for two-style models (2S3A, 2S4A, 2S5A).

Overall, the ablation results support the chosen latent-state design. Reducing the number of action states to three makes the behavioral partition too coarse, while removing

## 5. Results and Discussion

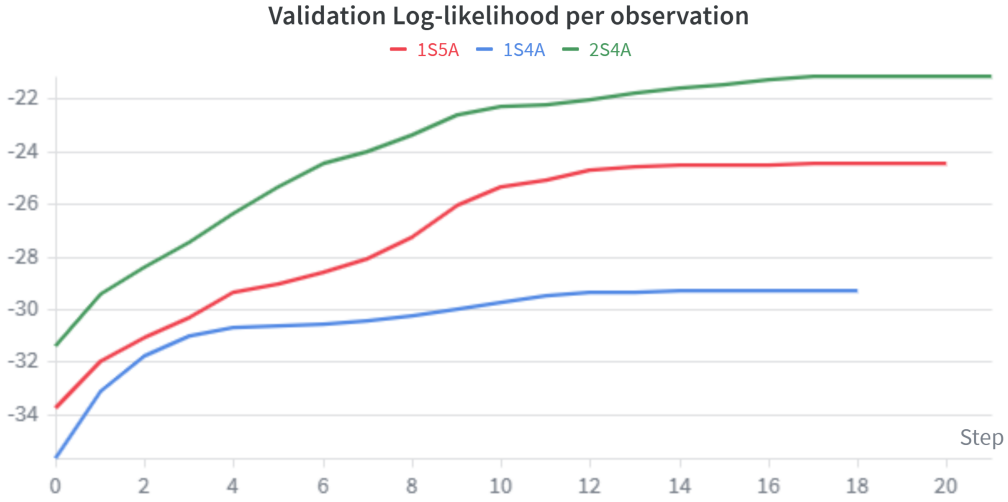


Figure 5.9.: Validation log-likelihood per observation for one-style models (1S4A, 1S5A) compared with the selected 2S4A reference.

the style variable causes a much larger drop in model quality. Increasing the number of action states can improve the fit, but it does not remove the need for a separate style latent. Taken together, these experiments show that the selected two-style, four-action structure provides a strong balance between performance, interpretability, and model complexity.

### 5.2.1. Ablation with 2 Style States and 3 Action States

The first ablation reduces the number of action states from four to three while keeping the two-style structure unchanged. This tests whether three action states are sufficient to capture the main maneuver patterns in the data.

Figure 5.8 shows that the 2S3A model improves quickly during the early training iterations, but then saturates at a lower validation log-likelihood than the 2S4A model. Its validation performance levels off at about  $-22.8$  per observation, whereas the 2S4A model continues improving and reaches about  $-21.2$ . This suggests that three action states are not enough to represent the full variation of the observed driver behavior.

The same pattern appears on the test set. The 2S3A model achieves a test NLL of 22.84 per time step, which is worse than the selected 2S4A model. This confirms that the weaker validation performance also carries over to unseen data.

The occupied-state statistics show that this weaker result is not caused by state collapse. The model still uses almost all available action capacity, with an effective number of occupied joint states of about  $k_{\text{eff}}^{\text{joint}} = 5.07$ . This suggests that the limitation comes from insufficient action resolution rather than from unused latent states.

The semantic analysis further shows that the 2S3A model still learns meaningful latent states. The two style states remain interpretable, and the three action states still capture broad behavior categories such as following, acceleration-related behavior, and braking-related behavior. However, the partition is coarser than in the 2S4A case. Some patterns that are more clearly separated in the selected model are merged into the same action state here. In other words, the model still works, but the action semantics are less distinct.

This result indicates that three action states are not sufficient for the desired level of behavioral separation. The fourth action state in the selected model therefore appears useful rather than redundant.

### 5.2.2. Ablation with 2 Style States and 5 Action States

The second ablation increases the number of action states from four to five while keeping two style states. This tests whether additional action capacity can further improve the model.

As shown in Figure 5.8, the 2S5A model reaches the best validation log-likelihood among the evaluated two-style variants. Its validation curve continues improving beyond both 2S3A and 2S4A and finally reaches about  $-18.4$  per observation. On the test set, it achieves a test NLL of 18.33 per time step. The occupied-state statistics further show that the additional capacity is actively used, with  $k_{\text{eff}}^{\text{joint}} = 7.99$ . Numerically, this is the strongest result among the tested action cardinalities.

The semantic analysis also shows that the extra action state is not wasted. The model separates a richer set of action patterns, including lane-change-related behavior, steady following, stronger acceleration, braking, and gentler acceleration. In the more interaction-dominant style, it also isolates more critical braking and stop-and-go-like acceleration behavior. This means that the additional action state is used to create a finer behavioral partition rather than simply duplicating existing states.

However, this finer partition is not automatically preferable for the final model choice. For the objectives of this thesis, high likelihood alone is not sufficient; the learned maneuver taxonomy should also remain compact and stable. In the 2S5A setting, several action states become very specific sub-variants of similar maneuvers. This increases semantic complexity and reduces the clarity of the resulting labels. The 2S4A model therefore remains the selected reference because it provides a better trade-off between predictive performance, interpretability, and actionable maneuver grouping.

### 5.2.3. Ablation with 1 Style State and 4 Action States

The next ablation removes the style hierarchy completely by using only one style state and four action states. This model tests whether the style variable is truly needed, or whether all relevant behavior can be represented through action states alone.

The result is clearly worse than in the two-style setting. Figure 5.9 shows that the 1S4A model remains far below the 2S4A model throughout training. Its validation

log-likelihood saturates near  $-29.3$  per observation, whereas the selected 2S4A model reaches about  $-21.2$ . This large gap shows that removing the style latent strongly reduces model quality.

The same pattern is visible on the test set. The 1S4A model achieves a test NLL of 28.30 per time step, which is substantially worse than the two-style model. Again, the poor result cannot be explained by unused action states. The model still occupies nearly all available action capacity, with  $k_{\text{eff}}^A = 3.92$ . Thus, the performance drop is not limited to training or validation, but also appears on the held-out test set.

The semantic analysis helps explain this result. Even without a style latent, the model still learns action states that roughly correspond to acceleration, braking, and stable following. However, the remaining state becomes broader and less clean, combining freer motion, lane-change-related behavior, and lower interaction pressure from front neighbors. This suggests that, without a separate style variable, the action states must represent two different aspects at the same time: the short-term maneuver itself and the broader traffic regime in which the maneuver occurs.

This ablation therefore shows that the style variable is not just an additional modeling detail. It plays an important role in separating persistent interaction context from immediate maneuver dynamics. Without it, the action states become more entangled and the overall model fit deteriorates strongly.

### 5.2.4. Ablation with 1 Style State and 5 Action States

A final ablation was carried out with one style state and five action states. The purpose of this experiment is to test whether the loss of the style variable can be compensated for by adding one more action state.

Compared with the 1S4A model, the 1S5A model performs clearly better. Its validation curve in Figure 5.9 rises well above the 1S4A curve and reaches about  $-24.4$  per observation. On the test set, it achieves a test NLL of 24.31 per time step. The occupied-state statistics show that the additional action capacity is partly used, with  $k_{\text{eff}}^A = 4.06$ .

This is a clear improvement over the 1S4A model, which means that the extra action state is useful.

The semantic analysis shows that the added action state improves the behavioral partition compared with the 1S4A case. In particular, braking, stable following, and acceleration appear more clearly as separate action states.

However, the separation is still not fully clean. Some states, especially those related to lane-change and low-speed adjustment behavior, still reflect more than one factor at the same time. For example, they are shaped not only by whether the vehicle is changing lane or adjusting speed, but also by lateral motion, nearby vehicles in adjacent lanes, front-vehicle presence, and the current speed level. This means that the action states still absorb part of the surrounding interaction context instead of representing only the maneuver type. Thus, adding more action states can reduce semantic mixing, but cannot eliminate it as long as no separate style latent is present.

## 6. Conclusions

This thesis proposed a DBN framework for probabilistic short-horizon prediction of human driver maneuvers from window-based highway observations. The results show that the proposed model can capture meaningful temporal behavior patterns and provide useful maneuver forecasts in a causal online setting.

Among the compared configurations, the hierarchical emission model gave the best overall results and was selected as the final setup. It consistently outperformed the PoE alternative and provided the best final balance between model fit, latent-state usage, and interpretability. The selected model achieved moderate performance in strict exact next-step prediction, which confirms that fine-grained maneuver discrimination is difficult. At the same time, the short-horizon results were clearly stronger, especially when semantically similar maneuvers were grouped. This shows that many errors are near-misses between related behaviors rather than completely wrong predictions. The horizon-based evaluation further supported this finding, with a strong hit rate over the 2 s prediction horizon.

The latent-cardinality ablation study further supported the final design choice. Using only three action states made the behavioral partition too coarse, while removing the style variable caused a much larger drop in model quality. Although five action states improved the numerical fit, the resulting state space became less compact and harder to interpret. Overall, the selected two-style, four-action structure provided the best trade-off between prediction quality, interpretability, and model complexity.

Finally, the runtime benchmark showed that the framework is suitable for online use. The measured end-to-end latency remained far below the available decision interval, which makes the proposed approach practical for real-time short-horizon maneuver prediction.

### 6.1. Future Work

Future work should focus on four main directions. First, the observation design can be improved by investigating alternative feature sets and grouping strategies in order to achieve cleaner separation between latent states. Second, richer driving datasets with more diverse maneuvers should be considered, since the selected highD dataset mainly contains lane-following highway behavior. Third, alternative latent-state structures could be explored to represent driving style and maneuver dynamics more effectively. Finally, the proposed predictor can be integrated into a larger AV system, where its probabilistic maneuver forecasts may support planning and risk-assessment modules.

# Bibliography

- [1] M. R. Abedi, Z. Rashidi, N. Mokari, H. Saeedi, and N. Zorba, *Precise hdv positioning through safety-aware integrated sensing and communication in a value-of-information-driven 6g v2x system*, 2025. arXiv: 2510.02363 [eess.SY].
- [2] W. Schwarting, J. Alonso-Mora, and D. Rus, "Planning and decision-making for autonomous vehicles", *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 1, no. 1, pp. 187–210, 2018.
- [3] S. Lefevre, D. Vasquez, and C. Laugier, "A survey on motion prediction and risk assessment for intelligent vehicles", *Robomech Journal*, vol. 1, Jul. 2014.
- [4] N. A. Madjid, A. Ahmad, M. Mebrahtu, Y. Babaa, A. Nasser, S. Malik, B. Hassan, N. Werghi, J. Dias, and M. Khonji, "Trajectory prediction for autonomous driving: Progress, limitations, and future directions", *Information Fusion*, vol. 126, p. 103 588, Feb. 2026.
- [5] N. M. Negash and J. Yang, "Driver behavior modeling toward autonomous vehicles: Comprehensive review", *IEEE Access*, vol. 11, pp. 22 788–22 821, 2023.
- [6] A. Rudenko, L. Palmieri, M. Herman, K. M. Kitani, D. M. Gavrila, and K. O. Arras, "Human motion trajectory prediction: A survey", *The International Journal of Robotics Research*, vol. 39, no. 8, pp. 895–935, Jun. 2020.
- [7] Y. Yang, N. Negash, and J. Yang, "Recent advances in interactive driving of autonomous vehicles: Comprehensive review of approaches", *Automotive Innovation*, vol. 8, Apr. 2025.
- [8] K. Murphy, *Dynamic Bayesian Networks: Representation, Inference and Learning*. University of California, Berkeley, 2002.
- [9] R. Krajewski, J. Bock, L. Kloecker, and L. Eckstein, *The highd dataset: A drone dataset of naturalistic vehicle trajectories on german highways for validation of highly automated driving systems*, 2018. arXiv: 1810.05642 [cs.CV].
- [10] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [11] D. Koller and N. Friedman, *Probabilistic Graphical Models: Principles and Techniques* (Adaptive computation and machine learning). MIT Press, 2009.
- [12] K. P. Murphy, "Machine learning - a probabilistic perspective", in *Adaptive computation and machine learning series*, 2012.
- [13] L. Rabiner, "A tutorial on hidden markov models and selected applications in speech recognition", *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.

## Bibliography

- [14] S. Zhou, S. Zheng, J. Tian, R. Jiang, and H. M. Zhang, *Twenty-five years of the intelligent driver model: Foundations, extensions, applications, and future directions*, 2025. arXiv: 2506.05909 [physics.soc-ph].
- [15] M. Treiber, A. Hennecke, and D. Helbing, “Congested traffic states in empirical observations and microscopic simulations”, *Phys. Rev. E*, vol. 62, pp. 1805–1824, 2 Aug. 2000.
- [16] P. Gipps, “A behavioural car-following model for computer simulation”, *Transportation Research Part B: Methodological*, vol. 15, no. 2, pp. 105–111, 1981.
- [17] M. Bando, K. Hasebe, A. Nakayama, A. Shibata, and Y. Sugiyama, “Dynamical model of traffic congestion and numerical simulation”, *Phys. Rev. E*, vol. 51, pp. 1035–1042, 2 Feb. 1995.
- [18] R. Jiang, Q. Wu, and Z. Zhu, “Full velocity difference model for a car-following theory”, *Phys. Rev. E*, vol. 64, p. 017 101, 1 Jun. 2001.
- [19] A. Kesting, M. Treiber, and D. Helbing, “General lane-changing model mobil for car-following models”, *Transportation Research Record*, vol. 1999, pp. 86–94, 2007.
- [20] S. Wang, K. Gao, L. Zhang, Y. Liu, and L. Chen, *Probabilistic prediction of longitudinal trajectory considering driving heterogeneity with interpretability*, 2023. arXiv: 2312.12123 [cs.LG].
- [21] Q. Deng, J. Wang, and D. Soffker, “Prediction of human driver behaviors based on an improved hmm approach”, in *2018 IEEE Intelligent Vehicles Symposium (IV)*, 2018, pp. 2066–2071.
- [22] D. Chen, M. Zhu, H. Yang, X. Wang, and Y. Wang, *Data-driven traffic simulation: A comprehensive review*, 2023. arXiv: 2310.15975 [cs.LG].
- [23] J. Pearl and S. Russell, “Bayesian networks”, in *The Handbook of Brain Theory and Neural Networks*, M. A. Arbib, Ed., Cambridge, MA: MIT Press, 1995, pp. 149–153.
- [24] J. Li, B. Dai, X. Li, X. Xu, and D. Liu, “A dynamic bayesian network for vehicle maneuver prediction in highway driving scenarios: Framework and verification”, *Electronics*, vol. 8, no. 1, 2019.
- [25] D. Młynarczyk, G. Calvo, F. Palmi-Perales, C. Armero, V. Gómez-Rubio, and U. Martinez-Iranzo, *Bayesian network approach to building an affective module for a driver behavioural model*, 2025. arXiv: 2502.03254 [stat.AP].
- [26] K. K. Talluri, A. L. Madsen, and G. Weidl, *Enhancing safety standards in automated systems using dynamic bayesian networks*, 2025. arXiv: 2505.02050 [cs.AI].
- [27] S. Mozaffari, M. A. Sormoli, K. Koufos, and M. Dianati, *Multimodal manoeuvre and trajectory prediction for automated driving on highways using transformer networks*, 2023. arXiv: 2303.16109 [cs.LG].
- [28] C. Zhang, K. Chen, M. Zhu, H. Yang, and L. Sun, *Learning car-following behaviors using bayesian matrix normal mixture regression*, 2024. arXiv: 2404.16023 [stat.AP].

## Bibliography

- [29] J. Bethge, M. Pfefferkorn, A. Rose, J. Peters, and R. Findeisen, “Model predictive control with gaussian-process-supported dynamical constraints for autonomous vehicles”, *ArXiv*, vol. abs/2303.04725, 2023.
- [30] R. Ghandour, A. J. Potams, I. Boulkaibet, B. Neji, and Z. Al Barakeh, “Driver behavior classification system analysis using machine learning methods”, *Applied Sciences*, vol. 11, no. 22, 2021.
- [31] Y. Bengio, A. Courville, and P. Vincent, *Representation learning: A review and new perspectives*, 2014. arXiv: 1206.5538 [cs.LG].
- [32] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org>.
- [33] L. Xin, P. Wang, C.-Y. Chan, J. Chen, S. E. Li, and B. Cheng, *Intention-aware long horizon trajectory prediction of surrounding vehicles using dual lstm networks*, 2019. arXiv: 1906.02815 [cs.LG].
- [34] J. Hao, H. Xie, G. Fan, Y. Chen, and K. Song, “Vehicle trajectory prediction with driving style identification and intention fusion”, Oct. 2024, pp. 1–6.
- [35] S. Hochreiter and J. Schmidhuber, “Long short-term memory”, *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [36] F. Hui, C. Wei, W. ShangGuan, R. Ando, and S. Fang, “Deep encoder–decoder-nn: A deep learning-based autonomous vehicle trajectory prediction and correction model”, *Physica A: Statistical Mechanics and its Applications*, vol. 593, p. 126 869, 2022.
- [37] F. Diehl, T. Brunner, M. T. Le, and A. Knoll, *Graph neural networks for modelling traffic participant interaction*, 2019. arXiv: 1903.01254 [cs.LG].
- [38] A. Abdelraouf, R. Gupta, and K. Han, *Interaction-aware personalized vehicle trajectory prediction using temporal graph neural networks*, 2023. arXiv: 2308.07439 [cs.LG].
- [39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, *Attention is all you need*, 2023. arXiv: 1706.03762 [cs.CL].
- [40] V. Winland. “What is self-attention?”, IBM, Accessed: Oct. 30, 2025. [Online]. Available: <https://www.ibm.com/think/topics/self-attention>
- [41] J. Ngiam et al., *Scene transformer: A unified architecture for predicting multiple agent trajectories*, 2022. arXiv: 2106.08417 [cs.CV].
- [42] S. Shi, L. Jiang, D. Dai, and B. Schiele, *Motion transformer with global intention localization and local movement refinement*, 2023. arXiv: 2209.13508 [cs.CV].
- [43] Z. Zikang, L. Ye, J. Wang, K. Wu, and K. Lu, “Hivt: Hierarchical vector transformer for multi-agent motion prediction”, Jun. 2022, pp. 8813–8823.



## Bibliography

- [44] K. Sohn, H. Lee, and X. Yan, “Learning structured output representation using deep conditional generative models”, in *Advances in Neural Information Processing Systems*, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, Eds., vol. 28, Curran Associates, Inc., 2015.
- [45] T. Salzmann, B. Ivanovic, P. Chakravarty, and M. Pavone, *Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data*, 2021. arXiv: 2001.03093 [cs.R0].
- [46] K. Mangalam, H. Girase, S. Agarwal, K.-H. Lee, E. Adeli, J. Malik, and A. Gaidon, *It is not the journey but the destination: Endpoint conditioned trajectory prediction*, 2020. arXiv: 2004.02025 [cs.CV].
- [47] H. Zhou, D. Ren, X. Yang, M. Fan, and H. Huang, “Csr: Cascade conditional variational auto encoder with socially-aware regression for pedestrian trajectory prediction”, *Pattern Recognition*, vol. 133, p. 109 030, 2023.
- [48] J. Ho, A. Jain, and P. Abbeel, *Denoising diffusion probabilistic models*, 2020. arXiv: 2006.11239 [cs.LG].
- [49] M. Peng, K. Chen, X. Guo, Q. Zhang, H. Zhong, M. Zhu, and H. Yang, *Diffusion models for intelligent transportation systems: A survey*, 2025. arXiv: 2409.15816 [eess.SY].
- [50] S. Lian, B. Zhou, S. Hu, J. Hu, G. Wang, J. Escribano, X. Na, and S. Jin, “Enhanced multimodal trajectory prediction for autonomous vehicles using advanced diffusion model techniques”, *2024 IEEE Intelligent Vehicles Symposium (IV)*, pp. 484–489, 2024.
- [51] Q. T. Liu, D. Li, S. S. Sohn, S. Yoon, M. Kapadia, and V. Pavlovic, “Trajdiffuse: A conditional diffusion model for environment-aware trajectory prediction”, in *Pattern Recognition*, A. Antonacopoulos, S. Chaudhuri, R. Chellappa, C.-L. Liu, S. Bhattacharya, and U. Pal, Eds., Cham: Springer Nature Switzerland, 2025, pp. 382–397.
- [52] C. M. Jiang, A. Cornman, C. Park, B. Sapp, Y. Zhou, and D. Anguelov, *Motiondiffuser: Controllable multi-agent motion prediction using diffusion*, 2023. arXiv: 2306.03083 [cs.R0].
- [53] S. Arora and P. Doshi, “A survey of inverse reinforcement learning: Challenges, methods and progress”, *Artificial Intelligence*, vol. 297, p. 103 500, 2021.
- [54] B. D. Ziebart, A. Maas, J. A. Bagnell, and A. K. Dey, “Maximum entropy inverse reinforcement learning”, in *Proceedings of the 23rd National Conference on Artificial Intelligence - Volume 3*, ser. AAAI’08, Chicago, Illinois: AAAI Press, 2008, pp. 1433–1438.
- [55] A. Y. Ng and S. J. Russell, “Algorithms for inverse reinforcement learning”, in *Proceedings of the Seventeenth International Conference on Machine Learning*, ser. ICML ’00, San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2000, pp. 663–670.

## Bibliography

- [56] J. A. Rodrigues da Silva, V. Grassi, and D. F. Wolf, "Maximum entropy inverse reinforcement learning using monte carlo tree search for autonomous driving", *Trans. Intell. Transport. Sys.*, vol. 25, no. 9, pp. 11 552–11 562, Sep. 2024.
- [57] S. Karimi and A. Vahidi, "Receding horizon motion planning for automated lane change and merge using monte carlo tree search and level-k game theory", in *2020 American Control Conference (ACC)*, 2020, pp. 1223–1228.
- [58] H. Niu, Q. Chen, Y. Li, Y. Zhang, and J. Hu, *Stackelberg driver model for continual policy improvement in scenario-based closed-loop autonomous driving*, 2023. arXiv: 2309.14235 [cs.LG].
- [59] K. Ji, M. Orsag, and K. Han, "Lane-merging strategy for a self-driving car in dense traffic using the stackelberg game approach", *Electronics*, vol. 10, no. 8, 2021.
- [60] Z. Huang, T. Li, S. Shen, and J. Ma, *Integrated decision making and trajectory planning for autonomous driving under multimodal uncertainties: A bayesian game approach*, 2024. arXiv: 2409.13993 [cs.R0].
- [61] C. Hubmann, M. Becker, D. Althoff, D. Lenz, and C. Stiller, "Decision making for autonomous driving considering interaction and uncertain prediction of surrounding vehicles", in *2017 IEEE Intelligent Vehicles Symposium (IV)*, 2017, pp. 1671–1678.
- [62] L. Westhofen, C. Neurohr, T. Koopmann, M. Butz, B. Schütt, F. Utesch, B. Neurohr, C. Gutenkunst, and E. Böde, "Criticality metrics for automated driving: A review and suitability analysis of the state of the art", *Archives of Computational Methods in Engineering*, vol. 30, no. 1, pp. 1–35, Aug. 2022.
- [63] X. Xiong, S. Zhang, and Y. Chen, "Review of intelligent vehicle driving risk assessment in multi-vehicle interaction scenarios", *World Electric Vehicle Journal*, vol. 14, no. 12, 2023.
- [64] T. Nyberg, C. Pek, L. Col, C. Norén, and J. Tumova, "Risk-aware motion planning for autonomous vehicles with safety specifications", May 2021.
- [65] E. Arnold, M. Dianati, and S. Fallah, "Early lane change prediction for automated driving systems using multi-task attention-based convolutional neural networks", *IEEE Transactions on Intelligent Vehicles*, vol. 7, pp. 1–1, Sep. 2022.
- [66] G. E. Hinton, "Training products of experts by minimizing contrastive divergence", *Neural Computation*, vol. 14, no. 8, pp. 1771–1800, 2002.
- [67] J. He and M. Cheng, "Weighting methods for rare event identification from imbalanced datasets", *Frontiers in Big Data*, vol. 4, Dec. 2021.
- [68] C. Holmes and S. Walker, *Assigning a value to a power likelihood in a general bayesian model*, 2017. arXiv: 1701.08515 [stat.ME].

## Bibliography

- [69] I. D. Gebru, X. Alameda-Pineda, F. Forbes, and R. Horaud, "Em algorithms for weighted-data clustering with application to audio-visual scene analysis", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 12, pp. 2402–2415, Dec. 2016.
- [70] G. Schwarz, "Estimating the dimension of a model", *Annals of Statistics*, vol. 6, pp. 461–464, 1978.
- [71] L. Jost, "Entropy and diversity", *Oikos*, vol. 113, no. 2, pp. 363–375, 2006. eprint: <https://nsojournals.onlinelibrary.wiley.com/doi/pdf/10.1111/j.2006.0030-1299.14714.x>.
- [72] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks", *Inf. Process. Manag.*, vol. 45, pp. 427–437, 2009.
- [73] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the em algorithm", *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 39, no. 1, pp. 1–22, Dec. 2018. eprint: [https://academic.oup.com/jrsssb/article-pdf/39/1/1/49117094/jrsssb\\_39\\_1\\_1.pdf](https://academic.oup.com/jrsssb/article-pdf/39/1/1/49117094/jrsssb_39_1_1.pdf).
- [74] J. Bilmes, "A gentle tutorial of the EM algorithm and its application to parameter estimation for Gaussian mixture and hidden Markov models", *International Computer Science Institute*, vol. 4, p. 126, 1998.
- [75] A. Wu, F. BYC., and W. Jeff, "On the convergence properties of the em algorithm", *Annals of Statistics*, vol. 11, pp. 95–103, 1983.

## A. Direction-Wise Sign Convention in Raw Kinematics

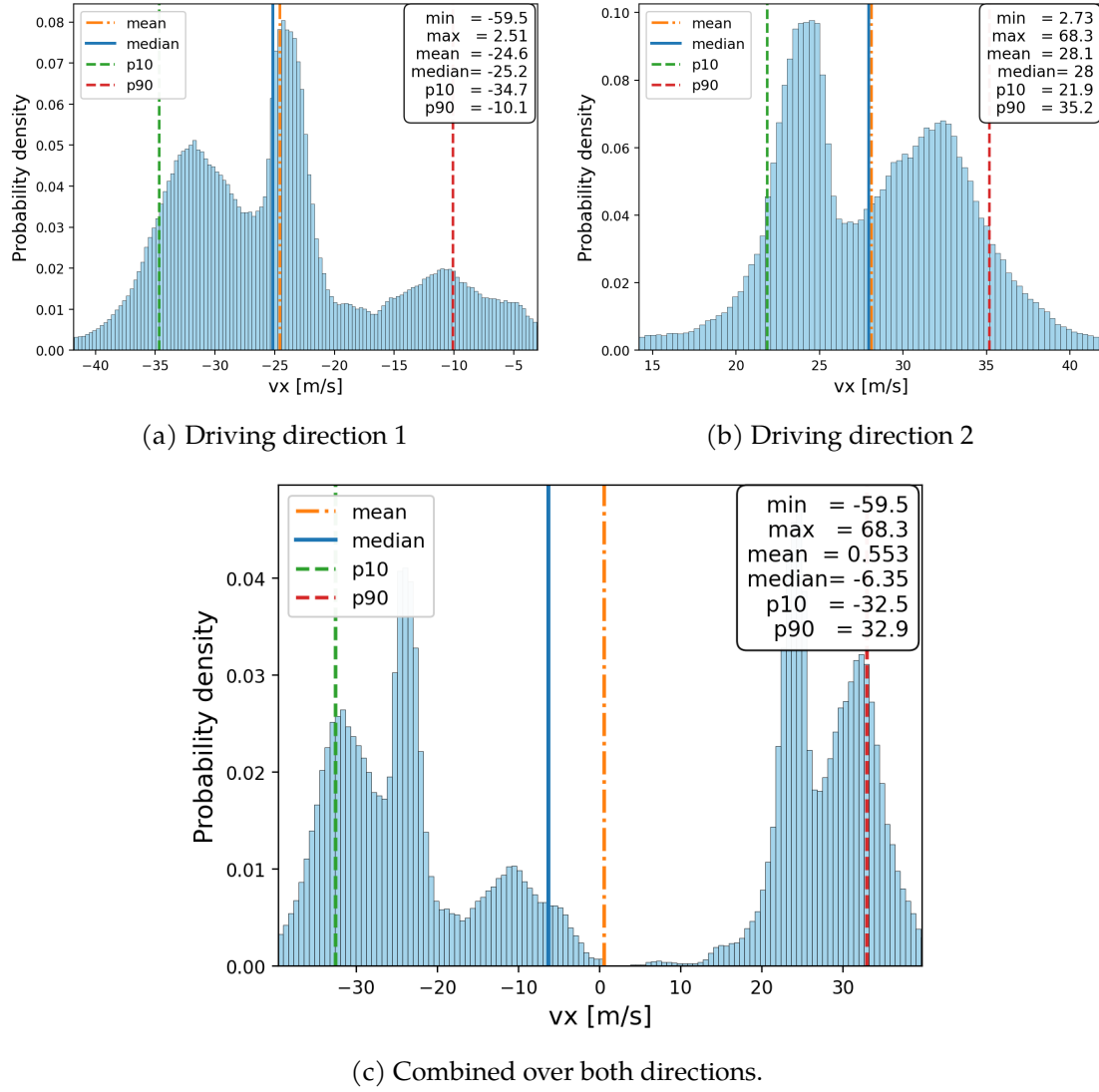


Figure A.1.: Direction-wise sign difference in raw longitudinal velocity.

### A. Direction-Wise Sign Convention in Raw Kinematics

This appendix illustrates the sign inconsistency in raw highD kinematic signals caused by opposite traffic directions. For the longitudinal velocity  $v_x$ , forward motion appears with opposite signs in the two directions. As a result, the combined raw distribution is bimodal around zero and cannot be interpreted with a single sign convention.

For completeness, the same raw longitudinal velocity signal is also shown by vehicle class:

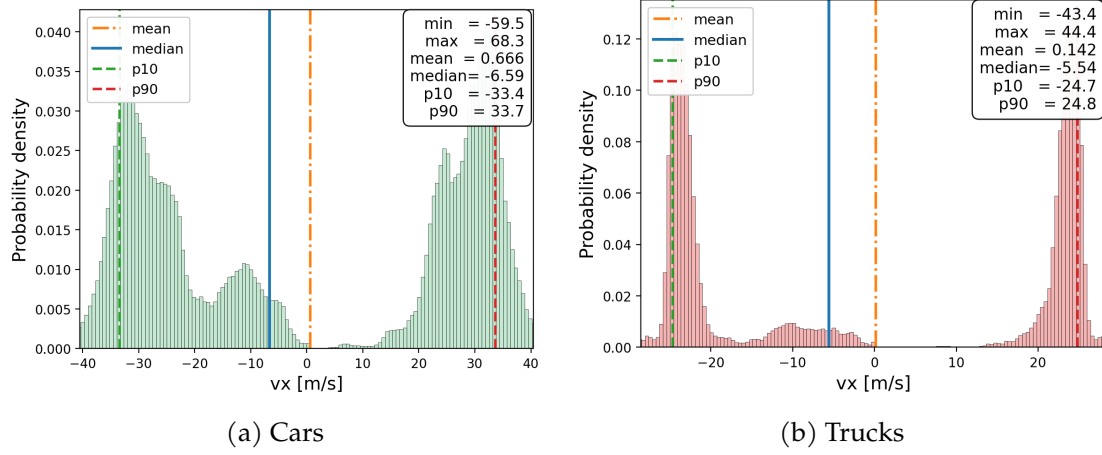


Figure A.2.: Class-wise raw longitudinal velocity distributions.

## B. Class-Wise Distribution Plots

This appendix provides class-wise versions of the main kinematic and trajectory-level variability distributions used in the exploratory data analysis. The plots separate cars and trucks in order to make the class-dependent differences in scale and spread more explicit.

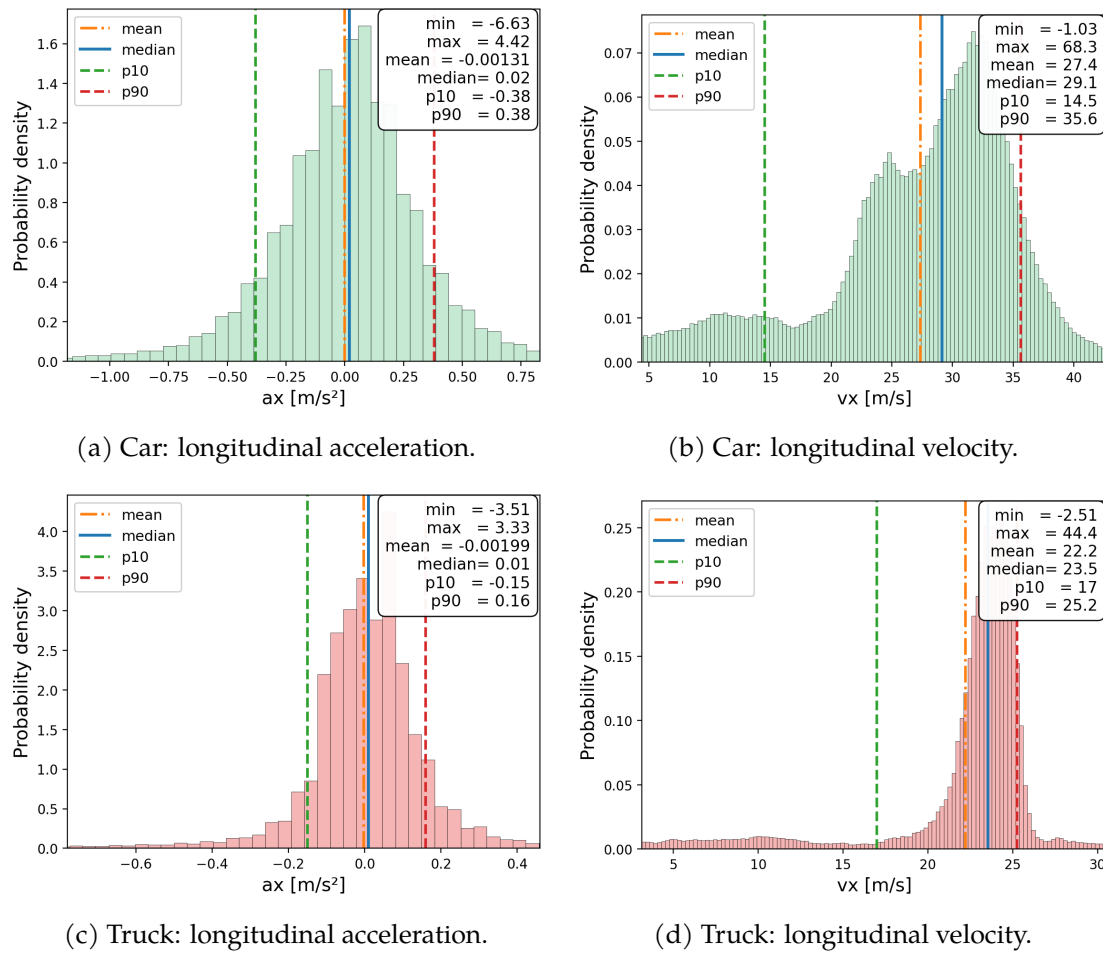
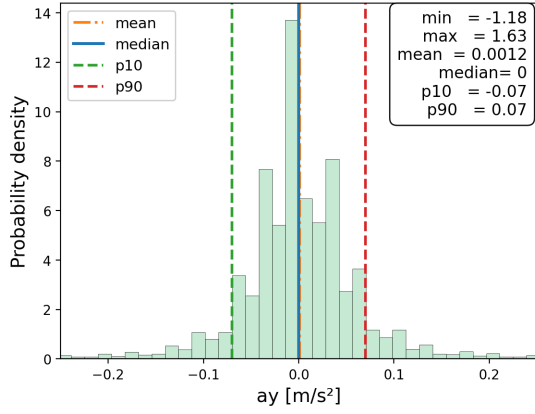
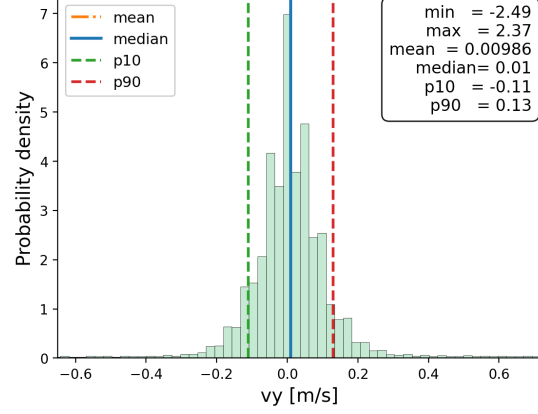


Figure B.1.: Class-wise frame-level distributions of the main kinematic variables.

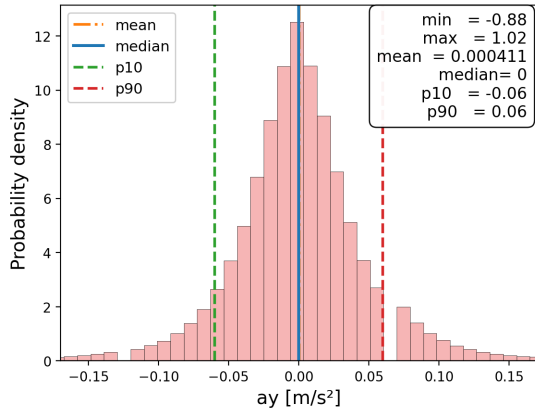
## B. Class-Wise Distribution Plots



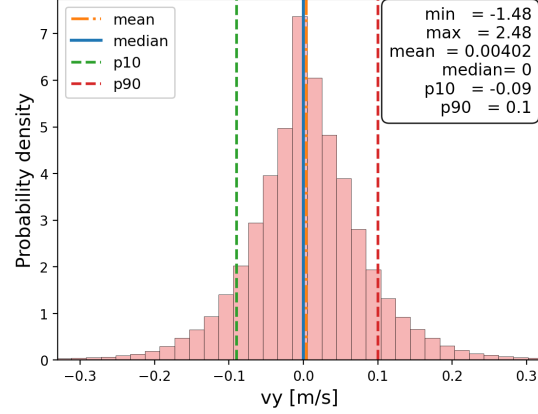
(e) Car: lateral acceleration.



(f) Car: lateral velocity.



(g) Truck: lateral acceleration.



(h) Truck: lateral velocity.

Figure B.1.: Class-wise frame-level distributions of the main kinematic variables (continued).

## B. Class-Wise Distribution Plots

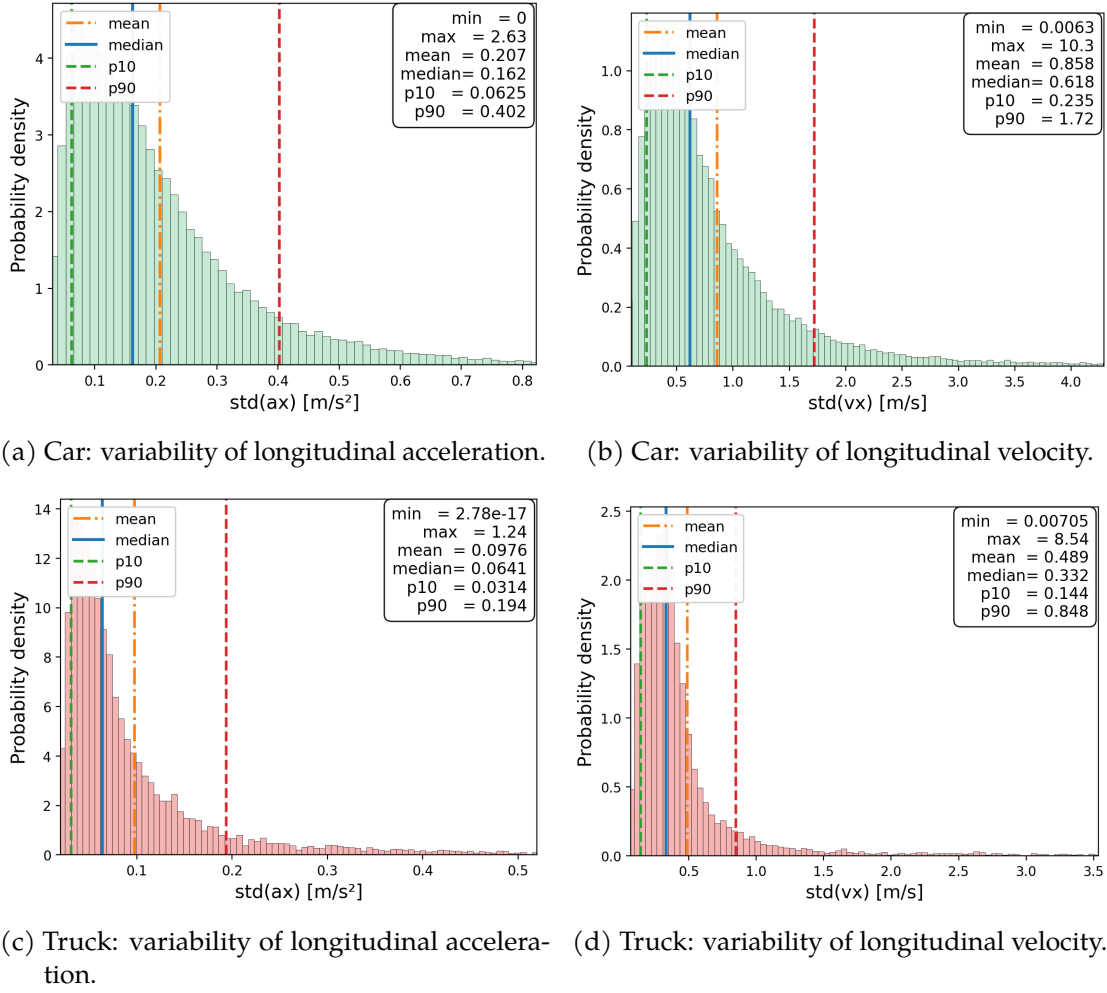
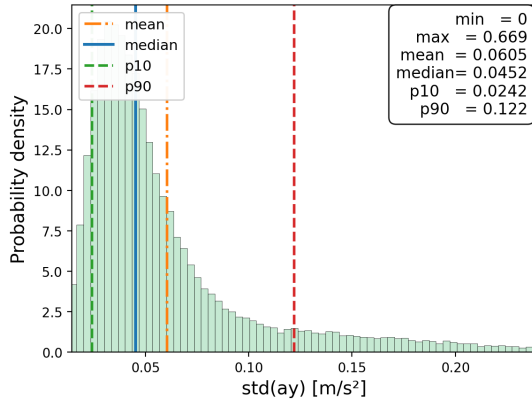


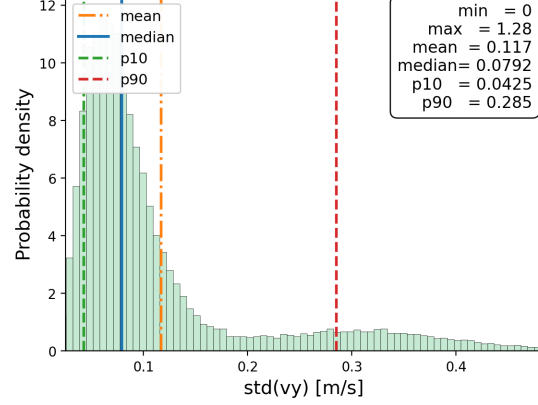
Figure B.2.: Class-wise trajectory-level variability distributions of the main kinematic variables.



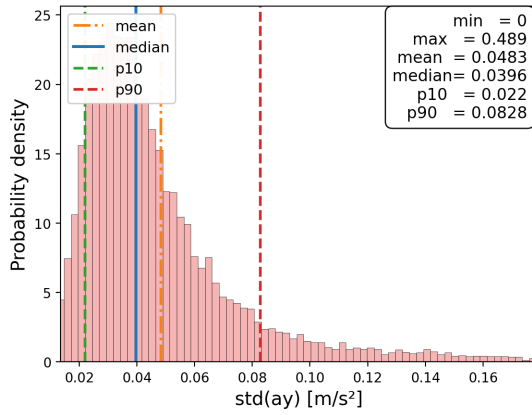
## B. Class-Wise Distribution Plots



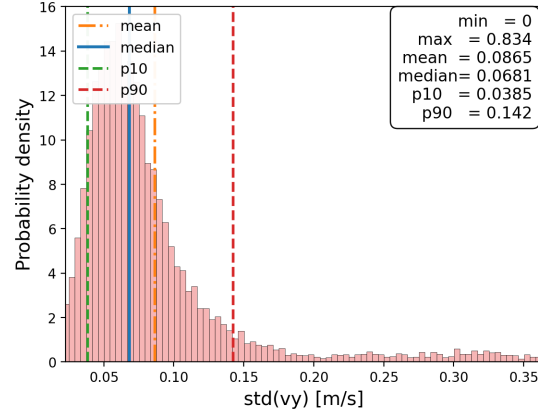
(e) Car: variability of lateral acceleration.



(f) Car: variability of lateral velocity.



(g) Truck: variability of lateral acceleration.



(h) Truck: variability of lateral velocity.

Figure B.2.: Class-wise trajectory-level variability distributions of the main kinematic variables (continued).

## C. Complete Window-Level Observation Features

This appendix lists the complete set of window-level features used to construct the observation vector of the proposed model. The features are grouped according to their role in the representation.

Table C.1.: Window-level ego-motion features.

| Window feature            | Aggregation rule                                                         |
|---------------------------|--------------------------------------------------------------------------|
| <code>vx_last</code>      | Most recent longitudinal velocity value in the window                    |
| <code>vx_slope</code>     | Temporal slope of longitudinal velocity over the window                  |
| <code>ax_last</code>      | Most recent longitudinal acceleration value in the window                |
| <code>vy_last</code>      | Most recent lateral velocity value in the window                         |
| <code>vy_slope</code>     | Temporal slope of lateral velocity over the window                       |
| <code>ay_last</code>      | Most recent lateral acceleration value in the window                     |
| <code>ax_neg_frac</code>  | Fraction of frames in the window with negative longitudinal acceleration |
| <code>ax_pos_frac</code>  | Fraction of frames in the window with positive longitudinal acceleration |
| <code>ax_zero_frac</code> | Fraction of frames in the window with zero longitudinal acceleration     |
| <code>ay_neg_frac</code>  | Fraction of frames in the window with negative lateral acceleration      |
| <code>ay_pos_frac</code>  | Fraction of frames in the window with positive lateral acceleration      |
| <code>ay_zero_frac</code> | Fraction of frames in the window with zero lateral acceleration          |
| <code>jerk_x_p95</code>   | 95th percentile of longitudinal jerk within the window                   |
| <code>jerk_y_p95</code>   | 95th percentile of lateral jerk within the window                        |

Table C.2.: Window-level lane-change and lane-geometry features.

| Window feature                 | Aggregation rule                                                                             |
|--------------------------------|----------------------------------------------------------------------------------------------|
| <code>lc_left_present</code>   | Binary indicator showing whether a left lane-change event occurs anywhere inside the window  |
| <code>lc_right_present</code>  | Binary indicator showing whether a right lane-change event occurs anywhere inside the window |
| <code>d_left_lane_last</code>  | Most recent distance to the left lane boundary in the window                                 |
| <code>d_left_lane_min</code>   | Minimum distance to the left lane boundary within the window                                 |
| <code>d_right_lane_last</code> | Most recent distance to the right lane boundary in the window                                |
| <code>d_right_lane_min</code>  | Minimum distance to the right lane boundary within the window                                |

### C. Complete Window-Level Observation Features

Table C.3.: Window-level front-risk features.

| Window feature  | Aggregation rule                                        |
|-----------------|---------------------------------------------------------|
| front_thw_last  | Most recent valid THW value within the window           |
| front_thw_vfrac | Fraction of frames in the window for which THW is valid |
| front_ttc_min   | Minimum valid TTC value within the window               |
| front_ttc_vfrac | Fraction of frames in the window for which TTC is valid |

Table C.4.: Window-level features related to the front neighbor.

| Window feature    | Aggregation rule                                                                   |
|-------------------|------------------------------------------------------------------------------------|
| front_dx_min      | Minimum relative longitudinal distance to the front vehicle within the window      |
| front_dvx_slope   | Temporal slope of relative longitudinal velocity with respect to the front vehicle |
| front_exists_frac | Fraction of frames in the window in which a front vehicle is present               |

Table C.5.: Window-level features related to the side neighbors.

| Window feature         | Aggregation rule                                                                         |
|------------------------|------------------------------------------------------------------------------------------|
| left_side_dvx_last     | Most recent relative longitudinal velocity with respect to the left-side neighbor        |
| left_side_dvx_slope    | Temporal slope of relative longitudinal velocity with respect to the left-side neighbor  |
| left_side_dy_last      | Most recent relative lateral position with respect to the left-side neighbor             |
| left_side_exists_frac  | Fraction of frames in the window in which a left-side neighbor is present                |
| right_side_dvx_last    | Most recent relative longitudinal velocity with respect to the right-side neighbor       |
| right_side_dvx_slope   | Temporal slope of relative longitudinal velocity with respect to the right-side neighbor |
| right_side_dy_last     | Most recent relative lateral position with respect to the right-side neighbor            |
| right_side_exists_frac | Fraction of frames in the window in which a right-side neighbor is present               |

### C. Complete Window-Level Observation Features

Table C.6.: Window-level features related to the adjacent front neighbors.

| Window feature          | Aggregation rule                                                                          |
|-------------------------|-------------------------------------------------------------------------------------------|
| left_front_dx_last      | Most recent relative longitudinal distance to the left-front neighbor                     |
| left_front_dvx_slope    | Temporal slope of relative longitudinal velocity with respect to the left-front neighbor  |
| left_front_dy_last      | Most recent relative lateral position with respect to the left-front neighbor             |
| left_front_exists_frac  | Fraction of frames in the window in which a left-front neighbor is present                |
| right_front_dx_last     | Most recent relative longitudinal distance to the right-front neighbor                    |
| right_front_dvx_slope   | Temporal slope of relative longitudinal velocity with respect to the right-front neighbor |
| right_front_dy_last     | Most recent relative lateral position with respect to the right-front neighbor            |
| right_front_exists_frac | Fraction of frames in the window in which a right-front neighbor is present               |

## D. Semantic Mapping Rules

This appendix documents the rule-based semantic mapping used to assign interpretable labels to the latent states of the selected S2A4 hierarchical model.

### D.1. Threshold Values

Table D.1.: Threshold values used by the semantic mapping rules.

| Parameter                        | Value |
|----------------------------------|-------|
| lc_flag_thresh                   | 0.5   |
| lateral_velocity_threshold       | 0.1   |
| lateral_acceleration_threshold   | 0.1   |
| lateral_velocity_slope_threshold | 0.003 |
| ay_zero_frac_threshold           | 0.5   |
| brake_ax_neg_frac                | 0.90  |
| brake_ax_last                    | -0.15 |
| tactical_brake_ax_last           | -0.25 |
| tactical_brake_vx_slope          | -0.01 |
| tactical_brake_thw               | 2.0   |
| tactical_brake_ttc               | 70.0  |
| interaction_thw_tight            | 2.0   |
| interaction_ttc_tight            | 60.0  |
| strong_acc_front_exists_min      | 0.50  |
| strong_acc_ax_pos_frac           | 0.90  |
| strong_acc_ax_last               | 0.15  |
| constrained_acc_ax_pos_frac      | 0.65  |
| constrained_acc_ax_last          | 0.08  |
| constrained_acc_front_exists     | 0.80  |
| free_flow_front_exists_max       | 0.50  |
| constrained_follow_front_exists  | 0.80  |
| constrained_follow_vx_last_max   | 21.0  |
| constrained_follow_jerk_min      | 0.75  |
| stable_follow_front_exists       | 0.80  |
| stable_follow_vx_slope_abs_max   | 0.008 |
| stable_follow_ax_last_abs_max    | 0.20  |

## **D.2. Priority Order**

When more than one rule condition is satisfied for the same window, the final semantic label is assigned according to the following precedence order:

1. lane change,
2. braking, followed by the split into tactical brake versus mild brake,
3. strong acceleration,
4. constrained acceleration,
5. constrained following,
6. stable following,
7. free flow,
8. unknown.

This priority order ensures that rarer or more distinctive maneuvers, such as lane changes and braking events, are not overridden by broader longitudinal behavior rules.

## **D.3. Composite Definitions**

The condition `tactical_brake` is true if at least one strong braking cue is present:

```
ax_last <= tactical_brake_ax_last,  
vx_slope <= tactical_brake_vx_slope,  
front_thw_last <= tactical_brake_thw when the feature is available, or  
front_ttc_min <= tactical_brake_ttc when the feature is available.
```

The condition `tight_interaction` is true if  
`front_exists_frac >= stable_follow_front_exists`  
and at least one interaction-risk cue is active:  
`front_thw_last <= interaction_thw_tight` when available, or  
`front_ttc_min <= interaction_ttc_tight` when available.

### **Lane-change rule**

The lane-change rule has the highest priority. A window is labeled as `lane_change` if either the left or right lane-change indicator exceeds the threshold. It can also be triggered by a sustained lateral-motion pattern defined through lateral velocity, lateral acceleration, lateral velocity slope, and the fraction of near-zero lateral acceleration within the window.

### **Braking rules**

Braking is detected when negative longitudinal acceleration dominates most of the window, the final longitudinal acceleration is sufficiently negative, and a front vehicle is present for a large fraction of the window. Once braking is detected, the window is split into either `mild_brake` or `tactical_brake`. If any tactical braking cue is present, the label becomes `tactical_brake`; otherwise it is assigned as `mild_brake`.

### **Acceleration rules**

Two acceleration-related labels are defined. A window is labeled `strong_acceleration` when positive longitudinal acceleration dominates the window, the final acceleration is clearly positive, and either a front vehicle is present or the situation is already classified as a tight interaction. A window is labeled `constrained_acceleration` when positive acceleration is present under stronger leader presence, indicating that the maneuver is still shaped by traffic context.

### **Following rules**

Two following-related labels are defined. A window is labeled `constrained_following` when a front vehicle is present for most of the window and either the motion shows low-speed, high-jerk behavior or the interaction is tight. This rule is intended to capture pressured or corrective following behavior. A window is labeled `stable_following` when a front vehicle is present, the speed slope and longitudinal acceleration remain close to zero, and the interaction is not tight. This rule therefore represents smoother and more stable car-following behavior.

### **Free-flow rule**

If none of the higher-priority maneuver rules fires and the fraction of front-vehicle presence remains below the free-flow threshold, the window is labeled `free_flow`. This captures low-interaction motion with little direct leader influence.

### **Fallback case**

If none of the semantic rules is satisfied, the window is assigned the fallback label `unknown`. This case corresponds to windows for which no lane-change, braking, acceleration, or following pattern is detected by the rule set.

## E. Emission Model Derivations

This appendix presents the theoretical derivations of the emission-model updates used in the proposed DBN. The derivation follows the standard EM formulation for latent-variable models [10], [73]. Exact implementation details and training-specific choices are described in the methodology chapter.

Let  $O_{1:T} = (O_1, \dots, O_T)$  denote the observed sequence, and let  $S_{1:T}$  and  $A_{1:T}$  denote the corresponding latent style and action state sequences. The model parameters are collected in  $\theta$ . Each observation at time step  $t$  is decomposed into a continuous part and a binary part:

$$O_t = (x_t, b_t) \quad (\text{E.1})$$

with

$$x_t \in \mathbb{R}^{D_c}, \quad (\text{E.2})$$

$$b_t \in \{0, 1\}^B \quad (\text{E.3})$$

In the E-step of the EM algorithm, the current parameter estimate  $\theta^{\text{old}}$  is used to compute posterior probabilities over the latent variables [8], [10], [13].

The main posterior marginal used in the emission updates is:

$$\gamma_t(s, a) = p(S_t = s, A_t = a \mid O_{1:T}, \theta^{\text{old}}) \quad (\text{E.4})$$

This quantity is the posterior responsibility assigned to the latent pair  $(s, a)$  at time step  $t$ . For each fixed  $t$ , the values  $\{\gamma_t(s, a)\}_{s,a}$  form a normalized joint posterior distribution:

$$\sum_{s,a} \gamma_t(s, a) = 1 \quad (\text{E.5})$$

The transition-update derivations also use the posterior transition marginals:

$$\xi_t^S(s', s) = p(S_{t-1} = s', S_t = s \mid O_{1:T}, \theta^{\text{old}}), \quad t = 2, \dots, T, \quad (\text{E.6})$$

$$\xi_t^A(s, a', a) = p(S_t = s, A_{t-1} = a', A_t = a \mid O_{1:T}, \theta^{\text{old}}), \quad t = 2, \dots, T \quad (\text{E.7})$$

### E.0.1. EM Objective

The goal of learning is to maximize the observed-data log-likelihood:

$$\log p(O_{1:T} \mid \theta) = \log \sum_{S_{1:T}, A_{1:T}} p(O_{1:T}, S_{1:T}, A_{1:T} \mid \theta) \quad (\text{E.8})$$



### E. Emission Model Derivations

Direct maximization of (E.8) is difficult because the latent state sequences are not observed and must be summed out [10], [73].

In the EM algorithm, this difficulty is addressed by introducing an auxiliary distribution  $q(S_{1:T}, A_{1:T})$  over the latent trajectories, with:

$$q(S_{1:T}, A_{1:T}) \geq 0, \quad \sum_{S_{1:T}, A_{1:T}} q(S_{1:T}, A_{1:T}) = 1 \quad (\text{E.9})$$

Using this distribution, the log-likelihood can be rewritten as:

$$\log p(O_{1:T} | \theta) = \log \sum_{S_{1:T}, A_{1:T}} q(S_{1:T}, A_{1:T}) \frac{p(O_{1:T}, S_{1:T}, A_{1:T} | \theta)}{q(S_{1:T}, A_{1:T})} \quad (\text{E.10})$$

Jensen's inequality states that for a convex function  $f$ ,

$$f\left(\sum_i q_i x_i\right) \leq \sum_i q_i f(x_i), \quad q_i \geq 0, \quad \sum_i q_i = 1. \quad (\text{E.11})$$

Equivalently, if  $f$  is concave, the inequality direction is reversed. Since  $\log(\cdot)$  is concave, applying Jensen's inequality to (E.10) gives:

$$\log p(O_{1:T} | \theta) \geq \sum_{S_{1:T}, A_{1:T}} q(S_{1:T}, A_{1:T}) \log \frac{p(O_{1:T}, S_{1:T}, A_{1:T} | \theta)}{q(S_{1:T}, A_{1:T})} \quad (\text{E.12})$$

Expanding the logarithm yields:

$$\begin{aligned} \log p(O_{1:T} | \theta) &\geq \sum_{S_{1:T}, A_{1:T}} q(S_{1:T}, A_{1:T}) \log p(O_{1:T}, S_{1:T}, A_{1:T} | \theta) \\ &\quad - \sum_{S_{1:T}, A_{1:T}} q(S_{1:T}, A_{1:T}) \log q(S_{1:T}, A_{1:T}) \end{aligned} \quad (\text{E.13})$$

The second term is the entropy of  $q$ . For fixed  $q$ , it does not depend on  $\theta$ . Therefore, in the M-step it is sufficient to maximize only the first term [10], [73].

In EM, the auxiliary distribution is chosen as the posterior under the current parameter estimate:

$$q(S_{1:T}, A_{1:T}) = p(S_{1:T}, A_{1:T} | O_{1:T}, \theta^{\text{old}}) \quad (\text{E.14})$$

Substituting this choice into the lower bound gives the auxiliary function:

$$Q(\theta, \theta^{\text{old}}) = \mathbb{E}_{p(S_{1:T}, A_{1:T} | O_{1:T}, \theta^{\text{old}})} [\log p(O_{1:T}, S_{1:T}, A_{1:T} | \theta)] \quad (\text{E.15})$$

Thus, the E-step computes the posterior over the latent trajectories under  $\theta^{\text{old}}$ , and the M-step updates  $\theta$  by maximizing the expected complete-data log-likelihood  $Q(\theta, \theta^{\text{old}})$  [8], [10], [13], [73].

### E. Emission Model Derivations

Using the factorization of the proposed DBN, the complete-data distribution can be written as:

$$\begin{aligned}
 p(O_{1:T}, S_{1:T}, A_{1:T} \mid \theta) &= p(S_1 \mid \theta) p(A_1 \mid S_1; \theta) \prod_{t=2}^T p(S_t \mid S_{t-1}; \theta) \\
 &\times \prod_{t=2}^T p(A_t \mid A_{t-1}, S_t; \theta) \prod_{t=1}^T p(O_t \mid S_t, A_t; \theta)
 \end{aligned} \tag{E.16}$$

Taking the logarithm converts the product into a sum:

$$\begin{aligned}
 \log p(O_{1:T}, S_{1:T}, A_{1:T} \mid \theta) &= \log p(S_1 \mid \theta) + \log p(A_1 \mid S_1; \theta) \\
 &+ \sum_{t=2}^T \log p(S_t \mid S_{t-1}; \theta) + \sum_{t=2}^T \log p(A_t \mid A_{t-1}, S_t; \theta) \\
 &+ \sum_{t=1}^T \log p(O_t \mid S_t, A_t; \theta)
 \end{aligned} \tag{E.17}$$

Substituting (E.17) into the auxiliary function (E.15) gives:

$$\begin{aligned}
 Q(\theta, \theta^{\text{old}}) &= \mathbb{E}_{p(S_{1:T}, A_{1:T} \mid O_{1:T}, \theta^{\text{old}})} [\log p(O_{1:T}, S_{1:T}, A_{1:T} \mid \theta)] \\
 &= \mathbb{E}_{p(S_{1:T}, A_{1:T} \mid O_{1:T}, \theta^{\text{old}})} \left[ \log p(S_1 \mid \theta) + \log p(A_1 \mid S_1; \theta) \right. \\
 &\quad \left. + \sum_{t=2}^T \log p(S_t \mid S_{t-1}; \theta) + \sum_{t=2}^T \log p(A_t \mid A_{t-1}, S_t; \theta) \right. \\
 &\quad \left. + \sum_{t=1}^T \log p(O_t \mid S_t, A_t; \theta) \right]
 \end{aligned} \tag{E.18}$$

By linearity of expectation, this becomes

$$\begin{aligned}
 Q(\theta, \theta^{\text{old}}) &= \mathbb{E}[\log p(S_1 \mid \theta)] + \mathbb{E}[\log p(A_1 \mid S_1; \theta)] \\
 &+ \sum_{t=2}^T \mathbb{E}[\log p(S_t \mid S_{t-1}; \theta)] + \sum_{t=2}^T \mathbb{E}[\log p(A_t \mid A_{t-1}, S_t; \theta)] \\
 &+ \sum_{t=1}^T \mathbb{E}[\log p(O_t \mid S_t, A_t; \theta)],
 \end{aligned} \tag{E.19}$$

where all expectations are taken with respect to  $p(S_{1:T}, A_{1:T} \mid O_{1:T}, \theta^{\text{old}})$  [10], [73].

These expectations can be written in terms of the posterior marginals from the E-step. For the initial style term, define:

$$\gamma_1^S(s) = p(S_1 = s \mid O_{1:T}, \theta^{\text{old}}) = \sum_a \gamma_1(s, a) \tag{E.20}$$

Using (E.4), (E.6), and (E.7), the auxiliary function becomes:

$$\begin{aligned}
Q(\theta, \theta^{\text{old}}) = & \sum_s \gamma_1^S(s) \log p(S_1 = s \mid \theta) + \sum_{s,a} \gamma_1(s,a) \log p(A_1 = a \mid S_1 = s; \theta) \\
& + \sum_{t=2}^T \sum_{s',s} \xi_t^S(s',s) \log p(S_t = s \mid S_{t-1} = s'; \theta) \\
& + \sum_{t=2}^T \sum_{s,a',a} \xi_t^A(s,a',a) \log p(A_t = a \mid A_{t-1} = a', S_t = s; \theta) \\
& + \sum_{t=1}^T \sum_{s,a} \gamma_t(s,a) \log p(O_t \mid S_t = s, A_t = a; \theta)
\end{aligned} \tag{E.21}$$

This is the expected complete-data log-likelihood written explicitly in terms of the posterior quantities computed in the E-step [8], [13], [73].

For the emission-parameter derivations in this appendix, only the last term in (E.21) is required. Therefore, the emission contribution is:

$$Q_{\text{emit}}(\theta, \theta^{\text{old}}) = \sum_{t=1}^T \sum_{s,a} \gamma_t(s,a) \log p(O_t \mid S_t = s, A_t = a; \theta) \tag{E.22}$$

The following sections specialize (E.22) to the hierarchical and PoE emission formulations considered in this work.

## E.1. Hierarchical Emission Model

For the hierarchical emission formulation introduced in Section 4.4, the emission model is parameterized directly for each joint latent-state pair  $(s, a)$ . The corresponding M-step therefore follows the standard EM pattern for latent-variable models, where each parameter block is updated using posterior weights [73], [74].

Using the decomposition  $O_t = (x_t, b_t)$ , the emission factorizes as:

$$p(O_t \mid s, a; \theta_{s,a}) = p(x_t \mid s, a; \theta_{s,a}^{(x)}) p(b_t \mid s, a; \theta_{s,a}^{(b)}), \tag{E.23}$$

with

$$\theta_{s,a}^{(x)} = (\mu_{s,a}, \Sigma_{s,a}), \quad \theta_{s,a}^{(b)} = \{p_{s,a,j}\}_{j=1}^B \tag{E.24}$$

For the continuous features, a Gaussian distribution with diagonal covariance is used:

$$p(x_t \mid s, a) = \mathcal{N}(x_t \mid \mu_{s,a}, \Sigma_{s,a}), \quad \Sigma_{s,a} = \text{diag}(\sigma_{s,a,1}^2, \dots, \sigma_{s,a,D_c}^2) \tag{E.25}$$

For the binary features, a factorized Bernoulli distribution is used:

$$p(b_t \mid s, a) = \prod_{j=1}^B (p_{s,a,j})^{b_t^{(j)}} (1 - p_{s,a,j})^{1-b_t^{(j)}} \tag{E.26}$$

Hence, for a fixed joint state pair  $(s, a)$ , the log-emission probability becomes:

$$\begin{aligned} \log p(O_t | s, a; \theta_{s,a}) &= \log p(x_t | s, a) + \log p(b_t | s, a) \\ &= \log \mathcal{N}(x_t | \mu_{s,a}, \Sigma_{s,a}) + \sum_{j=1}^B \left[ b_t^{(j)} \log p_{s,a,j} + (1 - b_t^{(j)}) \log(1 - p_{s,a,j}) \right] \end{aligned} \quad (\text{E.27})$$

Substituting (E.27) into the emission contribution of the auxiliary function, defined in (E.22), gives:

$$\begin{aligned} Q_{\text{emit}}(\theta, \theta^{\text{old}}) &= \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \log \mathcal{N}(x_t | \mu_{s,a}, \Sigma_{s,a}) \\ &\quad + \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \sum_{j=1}^B \left[ b_t^{(j)} \log p_{s,a,j} + (1 - b_t^{(j)}) \log(1 - p_{s,a,j}) \right] \end{aligned} \quad (\text{E.28})$$

For convenience, define the effective posterior count of the joint state pair  $(s, a)$  as:

$$N_{s,a} = \sum_{t=1}^T \gamma_t(s, a) \quad (\text{E.29})$$

### E.1.1. Parameter Updates

The emission parameters are obtained by maximizing (E.28) with respect to the parameters of each joint state pair  $(s, a)$ . Because the parameters for one joint state do not appear in the emission terms of any other joint state, the optimization separates across  $(s, a)$  and yields closed-form updates [73], [74].

**Gaussian mean.** For fixed covariance, the part of the objective that depends on  $\mu_{s,a}$  is:

$$-\frac{1}{2} \sum_{t=1}^T \gamma_t(s, a) (x_t - \mu_{s,a})^\top \Sigma_{s,a}^{-1} (x_t - \mu_{s,a}) \quad (\text{E.30})$$

Differentiating with respect to  $\mu_{s,a}$  and setting the derivative to zero yields:

$$\mu_{s,a} = \frac{\sum_{t=1}^T \gamma_t(s, a) x_t}{\sum_{t=1}^T \gamma_t(s, a)} = \frac{1}{N_{s,a}} \sum_{t=1}^T \gamma_t(s, a) x_t \quad (\text{E.31})$$

**Gaussian covariance.** Assuming a diagonal covariance matrix, each variance parameter can be optimized independently. For dimension  $d$ , the relevant term is

$$-\frac{1}{2} \sum_{t=1}^T \gamma_t(s, a) \left[ \log \sigma_{s,a,d}^2 + \frac{(x_t^{(d)} - \mu_{s,a}^{(d)})^2}{\sigma_{s,a,d}^2} \right]. \quad (\text{E.32})$$

### E. Emission Model Derivations

Differentiating with respect to  $\sigma_{s,a,d}^2$  and setting the result to zero gives:

$$\sigma_{s,a,d}^2 = \frac{1}{N_{s,a}} \sum_{t=1}^T \gamma_t(s,a) (x_t^{(d)} - \mu_{s,a}^{(d)})^2 \quad (\text{E.33})$$

Hence,

$$\Sigma_{s,a} = \text{diag}(\sigma_{s,a,1}^2, \dots, \sigma_{s,a,D_c}^2). \quad (\text{E.34})$$

**Bernoulli parameters.** For each binary feature  $j$ , the Bernoulli part of the objective is

$$\sum_{t=1}^T \gamma_t(s,a) \left[ b_t^{(j)} \log p_{s,a,j} + (1 - b_t^{(j)}) \log(1 - p_{s,a,j}) \right]. \quad (\text{E.35})$$

Differentiating with respect to  $p_{s,a,j}$  and setting this derivative to zero gives:

$$p_{s,a,j} = \frac{\sum_{t=1}^T \gamma_t(s,a) b_t^{(j)}}{\sum_{t=1}^T \gamma_t(s,a)} = \frac{1}{N_{s,a}} \sum_{t=1}^T \gamma_t(s,a) b_t^{(j)} \quad (\text{E.36})$$

Thus, the hierarchical emission model admits closed-form M-step updates, where each parameter is estimated as a posterior-weighted maximum-likelihood quantity for the corresponding joint latent-state pair [73], [74].

## E.2. Product-of-Experts Emission Model

For the PoE emission formulation introduced in Section 4.4, the emission distribution for the joint latent-state pair  $(s,a)$  is constructed by multiplying a style-dependent expert and an action-dependent expert, followed by normalization:

$$p(O_t \mid S_t = s, A_t = a) = \frac{p_s(O_t \mid s) p_a(O_t \mid a)}{Z_{s,a}} \quad (\text{E.37})$$

This is the standard product-of-experts construction, in which several simpler expert models are combined multiplicatively and then renormalized to obtain a valid probability distribution [66].

Here,  $Z_{s,a}$  is the normalization constant that ensures that (E.37) defines a valid probability distribution. Using the continuous-binary decomposition of the observation, it can be written more explicitly as:

$$Z_{s,a} = \sum_{b \in \{0,1\}^B} \int p_s(x, b \mid s) p_a(x, b \mid a) dx \quad (\text{E.38})$$

Using the decomposition  $O_t = (x_t, b_t)$ , both experts factorize into continuous and binary parts:

$$p_s(O_t \mid s) = p_s(x_t \mid s) p_s(b_t \mid s), \quad (\text{E.39})$$

$$p_a(O_t \mid a) = p_a(x_t \mid a) p_a(b_t \mid a) \quad (\text{E.40})$$

### E. Emission Model Derivations

The corresponding parameter sets are written as:

$$\theta_s = (\mu_s, \Sigma_s, \{p_j^s\}_{j=1}^B), \quad \theta_a = (\mu_a, \Sigma_a, \{p_j^a\}_{j=1}^B) \quad (\text{E.41})$$

For the continuous features, Gaussian experts with diagonal covariance are used:

$$p_s(x_t | s) = \mathcal{N}(x_t | \mu_s, \Sigma_s), \quad \Sigma_s = \text{diag}(\sigma_{s,1}^2, \dots, \sigma_{s,D_c}^2), \quad (\text{E.42})$$

$$p_a(x_t | a) = \mathcal{N}(x_t | \mu_a, \Sigma_a), \quad \Sigma_a = \text{diag}(\sigma_{a,1}^2, \dots, \sigma_{a,D_c}^2) \quad (\text{E.43})$$

For the binary features, factorized Bernoulli experts are used:

$$p_s(b_t | s) = \prod_{j=1}^B (p_j^s)^{b_t^{(j)}} (1 - p_j^s)^{1-b_t^{(j)}}, \quad (\text{E.44})$$

$$p_a(b_t | a) = \prod_{j=1}^B (p_j^a)^{b_t^{(j)}} (1 - p_j^a)^{1-b_t^{(j)}} \quad (\text{E.45})$$

Because the experts factorize into continuous and binary parts, the normalization constant also factorizes:

$$\begin{aligned} Z_{s,a} &= \sum_{b \in \{0,1\}^B} \int p_s(x, b | s) p_a(x, b | a) dx \\ &= \left( \int p_s(x | s) p_a(x | a) dx \right) \left( \sum_{b \in \{0,1\}^B} p_s(b | s) p_a(b | a) \right) \\ &= Z_{s,a}^{(x)} Z_{s,a}^{(b)} \end{aligned} \quad (\text{E.46})$$

The continuous normalization term is therefore:

$$Z_{s,a}^{(x)} = \int \mathcal{N}(x | \mu_s, \Sigma_s) \mathcal{N}(x | \mu_a, \Sigma_a) dx \quad (\text{E.47})$$

Using the standard Gaussian product identity, this becomes

$$Z_{s,a}^{(x)} = \mathcal{N}(\mu_s | \mu_a, \Sigma_s + \Sigma_a) \quad (\text{E.48})$$

Hence,

$$\begin{aligned} \log Z_{s,a}^{(x)} &= -\frac{D_c}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma_s + \Sigma_a| \\ &\quad - \frac{1}{2} (\mu_s - \mu_a)^\top (\Sigma_s + \Sigma_a)^{-1} (\mu_s - \mu_a) \end{aligned} \quad (\text{E.49})$$

For the binary part,

$$Z_{s,a}^{(b)} = \sum_{b \in \{0,1\}^B} \prod_{j=1}^B (p_j^s)^{b^{(j)}} (1 - p_j^s)^{1-b^{(j)}} (p_j^a)^{b^{(j)}} (1 - p_j^a)^{1-b^{(j)}} \quad (\text{E.50})$$

Since the Bernoulli experts factorize across dimensions, the sum also factorizes:

$$Z_{s,a}^{(b)} = \prod_{j=1}^B \left[ p_j^s p_j^a + (1 - p_j^s)(1 - p_j^a) \right] \quad (\text{E.51})$$

Taking the logarithm yields

$$\log Z_{s,a}^{(b)} = \sum_{j=1}^B \log \left[ p_j^s p_j^a + (1 - p_j^s)(1 - p_j^a) \right] \quad (\text{E.52})$$

Combining the two experts and the normalization term, the log-emission probability becomes:

$$\begin{aligned} \log p(O_t | s, a) &= \log p_s(x_t | s) + \log p_a(x_t | a) + \log p_s(b_t | s) + \log p_a(b_t | a) \\ &\quad - \log Z_{s,a}^{(x)} - \log Z_{s,a}^{(b)} \end{aligned} \quad (\text{E.53})$$

Substituting (E.53) into the emission contribution of the auxiliary function, defined in (E.22), gives:

$$\begin{aligned} Q_{\text{emit}}(\theta, \theta^{\text{old}}) &= \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \log \mathcal{N}(x_t | \mu_s, \Sigma_s) \\ &\quad + \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \log \mathcal{N}(x_t | \mu_a, \Sigma_a) \\ &\quad + \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \sum_{j=1}^B \left[ b_t^{(j)} \log p_j^s + (1 - b_t^{(j)}) \log(1 - p_j^s) \right] \\ &\quad + \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \sum_{j=1}^B \left[ b_t^{(j)} \log p_j^a + (1 - b_t^{(j)}) \log(1 - p_j^a) \right] \\ &\quad - \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \log Z_{s,a}^{(x)} - \sum_{t=1}^T \sum_{s,a} \gamma_t(s, a) \log Z_{s,a}^{(b)} \end{aligned} \quad (\text{E.54})$$

### E.2.1. Implication for the M-Step

In contrast to the hierarchical emission model, the PoE objective does not separate into independent parameter blocks for each joint state pair. The reason is that the normalization terms  $\log Z_{s,a}^{(x)}$  and  $\log Z_{s,a}^{(b)}$  depend on the same expert parameters that also appear in the data-fit terms. As a result, the style and action experts are coupled through the partition function [66].

More specifically, any update of a style-expert parameter affects not only the style likelihood term, but also all normalization terms involving that style state across the action experts. The same holds symmetrically for the action-expert parameters. Therefore,

the emission M-step no longer reduces to a set of independent closed-form weighted maximum-likelihood updates.

At the theoretical level, this means that the PoE emission parameters are optimized by increasing  $Q_{\text{emit}}(\theta, \theta^{\text{old}})$  with respect to the expert parameters using a generalized EM step, rather than by solving closed-form maximization equations for each parameter separately [73], [75].

### E.2.2. Gradient Form

Let  $\vartheta_s$  denote a parameter of the style expert and  $\vartheta_a$  a parameter of the action expert. Differentiating (E.54) gives the generic gradient structure:

$$\frac{\partial Q_{\text{emit}}}{\partial \vartheta_s} = \sum_{t=1}^T \sum_a \gamma_t(s, a) \left[ \frac{\partial}{\partial \vartheta_s} \log p_s(O_t | s) - \frac{\partial}{\partial \vartheta_s} \log Z_{s,a} \right], \quad (\text{E.55})$$

and similarly

$$\frac{\partial Q_{\text{emit}}}{\partial \vartheta_a} = \sum_{t=1}^T \sum_s \gamma_t(s, a) \left[ \frac{\partial}{\partial \vartheta_a} \log p_a(O_t | a) - \frac{\partial}{\partial \vartheta_a} \log Z_{s,a} \right] \quad (\text{E.56})$$

These expressions show that each gradient contains two competing contributions: a data-fit term from the expert likelihood and a normalization term arising from the partition function. The second term is specific to the PoE formulation and is the reason why the emission M-step does not admit the same closed-form updates as in the hierarchical case [66].



# Declaration of Compliance

I hereby declare to have written this work independently and to have respected in its preparation the relevant provisions, in particular those corresponding to the copyright protection of external materials. Whenever external materials (such as images, drawings, text passages) are used in this work, I declare that these materials are referenced accordingly (e. g. quote, source) and, whenever necessary, consent from the author to use such materials in my work has been obtained.

Signature:

Stuttgart, on the 15 April 2026